

Notas de Clase:

Aplicaciones de Machine Learning a través de R y Python, y Algunos Temas Selectos para el Análisis de Datos e Inteligencia Artificial

Benjamín Oliva ¹

Draft Julio 2026

¹`benjov@ciencias.unam.mx` y `https://github.com/benjov`

Este documento siempre está en proceso de mejora.
Comentarios, dudas, etc., siempre serán
bienvenidos...

Índice general

1. Aplicaciones de Machine Learning a través de R y Python	3
1.1. El concepto de esperanza condicional y Causalidad	3
1.1.1. Introducción a la esperanza condicional	4
1.1.2. Introducción a inferencia causal	6
1.1.3. Notación en el análisis de regresión	10
1.2. Aplicación de modelos de regresión lineal.	14
1.2.1. El estimador de Mínimos Cuadrados Ordinarios (MCO) y el análisis clásico de regresión	14
1.2.2. Ejemplo: Regresión lineal múltiple con datos de publi- cidad en medios de comunicación.	26
1.2.3. Modelos no lineales	27
1.2.4. Introducción al Aprendizaje Estadístico	32
1.2.5. Método de regresiones restringidas (Shrinkage methods).	35
1.3. Aplicación de modelos clasificación mediante clustering.	38
1.3.1. Análisis de Componentes Principales	38
1.3.2. Análisis Factorial	40
1.3.3. Clúster	40
1.3.4. Método de K-means con datos de epidemiológicos.	46
1.3.5. Agrupamiento de datos de series temporales	46
1.3.6. Método de K-means para series de tiempo.	53
1.4. Aplicación de modelos de regresión de respuesta binaria	54
1.4.1. Métodos de estimación basados en verosimilitud	54
1.4.2. Estimación de modelos no lineales	58
1.4.3. Modelos de respuesta multinomial y ordenada	62
1.4.4. Regresión logística para el estudio de violencia.	69
1.4.5. Regresión logística con información de videojuegos.	69

2. Temas Selectos para el Análisis de Datos	79
2.1. Aplicación de modelos de análisis de texto.	79
2.1.1. Elementos de Procesamiento de Lenguaje y Minado de Texto	79
2.1.2. Ejemplos y aplicaciones de minado de texto.	101
2.1.3. Implementación de modelo de lenguaje natural con N-Gramas.	101
2.1.4. Clasificación de texto con Naive Bayes.	101
2.1.5. Clasificación de texto con regresión logística.	102
2.2. Modelos de árboles de decisión y bosques aleatorios.	102
2.3. Modelos de redes neuronales.	106
2.3.1. Introducción y Motivación	106
2.3.2. Algunos conceptos relevantes	107
2.3.3. Preprocesamiento de datos	110
2.3.4. Entrenamiento del modelo	112
2.3.5. Ejemplo de Redes Neuronales	112
2.4. Aplicaciones de modelos de lenguaje natural con bases vecto- riales (word embeddings).	113
2.4.1. Distancias y similitud entre vectores	116
2.4.2. De palabras a oraciones: embeddings de token vs. de oración	117
2.4.3. De embeddings a recuperación de información: intro- ducción a RAG	118
2.5. Introducción a los LLMs y aplicaciones basadas en LLMs (Prompt Engineering).	122
2.5.1. ¿Cómo funciona un LLM? Breve introducción a la ar- quitectura Transformer	122
2.5.2. Guideline for Prompt engineering	123
2.5.3. Diseño de prompts	126
2.5.4. Agentes y uso de herramientas (tool use)	130
Bibliografía	133

Índice de figuras

1.1. Ejemplo 1 de un DAG	6
1.2. Ejemplo 2 de un DAG, confusor	7
1.3. Ejemplo 4 de un DAG, estimador basado en variables instru- mentales	8
1.4. Ejemplo 3 de un DAG, ganancias de la educación	8
1.5. Relación entre belleza y talento, retomado de [Cun21].	10
1.6. ¿Qué relación hay entre años de educación y el salario?, retoma- do de: https://wol.iza.org/articles/using-linear-reg- ression-to-establish-empirical-relationships/lang/es	11
1.7. Ilustración de la proyección de y en x_1 y x_2 y el significado del error e , retomado de Greene (2012, p. 31) [Gre12]	14
1.8. Ilustración del hiperplano generado de regresar X_1 y X_2 en Y , retomado de Hastie, Tibshirani y Friedman (2009, pp. 45) [HTF09]	15
1.9. Ilustración de la proyección de y en x_1 y x_2 y el significado del error e , retomado de Hastie, Tibshirani y Friedman (2009, pp. 46) [HTF09]	15
1.10. Funciones exponenciales, Retomado de Larsen y Marx (2012, p. 532) [LM12]	28
1.11. Funciones logarítmicas, Retomado de Larsen y Marx (2012, p. 536) [LM12]	29
1.12. Funciones logísticas, Retomado de Larsen y Marx (2012, p. 540) [LM12]	30
1.13. Funciones polinomiales, retomado de https://discdown.org /flexregression/smoothreg.html	32
1.14. División del conjunto de datos, retomado de Hastie, Tibshirani, y Friedman (2009, p. 222) [HTF09]	35

1.15. US Arrest Hierarchical Clustering, retomado de Chang (2020, p 216) [Cha20]	42
1.16. K-Means algorithm, retomado de Ng (2023, p 146) [Ng23] . . .	44
1.17. Agglomerative Clustering Algorithm and Dendrogram, retomado de Chang (2020, p 219) [Cha20]	45
1.18. División del conjunto de datos, retomado de Hastie, Tibshirani, y Friedman (2009, p. 222) [HTF09]	68
1.19. Probabilidades en un modelo de respuesta ordenada, considerando $J = 4$ (retomado de Greene (2012, 788) [Gre12])	72
1.20. En la tabla de datos ubicamos 417 tags distintas	76
2.1. Some simple regex searches, retomado de Jurafsky y Martin (2023, 3)	81
2.2. The use of the brackets <code>[]</code> to specify a disjunction of characters, retomado de Jurafsky y Martin (2023, 3)	81
2.3. The use of the brackets <code>[]</code> plus the dash <code>-</code> to specify a range, retomado de Jurafsky y Martin (2023, 3)	82
2.4. The caret <code>^</code> for negation or just to mean <code>^</code> . See below re: the backslash for escaping the period, retomado de Jurafsky y Martin (2023, 3)	82
2.5. Ejemplo de Tokenización con el modelo de OpenAI <code>https://platform.openai.com/tokenizer</code>	83
2.6. Retomado de Jurafsky y Martin (2023)	87
2.7. La función sigmoide $\sigma(z) = \frac{1}{1+e^{-z}}$ toma un valor real y lo mapea al rango $(0, 1)$. Es casi lineal alrededor de 0, pero los valores atípicos se comprimen hacia 0 o 1. Retomado de Jurafsky y Martin (2023)	95
2.8. Un documento de prueba mini muestra que exhibe las características extraídas en el vector x . Retomado de Jurafsky y Martin (2023)	97
2.9. Ejemplo de regiones. Retomado de Hastie et al (2009, 305) . .	102
2.10. Ejemplo de árbol. Retomado de Hastie et al (2009, 304) . . .	103
2.11. Ejemplo de redes neuronales. Retomado de Shmueli et al (2019, 285) [Shm+19]	107
2.12. Ejemplo de la función de activación Sigmoide. Retomado de Jurafsky, et al (2023, Cap. 7, p.2) [JM23]	109
2.13. Ejemplo de la función de activación <i>tanh</i> . Retomado de Jurafsky, et al (2023, Cap. 7, p.4) [JM23]	109

2.14. Ilustración de una unidad neuronal. Retomado de Jurafsky, et al (2023, Cap. 7, p.3) [JM23]	111
2.15. Ilustración de una red neuronal con 1 capa oculta. Retomado de Jurafsky, et al (2023, Cap. 7, p.11) [JM23]	111
2.16. Retomado de https://learn.deeplearning.ai/courses/vector-databases-embeddings-applications/lesson/1/introduction	114
2.17. Retomado de https://learn.deeplearning.ai/courses/vector-databases-embeddings-applications/lesson/1/introduction	114
2.18. Retomado de https://learn.deeplearning.ai/courses/vector-databases-embeddings-applications/lesson/1/introduction	115
2.19. Retomado de Jurafsky, et al (2023, Cap. 6, p.6) [JM23]	116
2.20. Retomado de Jurafsky, et al (2023, Cap. 6, p.8) [JM23]	117
2.21. Similaridad de Coseno	118
2.22. Retomado de Jurafsky, et al (2023, Cap. 7, p.13) [JM23]	119
2.23. Retomado de Jurafsky, et al (2023, Cap. 7, p.23) [JM23]	120
2.24. Retomado de Jurafsky, et al (2023, Cap. 7, p.24) [JM23]	121
2.25. LLM	128

Índice de cuadros

1.1. Matriz de Confusión	69
------------------------------------	----

Introducción, motivación y alcance del documento

Queremos discutir los documentos de la bibliografía como: Abadie (2021) [Aba21], Adams (2020) [Ada20], Cameron y Trivedi (2005) [CT05]; Cunningham (2021) [Cun21]; Greene (2012) [Gre12], Hastie, Tibshirani y Friedman (2009) [HTF09], James, Witten, Hastie y Tibshirani (2013) [Jam+13], Jurařsky (2023) [JM23] y Wooldridge (2010) [Woo10]; entre muchos más que utilizemos en el curso.

Conocimientos previos

Módulos I, II, III y IV del Diplomado.

Recursos en línea y otros materiales

Direcciones del GitHub, recomendaciones de cursos en línea, materiales recomendados y demás autoaprendizaje.

1

Aplicaciones de Machine Learning a través de R y Python

1.1. El concepto de esperanza condicional y Causalidad

El objetivo del análisis empírico en las ciencias sociales es determinar cuál es el efecto o cambio que una variable causa en otra. Por ejemplo, cuando pensamos en qué factores o variables determinan el nivel salarial promedio de las personas, es posible que relacionemos el efecto que tienen los años de estudio y, en particular, el efecto que tiene un año adicional de estudio. Otro ejemplo puede presentarse cuando tratamos de entender el efecto que tienen las horas de estudio en las calificaciones finales de un grupo de estudiantes.

Dentro del análisis empírico se suele hacer uso de dos términos o conceptos:

1. *Ceteris paribus*
2. *Causalidad*

El primero se emplea en el análisis empírico para establecer que el efecto de una variable se sostiene siempre que asumamos que el resto de las variables que pueden afectar a dicha variable que queremos explicar permanecen constantes.

El segundo sirve para entender que, en el análisis empírico, la existencia de correlaciones no significa la presencia de causalidad. La causalidad, en un primer momento, es la condición observada entre dos variables y deriva de la construcción teórica, modelación o racionalización que hagamos de los

fenómenos que queremos analizar. De forma más sofisticada, es una relación analizada bajo un enfoque conocido como **inferencia causal**.

1.1.1. Introducción a la esperanza condicional

Partamos de lo que en estadística se conoce como **esperanza condicional**. Supongamos dos variables, y y x , que tienen una distribución conjunta y para las cuales queremos estimar el efecto que tiene la segunda en la primera. Así, es posible que utilicemos una expresión de la esperanza condicional de y en x , misma que solemos representar como:

$$\mathbb{E}[y|x, \mathbf{C}] \quad (1.1)$$

Donde $\mathbf{C}$ representa un conjunto o vector de variables de control, en el sentido de que también explican la variabilidad de y ; por lo que no podemos omitirlas en un proceso de estimación de la esperanza condicional.

Dicho lo anterior, el análisis empírico que proponemos en este curso está basado en:

1. Identificar correlaciones que pueden ser interpretadas como causalidad, y
2. Estimar la ecuación (1.1).

En la ecuación (1.1) hemos asumido que $\mathbf{C}$ contiene toda la información disponible que sirve de control, por lo que es un vector que contiene sólo información que es observable. Sin embargo, no siempre es posible observar toda la información que podría servir de control.

Por ejemplo, al responder qué factores determinan el salario de las personas, podríamos pensar en múltiples factores cuantificables y observables, como la edad, el sexo, los años de educación, etc. No obstante, también consideraríamos factores como las habilidades propias de cada individuo, las cuales no son necesariamente observables.

Existen tres casos de los tipos de datos que son comúnmente analizados en el análisis de regresión lineal.

1. El primero es un análisis de datos de sección cruzada en el que la situación representativa de la ecuación (1.1) es como la siguiente:

$$\ln(\text{Salario}_i) = \beta_0 + \beta_1 \text{Educacion}_i + \beta_2 \text{Experiencia}_i$$

Donde las variables de $Salario_i$, $Educacion_i$ y $Experiencia_i$ son observadas para cada uno de los individuos en la muestra es indexado por $i = 1, 2, 3, \dots, N$. Es decir, en este punto el análisis se centra en diferentes variables de múltiples individuos, las cuales son observadas en un momento dado.

2. El segundo ejemplo se trata de datos de series de tiempo como la siguiente expresión:

$$\ln(PIB_t) = \beta_0 + \beta_1 Empleo_t + \beta_2 Salarios_t$$

Donde las variables PIB_t , $Empleo_t$ y $Salarios_t$ son observadas para un individuo o entidad a lo largo de una muestra del tiempo indexada por $t = 1, 2, 3, \dots, T$. Este caso puede pensarse como la información del PIB, el empleo y los salarios de un país, como México, que es observada a lo largo de los últimos 30 o 50 años.

3. El tercer ejemplo resulta de la combinación de los dos anteriores. De esta forma, podríamos observar a una muestra de un conjunto de individuos, $i = 1, 2, 3, \dots, N$, a lo largo del tiempo, $t = 1, 2, 3, \dots, T$, y analizar el comportamiento de una variable a través de una ecuación como la siguiente:

$$\ln(Salario_{it}) = \beta_0 + \beta_1 Educacion_{it} + \beta_2 Experiencia_{it}$$

Donde las variables $Salario_{it}$, $Educacion_{it}$ y $Experiencia_{it}$ se observan para los mismos individuos a lo largo del tiempo que se indexan con los pares (i, t) , $i = 1, 2, 3, \dots, N$ y $t = 1, 2, 3, \dots, T$. Pensemos estos casos como situaciones en las que tenemos información del nivel salarial, grado de educación y años de experiencia de, por ejemplo, los todos los egresados la UNAM en en 2020. De forma similar asumamos que la información la tenemos para los años y meses subsecuentes a que ingresaron al mercado laboral y hasta el mes de diciembre de 2025. De esta forma, si observa, ha generado un panel de información de un conjunto de individuos observados, en algunas de sus características, a lo largo de 5 años.

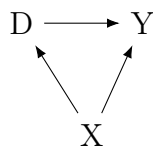


Figura 1.1: Ejemplo 1 de un DAG

1.1.2. Introducción a inferencia causal

Introducción a los Gráficos Acíclicos Dirigidos o Directed Acycle Graphical

La notación de los Gráficos Acíclicos Dirigidos (o DAG, en inglés) representa que la causalidad va en una dirección. Así, para mostrar causalidad en reversa, es necesario crear múltiples nodos. La simultaneidad, tal como en el caso de las curvas de oferta y demanda, no tiene una representación directa o inmediata con los DAG. Otra forma de plantear a los DAG es que se trata de una explicación de un fenómeno en términos de contrafactuales.

Definición 1.1 *Un DAG es una representación gráfica de una cadena de efectos causales. Los efectos causales están en sí mismos basados en algún proceso subyacente no observable.*

Los efectos causales pueden observarse en dos vías:

1. Directa: $D \longrightarrow Y$
2. Indirecta, a través de una tercer variable: $D \longrightarrow X \longrightarrow Y$

En este sentido, la ausencia de $\longrightarrow$ indica que no hay relación entre las variables.

Un ejemplo sencillo de DAG es el siguiente:

El DAG mostrado en la Figura (1.1) ilustra que hay una ruta directa de D a Y , lo cual representa un efecto causal. Por su parte, la ruta a D tiene una ruta trasera (backdoor path) a través de la ruta $D \longleftarrow X \longrightarrow Y$. En este caso, consideremos que la ruta directa es un efecto causal, pero la backdoor path es no causal. En su caso, este proceso crea una correlación espúrea entre D y Y .

Pensemos el proceso de ruta trasera como una situación en la que a veces cuando D toma diferentes valores, Y toma diferentes valores debido a que X

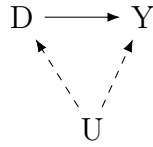


Figura 1.2: Ejemplo 2 de un DAG, confusor

toma diferentes valores. De esta forma, decimos que existe una correlación espuria entre D y Y .

Pensemos este ejemplo a partir de la siguiente pregunta: ¿tener un título universitario (variable D) causa un mayor ingreso económico (variable Y)? Pero existe un problema: no todas las personas acceden a la universidad de forma aleatoria. Algunas variables anteriores pueden estar influyendo tanto en la educación como en el ingreso, generando una asociación espuria si no se controlan. ¿Cómo cuáles variables? Nivel de motivación o esfuerzo personal y Clase socioeconómica del hogar de origen, pueden ser variables que afectan tanto a la variable de tener un título universitario o a tener mayores ingresos.

Un segundo ejemplo de una DAG que nos permite ilustrar el concepto de confusor, el cual describe a una variable que no es observable, U , y que representamos su relación en el DAG con líneas punteadas de la forma que se muestra en la Figura (1.2).

De forma similar al caso anterior, existen dos formas para ir de D a Y . Existe la ruta directa de D a Y , lo cual representa un efecto causal. Por su parte, la ruta a D tiene una ruta trasera a través de la ruta $D \leftarrow U \rightarrow Y$, pero con la diferencia de que la variable U es no observable. Retomando el ejemplo anterior, esa U podría ser las habilidades suaves o capacidad de resiliencia, variables que no son observables regularmente.

¿Cómo solucionar o mitigar este efecto confusor? Una manera es conocida como usar variables instrumentales. La Figura 1.3 ilustra que U son un conjunto de factores no observables que impiden indentificar la causalidad entre D y Y . De esta forma, buscamos un Z que no está correlacionado con U pero sí con D con el objeto de hacer una estimación auxiliar. A este Z se le suele conocer como variable instrumental.

Veamos otro ejemplo. Una pregunta clásica en economía y, a veces, en el discurso popular, es que la educación, en general, tiene el potencial de incrementar los ingresos laborales de las personas. Una posible teoría indicaría

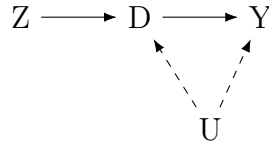


Figura 1.3: Ejemplo 4 de un DAG, estimador basado en variables instrumentales

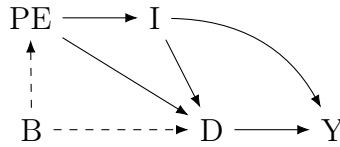


Figura 1.4: Ejemplo 3 de un DAG, ganancias de la educación

que la educación incrementa la productividad de las personas. De esta forma, los trabajadores más educados deberían ser mejor pagados, ya que sus salarios son establecidos en función de su productividad. Así, la teoría afirma que la educación incrementa los ingresos.

Particularicemos el ejemplo a un caso en el cual analizamos el efecto que tiene la educación secundaria (a veces denominada, educación media superior) en los ingresos. Partamos del hecho de que educarse a un nivel secundario no es un fenómeno aleatorio, puesto que existen múltiples factores que lo pueden explicar.

Iniciemos con el siguiente DAG, en el cual D será la variable de tratamiento o la variable que indica si una persona tiene formación de educación secundaria; Y es nuestra variable de interés o los ingresos; PE es la variable que indica el nivel educativo de los padres; I es una variable de ingreso familiar, y B es el conjunto de background no observable de la persona (factores como la genética, habilidades individuales, ambiente familiar, etc.).

El DAG de la Figura 1.4 es en sí mismo una historia. Cada persona tiene un cierto background que normalmente no se reporta en los conjuntos de datos. Medidas como el grado de inteligencia, la personalidad, la estabilidad emocional, capacidad de resiliencia, dinámica familiar y cualquier otro relacionado con factores ambientales o del entorno no se encuentran en los datos comúnmente. Por lo que se les denomina factores no observables.

Estos factores “ambientales” (B) están correlacionados o expresados en variables de padres e hijos. Este background causa que los padres elijan un nivel educativo o ruta de educación para los hijos y también afecta las decisiones individuales de los niños respecto a la ruta educativa que quieren seguir.

En este caso, también es posible notar que los DAG cuentan 2 historias. Nos dicen lo que está pasando, pero también nos dicen lo que no está pasando. Así, B no tiene un efecto directo en los ingresos laborales, excepto a través de su efecto en la elección educativa. Sin embargo, en muchas ocasiones se suele criticar este tipo de planteamientos por parecer más un supuesto que un hecho real. En estos casos, la decisión es del investigador.

Truncamiento o cómo evitar sesgar nuestros análisis

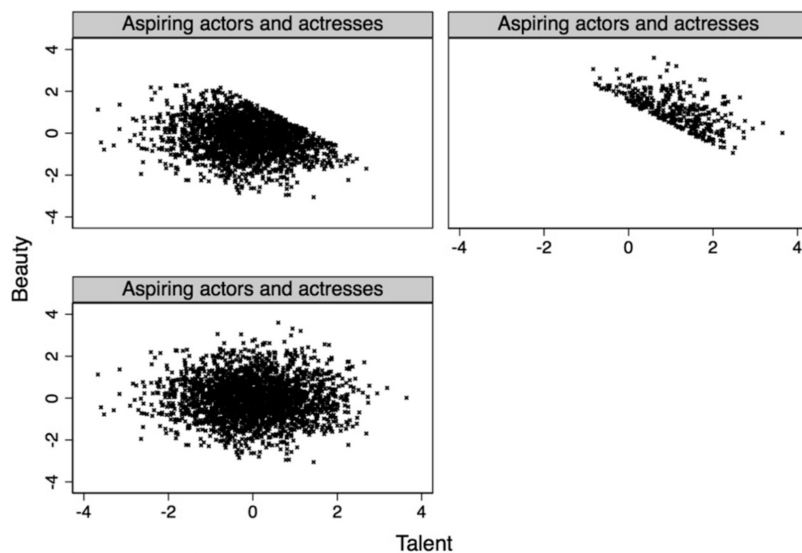
Tomemos un ejemplo (retomado de Cunningham (2021) [Cun21]). Una publicación del blog de CNN de 2009 informó que Megan Fox, quien protagonizó la película Transformers, fue votada como la peor y la más atractiva actriz de 2009 en una encuesta sobre estrellas de cine.

La publicación infería que había una implicación de que el talento y la belleza están negativamente correlacionados. ¿Pero lo son? ¿Y por qué podrían serlo? ¿Qué pasa si en realidad son independientes entre sí pero están correlacionados negativamente en una muestra de estrellas de cine debido al sesgo de cómo se seleccionó la información? ¿Es eso posible? Es decir, ¿el talento y la belleza están negativamente correlacionados?

En la figura 1.5 muestra una simulación de la situación. El panel inferior izquierdo muestra el diagrama de dispersión entre talento y belleza. Observe que las dos variables son extracciones aleatorias e independientes de la distribución normal estándar, lo que crea una nube de datos alargada. Pero debido a que “estrella de cine” se encuentra en el percentil 85 superior de la distribución de una combinación lineal de talento y belleza, la muestra consta de personas cuya puntuación combinada se encuentra en la parte superior derecha de la distribución conjunta. Esta frontera tiene una pendiente negativa y se encuentra en la parte superior derecha de la nube de datos, lo que crea una correlación negativa entre las observaciones en la muestra de estrellas de cine.

Sin embargo, sabemos que, de hecho, no hay relación entre las dos variables. Este tipo de selección de muestras crea correlaciones espurias. Una muestra aleatoria de la población completa sería suficiente para mostrar que no existe

Figura 1.5: Relación entre belleza y talento, retomado de [Cun21].



relación entre las dos variables, pero al dividir la muestra en estrellas de cine únicamente, introducimos correlaciones espurias entre las dos variables de interés.

El mensaje final es que debemos tener cuidado de no estar analizando una muestra truncada de la información y, más aún, pretender hacer inferencias generales con muestras truncadas de la población.

1.1.3. Notación en el análisis de regresión

Iniciemos con un ejemplo. Pensemos la regresión como un proceso de formalizar un gráfico de dispersión (scatter plot). Como ejemplo, considere la Figura 1.6 en la que se muestra el scatter plot entre los años de educación y el salario. Una hipótesis sensata es pensar que la relación es positiva e, incluso, lineal: a más años de estudio, más salario.

Este planteamiento, como se puede ver, se centra en la creación de una **hipótesis**. Es decir, hacer regresión lineal es asumir la hipótesis de que existe una relación entre las variables que puede ser capturada por un modelo analítico que llamamos ecuación de regresión.

En primer lugar, estableceremos un poco de notación. En el análisis de regresión, siempre partimos de una representación de una ecuación lineal o

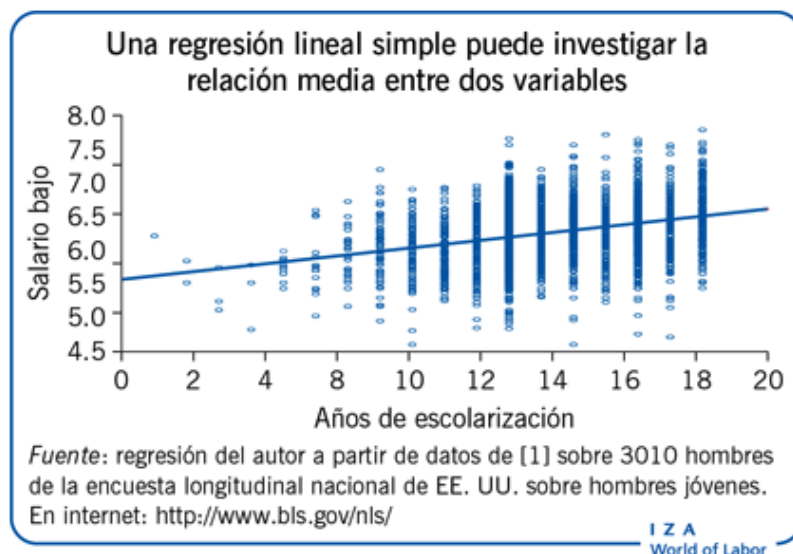


Figura 1.6: ¿Qué relación hay entre años de educación y el salario?, retomado de: <https://wol.iza.org/articles/using-linear-regression-to-establish-empirical-relationships/lang/es>

hipótesis (h) como:

$$y_i = \beta_1 x_{i1} + \beta_2 x_{i2} + \beta_3 x_{i3} + \dots + \beta_K x_{iK} + \varepsilon_i = h(x_{i1}, x_{i2}, \dots, x_{iK}) + \varepsilon_i \quad (1.2)$$

Donde cada una de las variables y_i , x_{ik} y ε_i se observan para $i = 1, 2, 3, \dots, N$ y $k = 1, 2, \dots, K$. Al conjunto de las variables x_{ik} se les llama características y a y_i se le suele decir variable objetivo, la cual estamos tratando de predecir. En este curso, al par (y_i, x_{ik}) le llamaremos ejemplo de entrenamiento, del cual tenemos N casos.

Por lo general, la hipótesis representada por la ecuación de regresión la representaremos para cada i de la forma matricial:

$$\begin{aligned} y_i &= \begin{pmatrix} x_{i1} & x_{i2} & \dots & x_{iK} \end{pmatrix} \begin{pmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_K \end{pmatrix} + \varepsilon_i \\ &= \mathbf{X}'_i \boldsymbol{\beta} + \varepsilon_i \\ &= h(\mathbf{X}'_i) + \varepsilon_i \end{aligned} \quad (1.3)$$

En este sentido, de forma relacionada, podemos definir la función de costos:

$$\begin{aligned} J(\boldsymbol{\beta}) &= \frac{1}{2} \sum_{i=1}^N (h(\mathbf{X}'_i) - y_i)^2 \\ &= \frac{1}{2} \sum_{i=1}^N (\mathbf{X}'_i \boldsymbol{\beta} - y_i)^2 \end{aligned} \quad (1.4)$$

La ecuación (1.3) se puede generalizar para el total de elementos en la muestra como:

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon} \quad (1.5)$$

Donde:

$$\mathbf{Y} = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_N \end{pmatrix}$$

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1K} \\ x_{21} & x_{22} & \dots & x_{2K} \\ x_{31} & x_{32} & \dots & x_{3K} \\ \vdots & \vdots & \dots & \vdots \\ x_{N1} & x_{N2} & \dots & x_{NK} \end{bmatrix}$$

$$\boldsymbol{\beta} = \begin{pmatrix} \beta_1 \\ \beta_2 \\ \beta_3 \\ \vdots \\ \beta_K \end{pmatrix}$$

$$\boldsymbol{\varepsilon} = \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \varepsilon_3 \\ \vdots \\ \varepsilon_N \end{pmatrix}$$

En la mayoría de los casos analizados vamos a asumir que la ecuación (1.2) tiene un término constante, por lo que $x_{i1} = 1$ para todo $i = 1, 2, \dots, N$. Bajo este escenario, representaremos la matriz $\mathbf{X}$ con una columna compuesta del número 1 (uno) en todas sus entradas, tal que, el parámetro β_1 es un término constante en las ecuaciones (1.3) y (1.5). De esta forma, la matriz anteriormente mostrada se puede ver como:

$$\mathbf{X} = \begin{bmatrix} 1 & x_{12} & \dots & x_{1K} \\ 1 & x_{22} & \dots & x_{2K} \\ 1 & x_{32} & \dots & x_{3K} \\ \vdots & \vdots & \dots & \vdots \\ 1 & x_{n2} & \dots & x_{nK} \end{bmatrix}$$

De forma similar al caso de una regresión lineal que incluye un término constante, podemos construir matrices $\mathbf{X}$ para los casos en que alguna de las variables explicativas es dicotómica –que toma valores de 0 y 1– y los casos en los que las variables explicativas han sido interactuadas –multiplicadas entre sí– o transformadas mediante potencias o logaritmos.

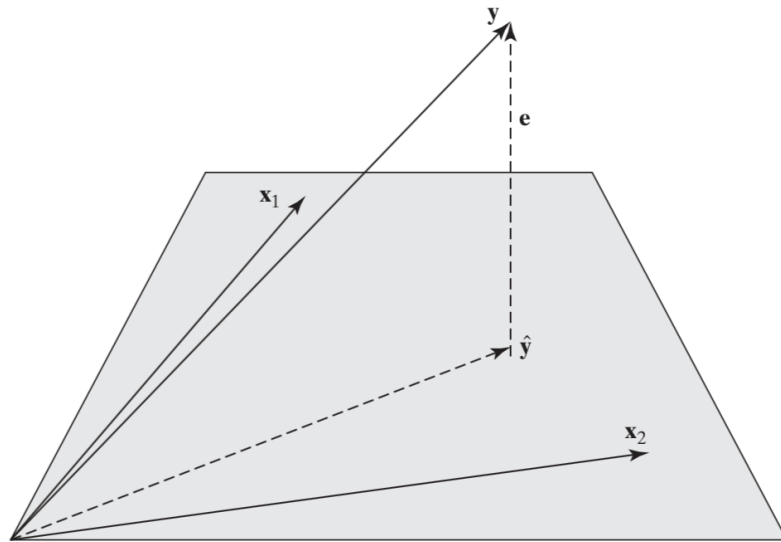


Figura 1.7: Ilustración de la proyección de y en x_1 y x_2 y el significado del error e , retomado de Greene (2012, p. 31) [Gre12]

1.2. Aplicación de modelos de regresión lineal.

1.2.1. El estimador de Mínimos Cuadrados Ordinarios (MCO) y el análisis clásico de regresión

El concepto de regresión se puede ilustrar en las Figuras 1.7, 1.8 y 1.9. Si partimos de las ecuaciones (1.3) u (1.5), podemos establecer que:

- $\mathbf{X}'_i$ son las características que explican el fenómeno de interés.
- y_i es la variable objetivo que estamos tratando de predecir.
- El par $(\mathbf{X}'_i, y_i)$ es el ejemplo de entrenamiento.
- El conjunto de datos es la lista de los N ejemplos de entrenamiento.
- La forma funcional $\mathbf{X}'_i\boldsymbol{\beta}$ es la forma que representa como hipótesis, $h(\cdot)$, del comportamiento de y_i

Así, nuestro objetivo es, dado el conjunto de entrenamiento, aprender una función $h : X \rightarrow Y$ a la que se le ha llamado “hipótesis”—que aplica

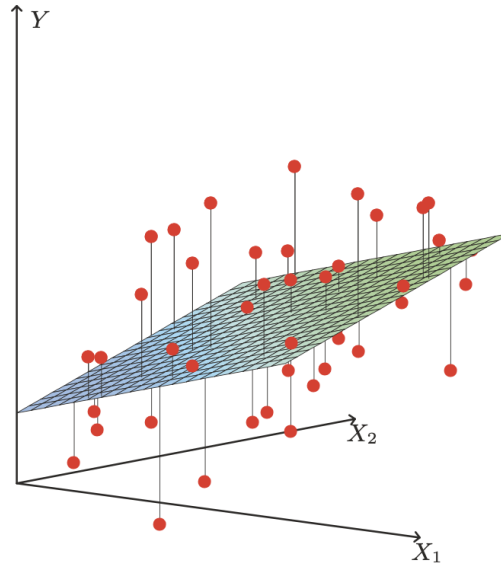


Figura 1.8: Ilustración del hiperplano generado de regesar X_1 y X_2 en Y , retomado de Hastie, Tibshirani y Friedman (2009, pp. 45) [HTF09]

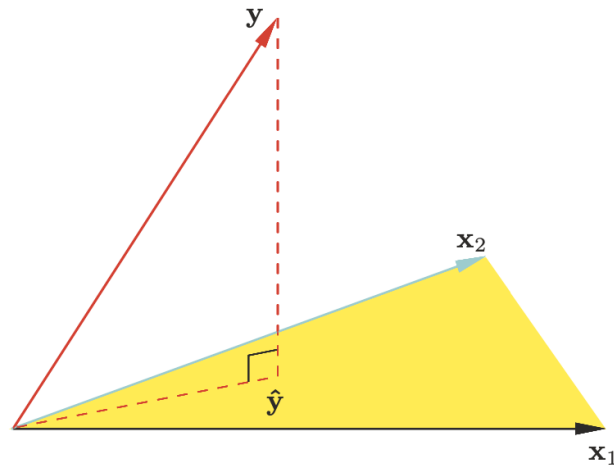


Figura 1.9: Ilustración de la proyección de y en x_1 y x_2 y el significado del error e , retomado de Hastie, Tibshirani y Friedman (2009, pp. 46) [HTF09]

sin importar si se trata de un problema de regresión o de un problema de clasificación.

Definamos el término de error como:

$$\varepsilon_i = y_i - \mathbf{X}'_i \boldsymbol{\beta} \quad (1.6)$$

Donde $i = 1, 2, \dots, N$. De forma similar, podremos decir que un estimador de este término de error será aquel que resulte de:

$$e_i = y_i - \mathbf{X}'_i \hat{\boldsymbol{\beta}} \quad (1.7)$$

Donde $\hat{\boldsymbol{\beta}}$ es un vector de estimadores de los parámetros $\boldsymbol{\beta}$. De lo dicho hasta ahora, es fácil ver que la siguiente ecuación es cierta $\forall i$:

$$y_i = \mathbf{X}'_i \boldsymbol{\beta} + \varepsilon_i = \mathbf{X}'_i \hat{\boldsymbol{\beta}} + e_i \quad (1.8)$$

Intuitivamente, la ecuación (1.8) significa que siempre que poseamos una muestra de los elementos de la población, podremos explicar una parte de la variable dependiente, no su totalidad.

En este sentido, el análisis de regresión consiste en un proceso de ajuste a la variable dependiente. Esta es la idea que da origen al R^2 y otras medidas de bondad de ajuste, mismas que se analizan en textos convencionales de análisis de regresión.

El método de MCO, en consecuencia, resulta en encontrar la combinación de valores de los estimadores de los parámetros $\hat{\boldsymbol{\beta}}$ que permitan minimizar la suma de los residuales (estimadores de los términos de error ε) al cuadrado dada por:

$$\sum_{i=1}^N e_i^2 = \sum_{i=1}^N (y_i - \mathbf{X}'_i \hat{\boldsymbol{\beta}})^2 \quad (1.9)$$

Donde $\hat{\boldsymbol{\beta}}$ denota el vector de estimadores $\hat{\beta}_1, \dots, \hat{\beta}_K$. En términos matriciales, dado que $(e_1, e_2, \dots, e_n)'(e_1, e_2, \dots, e_n) = \mathbf{e}'\mathbf{e}$, el problema del método de MCO consiste en resolver el problema de optimización:

$$\begin{aligned} \text{Minimizar}_{\hat{\boldsymbol{\beta}}} \{S(\hat{\boldsymbol{\beta}})\} &= \text{Minimizar}_{\hat{\boldsymbol{\beta}}} \{\mathbf{e}'\mathbf{e}\} \\ &= \text{Minimizar}_{\hat{\boldsymbol{\beta}}} \{(\mathbf{Y} - \mathbf{X}\hat{\boldsymbol{\beta}})'(\mathbf{Y} - \mathbf{X}\hat{\boldsymbol{\beta}})\} \end{aligned} \quad (1.10)$$

Expandiendo la expresión $\mathbf{e}'\mathbf{e}$ contenida en la ecuación (1.10), obtenemos:

$$\mathbf{e}'\mathbf{e} = \mathbf{Y}'\mathbf{Y} - 2\mathbf{Y}'\mathbf{X}\hat{\boldsymbol{\beta}} + \hat{\boldsymbol{\beta}}'\mathbf{X}'\mathbf{X}\hat{\boldsymbol{\beta}} \quad (1.11)$$

De esta forma, obtenemos que las condiciones necesarias de un mínimo son:

$$\frac{\partial S(\hat{\beta})}{\partial \hat{\beta}} = -2\mathbf{X}'\mathbf{Y} + 2\mathbf{X}'\mathbf{X}\hat{\beta} = \mathbf{0} \quad (1.12)$$

De la ecuación anterior, obtenemos como la solución del problema del mínimo a las ecuaciones siguientes—conocidas como *ecuaciones normales*—dadas por:

$$\mathbf{X}'\mathbf{X}\hat{\beta} = \mathbf{X}'\mathbf{Y} \quad (1.13)$$

Notemos que dichas ecuaciones normales son, en realidad, un sistema de ecuaciones de K variables o incógnitas. Por un lado, recordemos que $\mathbf{X}$ es una matriz de dimensión $N \times K$, con lo cual $\mathbf{X}'$ es de dimensión $K \times N$.

Así, el producto $\mathbf{X}'\mathbf{X}$ dará como resultado una matriz cuadrada de dimensión $K \times K$. Por otro lado, sabemos que $\mathbf{Y}$ es un vector de tamaño $N \times 1$, con lo cual el producto $\mathbf{X}'\mathbf{Y}$ da como resultado un vector de dimensión $K \times 1$. En conclusión, el sistema de ecuaciones normales consiste en K ecuaciones con K incógnitas ($\hat{\beta}_1, \dots, \hat{\beta}_K$).

Ante este hecho, existen múltiples formas mediante las cuales se puede solucionar dicho sistema. Sin embargo, en nuestro caso, seguiremos el siguiente procedimiento de operaciones matriciales.

Si la matriz $\mathbf{X}$ es de rango completo por columnas, entonces la inversa de la matriz $\mathbf{X}'\mathbf{X}$ existe. De esta forma, la solución esta dada por la siguiente expresión:

$$\hat{\beta} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{Y} \quad (1.14)$$

Finalmente, para que esta solución dada para el procedimiento de MCO y mostrada en la ecuación (1.14) sea un mínimo, debemos buscar las condiciones de segundo orden:

$$\frac{\partial^2 S(\hat{\beta})}{\partial \hat{\beta} \partial \hat{\beta}'} = 2\mathbf{X}'\mathbf{X} \quad (1.15)$$

Donde la matriz $\mathbf{X}'\mathbf{X}$ debe ser positiva definida para que la solución de MCO sea un mínimo. Sea $q = \mathbf{c}'\mathbf{X}'\mathbf{X}\mathbf{c}$ para algún vector $\mathbf{c}$ distinto de cero. Entonces:

$$q = \mathbf{v}'\mathbf{v} = \sum_{i=1}^n v_i^2, \text{ donde } \mathbf{v} = \mathbf{X}\mathbf{c}$$

Así, q es positivo. Si $\mathbf{v}$ fuera cero, entonces existe una combinación lineal de las columnas de $\mathbf{X}$ que da como resultado cero, lo cual contradice el supuesto

de que $\mathbf{X}$ es de rango completo. En todos los casos, si $\mathbf{X}$ es de rango completo, entonces la solución del método de MCO, $\hat{\beta}$, es la única que minimiza la suma de los residuales al cuadrado.

Bondad de ajuste

Una vez obtenida la solución de MCO, plantearemos una medida para determinar en qué grado los datos estimados, definidos como $\hat{\mathbf{Y}} = \mathbf{X}\hat{\beta}$, se ajustan al valor real de $\mathbf{Y}$.

La medida o métrica es el R^2 , la cual contrasta el análisis de regresión respecto de hacer una simple estimación de una media para dar un pronóstico de $\mathbf{Y}$. El coeficiente R^2 está montado en el supuesto de que el modelo incluye un término constante. **Si el modelo no incluye una constante, no es posible hacer una interpretación del R^2** , en los siguientes párrafos abundaremos al respecto.

Antes de iniciar el desarrollo del R^2 , partamos de que la suma de residuales es igual a cero ($\sum_{i=1}^n e_i = 0$), si y sólo si, el modelo tiene un término constante. Un corolario de este hecho es que el valor promedio de los residuales es cero, si y solo si, el modelo tiene un término constante. Esto es, la única forma en que se cumpla la siguiente expresión es que la suma de residuales sea cero:

$$\begin{aligned}\bar{e} &= \frac{\sum_{i=1}^n e_i}{n} \\ &= \frac{1}{n} \cdot \begin{bmatrix} 1, & 1, & 1, & \dots & 1 \end{bmatrix} \begin{bmatrix} e_1 \\ e_2 \\ e_3 \\ \vdots \\ e_n \end{bmatrix} \\ &= 0\end{aligned}$$

Condición que resulta de las ecuaciones normales de MCO, recordemos que ellas se derivan de que $\mathbf{X}'\mathbf{e} = \mathbf{0}$, y que el primer producto punto de vectores implica el uso de la columna de la constante en la matriz $\mathbf{X}$ y que está definido por $\begin{bmatrix} 1, & 1, & 1, & \dots & 1 \end{bmatrix}$.

El R^2 descompone la variación total en dos tramos: la variación originada por la regresión y la variación originada por el término de error. Como primer

paso definamos la variación total observada respecto de la media ($\bar{Y}$) como:

$$\begin{aligned}
\sum_{i=1}^n (y_i - \bar{Y})^2 &= \begin{bmatrix} y_1 - \bar{Y} & y_2 - \bar{Y} & y_3 - \bar{Y} & \dots & y_n - \bar{Y} \end{bmatrix} \begin{bmatrix} y_1 - \bar{Y} \\ y_2 - \bar{Y} \\ y_3 - \bar{Y} \\ \vdots \\ y_n - \bar{Y} \end{bmatrix} \\
&= (\mathbf{Y} - \bar{\mathbf{Y}})'(\mathbf{Y} - \bar{\mathbf{Y}}) \\
&= \left[\mathbf{Y} - \frac{1}{n} \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \\ \vdots & \vdots & \dots & \vdots \\ 1 & 1 & \dots & 1 \end{bmatrix} \mathbf{Y} \right]' \left[\mathbf{Y} - \frac{1}{n} \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \\ \vdots & \vdots & \dots & \vdots \\ 1 & 1 & \dots & 1 \end{bmatrix} \mathbf{Y} \right] \\
&= \mathbf{Y}' \left[\mathbb{I}_n - \frac{1}{n} \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \\ \vdots & \vdots & \dots & \vdots \\ 1 & 1 & \dots & 1 \end{bmatrix} \right]' \left[\mathbb{I}_n - \frac{1}{n} \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \\ 1 & 1 & \dots & 1 \\ \vdots & \vdots & \dots & \vdots \\ 1 & 1 & \dots & 1 \end{bmatrix} \right] \mathbf{Y} \\
&= \mathbf{Y}' \mathbf{M}^0 \mathbf{Y} \tag{1.16}
\end{aligned}$$

En la expresión (1.16) a $\mathbf{M}^0$ se puede interpretar como una matriz que resta a una matriz o vector su promedio, notemos que como resultado dará las desviaciones respecto de la media. Asimismo, $\mathbf{M}^0$ tiene un par de propiedades que son fácilmente demostrables: idempotencia y simetría, es decir, i) $\mathbf{M}^0 = \mathbf{M}^{0'}$ y ii) $\mathbf{M}^0 \mathbf{M}^0 = \mathbf{M}^0$. Dicho lo anterior, retomemos a (1.16) para mostrar que:

$$\begin{aligned}
\mathbf{Y}' \mathbf{M}^0 \mathbf{Y} &= \mathbf{Y}' \mathbf{M}^{0'} \mathbf{M}^0 \mathbf{Y} \\
&= (\mathbf{X} \hat{\boldsymbol{\beta}} + \mathbf{e})' \mathbf{M}^{0'} \mathbf{M}^0 (\mathbf{X} \hat{\boldsymbol{\beta}} + \mathbf{e}) \\
&= (\hat{\boldsymbol{\beta}}' \mathbf{X}' + \mathbf{e}') \mathbf{M}^{0'} \mathbf{M}^0 (\mathbf{X} \hat{\boldsymbol{\beta}} + \mathbf{e}) \\
&= \hat{\boldsymbol{\beta}}' \mathbf{X}' \mathbf{M}^{0'} \mathbf{M}^0 \mathbf{X} \hat{\boldsymbol{\beta}} + \hat{\boldsymbol{\beta}}' \mathbf{X}' \mathbf{M}^{0'} \mathbf{M}^0 \mathbf{e} + \mathbf{e}' \mathbf{M}^{0'} \mathbf{M}^0 \mathbf{X} \hat{\boldsymbol{\beta}} + \mathbf{e}' \mathbf{M}^{0'} \mathbf{M}^0 \mathbf{e} \\
&= \hat{\boldsymbol{\beta}}' \mathbf{X}' \mathbf{M}^{0'} \mathbf{M}^0 \mathbf{X} \hat{\boldsymbol{\beta}} + \hat{\boldsymbol{\beta}}' \mathbf{X}' \mathbf{e} + \mathbf{e}' \mathbf{X} \hat{\boldsymbol{\beta}} + \mathbf{e}' \mathbf{e} \\
&= \hat{\boldsymbol{\beta}}' \mathbf{X}' \mathbf{M}^{0'} \mathbf{M}^0 \mathbf{X} \hat{\boldsymbol{\beta}} + \mathbf{e}' \mathbf{e} \\
&= \hat{\boldsymbol{\beta}}' \mathbf{X}' \mathbf{M}^0 \mathbf{X} \hat{\boldsymbol{\beta}} + \mathbf{e}' \mathbf{e} \tag{1.17}
\end{aligned}$$

No debe pasar desapercibido que el desarrollo algebraico para llegar a la ecuación (1.17) sólo es posible si el promedio de los residuales es cero, es decir:

$$\mathbf{M}^0 \mathbf{e} = \mathbf{e} - \begin{bmatrix} \bar{e} \\ \bar{e} \\ \bar{e} \\ \vdots \\ \bar{e} \end{bmatrix} = \mathbf{e} - \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \mathbf{e} \quad (1.18)$$

De otra forma, no se puede concluir la expresión (1.17). Finalmente, (1.17) la expresaremos así:

$$\mathbf{Y}'\mathbf{M}^0\mathbf{Y} = \hat{\boldsymbol{\beta}}'\mathbf{X}'\mathbf{M}^0\mathbf{X}\hat{\boldsymbol{\beta}} + \mathbf{e}'\mathbf{e} \quad (1.19)$$

Así, como mencionamos anteriormente, (1.19) se puede interpretar como que la variación total respecto de la media se puede descomponer en dos variaciones, una que se origina de la regresión, $\hat{\boldsymbol{\beta}}'\mathbf{X}'\mathbf{M}^0\mathbf{X}\hat{\boldsymbol{\beta}}$, y otra de los residuales, $\mathbf{e}'\mathbf{e}$. De esta forma, planteamos que R^2 es una metrica que cuantifica cuánto de la variación total es explicada por la regresión y cuánto es explicada por los residuales (es decir, por la información no observable):

$$\begin{aligned} R^2 &= \frac{Var.Regresion}{Var.Total} \\ &= \frac{\hat{\boldsymbol{\beta}}'\mathbf{X}'\mathbf{M}^0\mathbf{X}\hat{\boldsymbol{\beta}}}{\mathbf{Y}'\mathbf{M}^0\mathbf{Y}} \\ &= \frac{\mathbf{Y}'\mathbf{M}^0\mathbf{Y}}{\mathbf{Y}'\mathbf{M}^0\mathbf{Y}} - \frac{\mathbf{e}'\mathbf{e}}{\mathbf{Y}'\mathbf{M}^0\mathbf{Y}} \\ &= 1 - \frac{\mathbf{e}'\mathbf{e}}{\mathbf{Y}'\mathbf{M}^0\mathbf{Y}} \end{aligned} \quad (1.20)$$

Existe una expresión más que se le denomina como un R^2 ajustado o $R^2 Adj.$, el cual castiga por un uso excesivo de variables independientes:

$$R^2 Adj. = 1 - \frac{n-1}{n-K} \frac{\mathbf{e}'\mathbf{e}}{\mathbf{Y}'\mathbf{M}^0\mathbf{Y}} \quad (1.21)$$

Así, es obvio que la siguiente relación siempre es cierta: $R^2 \geq R^2 Adj.$.

Inferencia asintótica bajo MCO

En estadística es común que propongamos como estimador de σ^2 a:

$$\hat{\sigma}^2 = \frac{\mathbf{e}'\mathbf{e}}{n - K} \quad (1.22)$$

El cual es un estimador insesgado de la varianza. Además, queda pendiente en esta sección demostrar que el estimador de MCO ($\hat{\beta}$) alcanza la cota inferior Cramér-Rao. De esta forma, podríamos afirmar que el estimador es de mínima varianza o, equivalentemente, es el más eficiente.

Dicho lo anterior, podemos construir las dos pruebas: t y F. Previo a dicha construcción, debemos recordar que una prueba t se construye por la relación de una función normal con media cero y varianza 1 (uno), y la raíz cuadrada de una función chi-cuadrado. Supongamos que una variable Z se distribuye de forma normal con media cero y varianza 1 (uno), y que la variable χ_m^2 se distribuye de forma chi-cuadrado con m grados de libertad, entonces:

$$t = \frac{Z}{\sqrt{\frac{\chi_m^2}{m}}} \sim t_m \quad (1.23)$$

Donde t_m tal y como se describe en (1.23) es una distribución t de Student con m grados de libertad. Por otro lado, cuando se tiene una variable Z con distribución normal con media cero y varianza $\sigma^2 = 1$, entonces lo siguiente es cierto:

$$Z^2 \sim \chi_1^2$$

Así, la suma de variables chi-cuadrado es una chi-cuadrado en los siguientes términos:

$$\sum_{i=1}^m Z^2 \sim \chi_m^2$$

Dada la distribución $\hat{\beta}$, y que la varianza de cada uno de los elementos de $\hat{\beta}$, digamos $\hat{\beta}_k$, es: $\sigma^2(\mathbf{X}'\mathbf{X})_{kk}^{-1}$. En este punto denotaremos con $\hat{\beta}_k$ al elemento k -ésimo dentro de $\hat{\beta}$. Adicionalmente, con $(\mathbf{X}'\mathbf{X})_{kk}^{-1}$ al elemento en la fila k y la columna k en la matriz $(\mathbf{X}'\mathbf{X})^{-1}$. Derivado de lo anterior, lo siguiente es cierto:

$$Z = \frac{\hat{\beta}_k - \beta_k}{\sqrt{\sigma^2(\mathbf{X}'\mathbf{X})_{kk}^{-1}}} \sim \mathcal{N}(0, 1)$$

Considerando la distribución de ε_i y de que e_i es un estimador de ε_i , podemos afirmar que:

$$Z = \frac{e_i - 0}{\sigma} = \frac{e_i}{\sigma} \sim \mathcal{N}(0, 1)$$

De esta forma:

$$\sum_{i=1}^n Z_i^2 = \sum_{i=1}^n \frac{e_i^2}{\sigma^2} \sim \chi_n^2$$

Dada la distribución t de Student en la ecuación (1.23) y los elementos antes mencionados podemos llegar a las siguientes conclusiones:

$$\begin{aligned} t &= \frac{\frac{\hat{\beta}_k - \beta_k}{\sqrt{\sigma^2(\mathbf{X}'\mathbf{X})_{kk}^{-1}}}}{\sqrt{\sum_{i=1}^n \frac{e_i^2}{\sigma^2} / (n - K)}} \\ &= \frac{\frac{\hat{\beta}_k - \beta_k}{\sqrt{\sigma^2(\mathbf{X}'\mathbf{X})_{kk}^{-1}}}}{\sqrt{\sum_{i=1}^n \frac{e_i^2}{(n-K)\sigma^2}}} \\ &= \frac{\frac{\hat{\beta}_k - \beta_k}{\sqrt{\sigma^2(\mathbf{X}'\mathbf{X})_{kk}^{-1}}}}{\sqrt{\frac{\hat{\sigma}^2}{\sigma^2}}} \\ &= \frac{\frac{\hat{\beta}_k - \beta_k}{\sqrt{\sigma^2} \sqrt{(\mathbf{X}'\mathbf{X})_{kk}^{-1}}}}{\frac{\sqrt{\hat{\sigma}^2}}{\sqrt{\sigma^2}}} \\ &= \frac{(\hat{\beta}_k - \beta_k) \sqrt{\sigma^2}}{\sqrt{\hat{\sigma}^2} \sqrt{\sigma^2} \sqrt{(\mathbf{X}'\mathbf{X})_{kk}^{-1}}} \\ &= \frac{\hat{\beta}_k - \beta_k}{\sqrt{\hat{\sigma}^2(\mathbf{X}'\mathbf{X})_{kk}^{-1}}} \end{aligned}$$

De esta forma:

$$t = \frac{\hat{\beta}_k - \beta_k}{\sqrt{\hat{\sigma}^2(\mathbf{X}'\mathbf{X})_{kk}^{-1}}} \sim t_{n-k} \quad (1.24)$$

Construida esta expresión (1.24) podemos establecer el siguiente intervalo de confianza para cada uno de nuestros estimadores $\hat{\beta}_k$:

$$-t_{\alpha/2, n-K} \sqrt{\hat{\sigma}^2(\mathbf{X}'\mathbf{X})_{kk}^{-1}} < \hat{\beta}_k - \beta_k < t_{\alpha/2, n-K} \sqrt{\hat{\sigma}^2(\mathbf{X}'\mathbf{X})_{kk}^{-1}}$$

$$\hat{\beta}_k - t_{\alpha/2, n-K} \sqrt{\hat{\sigma}^2(\mathbf{X}'\mathbf{X})_{kk}^{-1}} < \beta_k < \hat{\beta}_k + t_{\alpha/2, n-K} \sqrt{\hat{\sigma}^2(\mathbf{X}'\mathbf{X})_{kk}^{-1}}$$

Donde $t_{\alpha/2, n-K}$ es el valor crítico de tablas t-Student. Así, podemos enunciar que el estimador de MCO de la ecuación (1.14) se sujetará a la siguiente prueba de hipótesis general:

$$\begin{aligned} H_0 : \beta_k &= r \\ H_a : \beta_k &\neq r \end{aligned}$$

Donde $k = 1, 2, \dots, K$ y r es un valor respecto del cual se desea comparar a $\hat{\beta}_k$. Dicha prueba se distribuye como una t_{n-K} y su especificación más común en el análisis de regresión es la siguiente:

$$\begin{aligned} H_0 : \beta_k &= 0 \\ H_a : \beta_k &\neq 0 \end{aligned}$$

Lo que en términos de una prueba t es la siguiente:

$$t = \frac{\hat{\beta}_k - 0}{\sqrt{\hat{\sigma}^2(\mathbf{X}'\mathbf{X})_{kk}^{-1}}} \sim t_{n-k} \quad (1.25)$$

Dicha prueba la podemos interpretar como sí es posible que el estimador $\hat{\beta}_k$ sea igual a 0, o en términos económicos, si existe evidencia estadística de que el efecto de la variable x_k en y es nulo.

Como se mostró anteriormente, sólo se hemos hablado de pruebas de hipótesis cuándo se desea comprobar si alguno de los estimadores $\hat{\beta}_k$ de MCO es estadísticamente igual a algún valor determinado. No obstante, en ciertas condiciones puede ser interesante cuestionar si en conjunto todos los estimadores cumplen una cierta restricción. A este tipo de situaciones se les suele conocer como pruebas de hipótesis globales. Sin pérdida de generalidad podemos enunciar a una prueba global como:

$$\begin{aligned} H_0 : \mathbf{R}\boldsymbol{\beta} &= \mathbf{r} \\ H_a : \mathbf{R}\boldsymbol{\beta} &\neq \mathbf{r} \end{aligned}$$

Donde, β es el vector de parámetros del MCO de la ecuación (1.14), $\mathbf{R}$ y $\mathbf{r}$ son de la siguiente manera:

$$\mathbf{R} = \begin{bmatrix} R_{11} & R_{12} & \dots & R_{1K} \\ R_{21} & R_{22} & \dots & R_{2K} \\ R_{31} & R_{32} & \dots & R_{3K} \\ \vdots & \vdots & \dots & \vdots \\ R_{J1} & R_{J2} & \dots & R_{JK} \end{bmatrix}$$

En $\mathbf{R}$ cada una de las R_{jk} toma el valor de 1 o 0, según sea la estructura de la restricción. Con $j = 1, 2, \dots, J$. En el caso de $\mathbf{r}$:

$$\mathbf{r} = \begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ \vdots \\ r_J \end{bmatrix}$$

Donde cada uno de los r_j representa el valor de la restricción que es evaluada. No debe pasar desapercibido que en este caso existen J restricciones ($J \leq K$), mismas que se evalúan de forma conjunta en una sola prueba. Esa es la ventaja de la prueba que se enuncia a continuación.

Definamos a F de Fisher como la razón de dos pruebas t de Student, la primera para los valores de una combinación lineal de $\hat{\beta}$, y la segunda para el estimador de la varianza $\hat{\sigma}^2$:

$$\begin{aligned} \mathbf{F} &= \frac{(\mathbf{R}\hat{\beta} - \mathbf{r})'[\mathbf{R}(\mathbf{X}'\mathbf{X})^{-1}\mathbf{R}']^{-1}(\mathbf{R}\hat{\beta} - \mathbf{r})/J}{\hat{\sigma}^2} \\ &= \frac{(\mathbf{R}\hat{\beta} - \mathbf{r})'[\mathbf{R}\hat{\sigma}^2(\mathbf{X}'\mathbf{X})^{-1}\mathbf{R}']^{-1}(\mathbf{R}\hat{\beta} - \mathbf{r})}{J} \\ &= \frac{(\mathbf{R}\hat{\beta} - \mathbf{r})'[\mathbf{R}Var(\hat{\beta})\mathbf{R}']^{-1}(\mathbf{R}\hat{\beta} - \mathbf{r})}{J} \end{aligned}$$

Por lo tanto la estadística F de prueba será:

$$F = \frac{(\mathbf{R}\hat{\beta} - \mathbf{r})'[\mathbf{R}\hat{\sigma}^2(\mathbf{X}'\mathbf{X})^{-1}\mathbf{R}']^{-1}(\mathbf{R}\hat{\beta} - \mathbf{r})}{J} \sim F_{J, n-K} \quad (1.26)$$

Así, la prueba de hipótesis que es la más común en el análisis de regresión y que se le conoce como prueba global. Dicha prueba asume una forma de la

matriz $\mathbf{R}$ definida así:

$$\mathbf{R} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ 0 & 0 & \dots & 0 \\ \vdots & \vdots & \dots & \vdots \\ 0 & 0 & \dots & 1 \end{bmatrix}$$

Y un vector $\mathbf{r}$:

$$\mathbf{r} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

De esta forma la hipótesis nula de una prueba global se puede escribir como:

$$\begin{aligned} H_0 : & \quad \beta_1 = \beta_2 = \dots = \beta_K = 0 \\ H_a : & \quad NoH_0 \end{aligned}$$

Esta prueba se le conoce como prueba global, ya que prueba si en conjunto todas las variables independientes tienen un efecto nulo en $\mathbf{Y}$.

En este caso nos referimos a casos como:

$$H_0 : \mathbf{R}\boldsymbol{\beta} = r$$

Donde $\mathbf{R}$ es una matriz de restricciones de dimensión $Q \times K$, con $Q \leq K$, r es un vector de dimensión $Q \times 1$. Así en casos de heterocedasticidad utilizaremos una prueba de Wald dada por:

$$\mathbf{W} = (\mathbf{R}\hat{\boldsymbol{\beta}} - r)'(\mathbf{R}\hat{\mathbf{V}}\mathbf{R}')^{-1}(\mathbf{R}\hat{\boldsymbol{\beta}} - r)$$

Donde $\hat{\mathbf{V}} = \widehat{AVar(\hat{\boldsymbol{\beta}})}$, por lo que la distribución de $\mathbf{W}$ será como:

$$\mathbf{W} \sim \chi_Q^2$$

Lo que implica que la formulación $F_{Q,N-K}$ no es de aplicación.

Otro tipo de pruebas que pueden realizarse son aquellas basadas en Multiplicadores de Lagrange. Supongamos un modelo lineal particionado de la siguiente manera:

$$y_i = \mathbf{x}_{i1}\boldsymbol{\beta}_1 + \mathbf{x}_{i2}\boldsymbol{\beta}_2 + \varepsilon \quad (1.27)$$

Supongamos que deseamos probar la hipótesis de que:

$$H_0 : \beta_2 = 0$$

Sea $\tilde{\beta}_1$ el estimador de β_1 bajo la hipótesis nula anterior, digamos que este estimador corresponde al modelo restringido. Definamos los residuales:

$$e_i = y_i - \mathbf{x}_{i1}\tilde{\beta}_1$$

Bajo la hipótesis nula, $\mathbf{x}_{i2}$ debería ser una variable no correlacionada con e_i . En este caso, la prueba estadística es obtenida como sigue:

1. Corremos una regresión de e_i en $\mathbf{x}_{i1}$ y $\mathbf{x}_{i2}$
2. Asumiendo que $\mathbf{x}_{i1}$ contiene término constante, digamos que R_E^2 denota al R^2 asociado a la regresión.
3. Entonces proponemos que la estadística de Multiplicadores de Lagrange es:

$$LM = N \cdot R_E^2$$

Para llegar a la expresión anterior se requiere de solucionar un problema de optimización.

4. Bajo la H_0 , tenemos que:

$$LM \sim \chi_{K_2}^2$$

Donde K_2 es el número de restricciones que hemos probado. De esta forma, si $N \cdot R_E^2$ es suficientemente grande, entonces e_i estaba significativamente correlacionado con $\mathbf{x}_{i2}$, por lo que la hipótesis debió ser rechazada.

En este punto, debe quedar claro que R^2 requiere que $\sum_{i=1}^N e_i = 0$, lo que implica que el modelo tiene un término constante.

1.2.2. Ejemplo: Regresión lineal múltiple con datos de publicidad en medios de comunicación.

Ver RMarkdown: 01_Ejemplo_Datos_Publicidad.Rmd

1.2.3. Modelos no lineales

¿Qué pasa cuando la relación observada entre las variables no es lineal? Casi siempre pasa eso. A continuación discutiremos tres casos de ecuaciones que nos ayudarán a describir una relación no lineal mediante una transformación que hace a las relaciones lineales. Cada uno responde a condiciones o características que la experiencia del investigador tiene que solventar.

Cada una de las ecuaciones se distingue por la tasa de cambio de la variable y en relación a los cambios de la variable x . Partamos de que la forma más simple de una relación lineal está dada por la ecuación (1.28). Note que hemos eliminado los subíndices para indicar que la ecuación se cumple para cada uno de los elementos en la muestra, $i = 1, 2, \dots, n$. Sin pérdida de generalidad, digamos que en lugar de β_1 y β_2 empleamos a y b como constantes, como una forma de ilustrar cada uno de los fenómenos que discutimos a continuación.

$$y = a + bx \quad (1.28)$$

Esta ecuación (1.28) tiene la característica de que tiene una tasa de variación constante para y en relación a x , de esta forma:

$$\frac{dy}{dx} = b \iff dy = bdx \iff \int dy = \int bdx \iff y = a + bx$$

Los siguientes modelos no lineales tienen una motivación similar, dependen de la forma en que se asuma la variación de y en función de la variación de x .

Regresión exponencial

Supongamos que y depende de x y que los cambios en y derivados de x son proporcionales a y , es decir:

$$\frac{dy}{dx} = by$$

Donde b es una constante. De esta forma podemos encontrar que:

$$\frac{dy}{dx} = by \iff \frac{dy}{y} = bdx \iff \int \frac{dy}{y} = \int bdx \iff \ln(y) = a + bx$$

De esta forma, tenemos que:

$$y = e^{bx} e^a \iff y = ke^{bx}$$

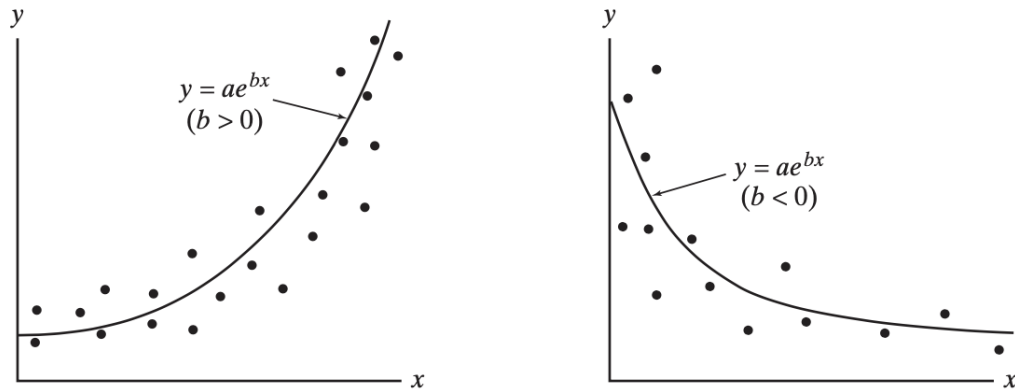


Figura 1.10: Funciones exponenciales, Retomado de Larsen y Marx (2012, p. 532) [LM12]

Por lo tanto, cuando estimamos una relación: $\ln(y) = a + bx$ estamos asumiendo que existe la siguiente relación: $\frac{dy}{dx} = by$. A este tipo de ecuaciones se les conoce como log-lineales. En la Figura 1.10 se ilustra la relación log-lineal - exponencial.

Regresión logarítmica

En este caso suponemos que los cambios en y por causa de cambios en x son proporcionales a la razón que guardan y y x , es decir:

$$\frac{dy}{dx} = b \frac{y}{x}$$

Por lo que podemos establecer:

$$\frac{dy}{dx} = b \frac{y}{x} \iff \frac{dy}{y} = b \frac{dx}{x} \iff \int \frac{dy}{y} = \int b \frac{dx}{x} \iff \ln(y) = a + b \ln(x)$$

Así, podemos establecer que la forma funcional de la relación de y y x es como sigue y como se muestra en la Figura 1.11.

$$y = kx^b$$

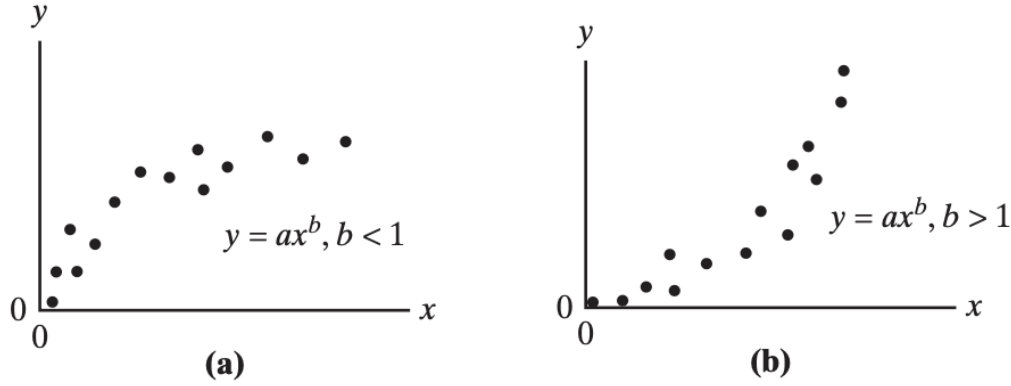


Figura 1.11: Funciones logarítmicas, Retomado de Larsen y Marx (2012, p. 536) [LM12]

Funciones logísticas

Este tipo de ecuaciones tienen la característica de que permiten modelar crecimientos poblacionales, aceptación de políticas públicas, aceptación de tecnologías, epidemias, etc. En este caso suponemos que enfrentamos que los cambios en y dados los cambios en x son proporcionales a y y a la distancia que y tiene respecto de un factor de saturación L (límite superior o punto de saturación poblacional, cobertura universal, infecciones en el total de la población, etc.).

En este caso suponemos que la variación estará dada por:

$$\frac{dy}{dx} = ky(L - y)$$

Donde k y L son constantes. Una vez solucionada la ecuación diferencial anterior, encontraremos que, si $L = 1$, entonces la solución será como sigue, la Figura 1.12 ilustra ecuaciones como la siguiente:

$$y = \frac{1}{1 + e^{a+bx}} \quad (1.29)$$

Ahora mostremos como llegar a la solución de la ecuación logística:

$$\frac{dy}{dx} = ky(L - y) \iff \frac{dy}{y(L - y)} = kdx \iff \left(\frac{\frac{1}{L}}{y} + \frac{\frac{1}{L}}{(L - y)} \right) dy = kdx$$

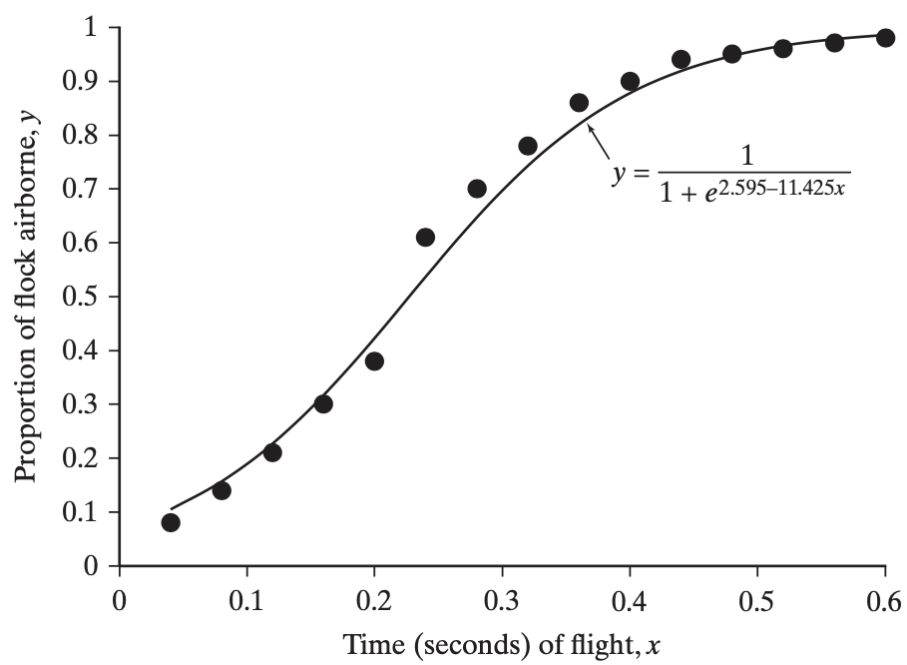


Figura 1.12: Funciones logísticas, Retomado de Larsen y Marx (2012, p. 540) [LM12]

$$\iff \frac{1}{L} \left(\frac{dy}{y} + \frac{dy}{(L-y)} \right) = kdx \iff \int \frac{1}{L} \left(\frac{dy}{y} + \frac{dy}{(L-y)} \right) = \int kdx$$

$$\iff \left(\int \frac{dy}{y} + \int \frac{dy}{(L-y)} \right) = \int Lkdx \iff (\ln(y) - \ln(L-y)) = Lkx + C$$

$$\iff (\ln(L-y) - \ln(y)) = -LC - Lkx \iff \ln \left(\frac{L-y}{y} \right) = -LC - Lkx$$

$$\iff \left(\frac{L-y}{y} \right) = e^{-LC-Lkx} \iff y = \frac{L}{1 + e^{-LC-Lkx}}$$

Por lo tanto, la solución general de la ecuación será:

$$y = \frac{L}{1 + e^{a+bx}}$$

Donde asumimos que la forma lineal de la ecuación logística es:

$$\ln \left(\frac{L-y}{y} \right) = a + bx$$

Funciones polinomiales

Otro tipo de ecuaciones o funciones se asocian con una relación entre y y x del tipo no lineal, pero que puede describirse bajo una formulación de un polinomio de algún grado l . Este tipo de ecuaciones tienen la característica de que permiten modelar crecimientos poblacionales, aceptación de políticas públicas, aceptación de tecnologías, epidemias, etc., mediante una especificación menos compleja en términos matemáticos.

En muchas aplicaciones, una simple transformación logarítmica de la variable x ya no puede ser suficiente para lograr un ajuste satisfactorio. Una forma sencilla de obtener más flexibilidad es utilizar la regresión polinomial. Normalmente, se prefieren polinomios de bajo grado l . En la práctica, rara vez utilizamos polinomios de grado superior a $l = 3$, ya que las estimaciones pueden volverse bastante inestables con una alta variabilidad en los límites

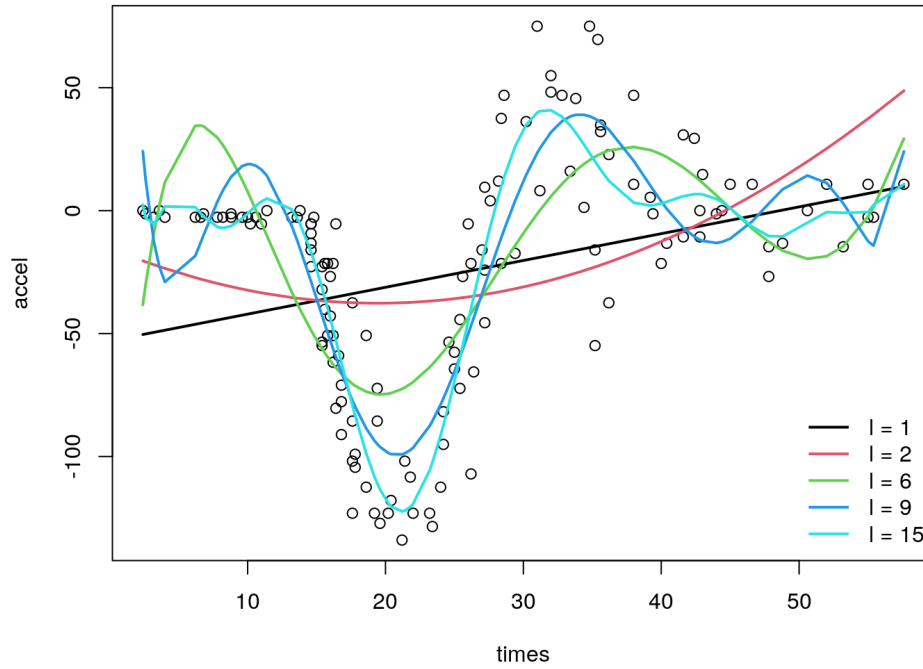


Figura 1.13: Funciones polinomiales, retomado de <https://discdown.org/flexregression/smoothreg.html>

del dominio de variables. Además, pueden surgir inestabilidades numéricas en grados muy altos, en cuyo caso se requiere de re-escalar los datos.

En general, partamos de ecuaciones como:

$$y = a + b_1x^1 + b_2x^2 + \dots + b_lx^l \quad (1.30)$$

La figura 1.13 ilustra diferentes escenarios de ecuaciones como 1.30 con diferentes grados l .

1.2.4. Introducción al Aprendizaje Estadístico

El aprendizaje estadístico juega un rol esencial en muchas áreas de la ciencia, finanzas y la industria. Algunos ejemplos son:

1. Predecir si un paciente—que se encuentra hospitalizado debido a un ataque al corazón—tendrá un segundo ataque. La predicción estará basada en métricas demográficas, de la dieta y de registros clínicos.

2. Predecir el precio de una acción en los siguientes 6 meses; considerando la base de las medidas de desempeño de la compañía y de otros datos económicos.
3. Identificar los números en la digitalización de formas escritas a mano.
4. Identificar los factores de riesgo para el cáncer de próstata, basados en datos clínicos y de otras variables demográficas.

El aprendizaje estadístico comprende a un conjunto de herramientas para modelar y entender conjuntos de datos complejos. También se le conoce como Machine Learning (ML), el cual conjuga el desarrollo reciente en el área de la estadística junto con el crecimiento en paralelo de la computación.

El aprendizaje estadístico considera muchos métodos convencionales y de uso amplio como análisis de regresión, clasificación, árboles de decisión, etc. También se refiere a una amplia gama de herramientas para entender o interpretar datos clasificadas como supervisadas y no supervisadas.

- El aprendizaje estadístico supervisado involucra la construcción de un modelo estadístico para predecir o estimar un resultado (o variable que se puede supervisar) basado en uno o más variables explicativas.
- El aprendizaje no supervisado considera variables explicativas, pero los resultados observados no son una variable explicada que sea susceptible de supervisión.

Ahora introduzcamos un poco de notación. Denotaremos a una variable independiente o explicativa con x_{ij} , si dicha variable es un conjunto de variables acomodadas en un vector utilizaremos $\mathbf{X}_i$; en estos casos denotaremos a un elemento o variable del vector $\mathbf{X}_i$ como x_{ik} , donde $i = 1, 2, \dots, n$ denota a los individuos en la muestra y $k = 1, 2, \dots, K$ denota al número de variables. Por convención diremos que $x_{i1} = 1$ para todo $i = 1, 2, \dots, n$, ya que en dicha variable consideraremos al término constante en la regresión.

Por su parte, los resultados, variables dependientes o variables de respuesta se denotarán como:

- y_i denotará una respuesta que es una cantidad continua
- g_i denotará una respuesta cualitativa, discreta o de grupo

Así, con $\mathbf{X}$, y $\mathbf{Y}$ y $\mathbf{G}$ denotaremos a la matriz y vector columna que contiene a todos los valores de las variables dependientes y de respuesta apiladas para cada uno de los elementos en la muestra indexados con i .

Usaremos mayúsculas como X , Y o G para representar a los aspectos genéricos de las variables. De esta forma, debe ser claro que las letras minúsculas serán empleadas para representar a valores observados de las variables, así el valor observado de la variable k -ésima para el elemento de la muestra i -ésimo en $\mathbf{X}$ será representado como x_{ij} . Finalmente, con la notación $\hat{Y}$ o $\hat{G}$ representaremos a los valores estimados o predecidos.

El aprendizaje estadístico parte del establecimiento de que una variable dependiente Y es una función de un conjunto de variables explicativas $\mathbf{X} = [X_1, X_2, \dots, X_p]$. De esta forma plantearemos:

$$Y = f(\mathbf{X}) + \varepsilon \quad (1.31)$$

Así, el aprendizaje estadístico se trata de un conjunto de aproximaciones para f . ¿Para qué estimar f ? La respuesta es para hacer predicciones y para hacer inferencia. La más común de ambas razones es la predicción, con una predicción de Y podríamos establecer:

$$\hat{Y} = \hat{f}(\mathbf{X}) \quad (1.32)$$

Ajuste y separación del conjunto de datos

Supongamos la variable objetivo Y , un vector de variables explicativas o variables 'input' X y un modelo predictivo $\hat{f}(X)$ que es estimado a partir de un conjunto de entrenamiento τ .

Definiremos la función de pérdida derivada de la estimación y capturada por los errores entre Y y $\hat{f}(X)$ estará dada por:

$$L(Y, \hat{f}(X)) = \begin{cases} (Y - \hat{f}(X))^2 & \text{error cuadrático} \\ |Y - \hat{f}(X)| & \text{error absoluto} \end{cases} \quad (1.33)$$

De esta forma, podemos establecer un error cuadrático como:

$$\begin{aligned} \mathbb{E}[Y - \hat{Y}]^2 &= \mathbb{E}[f(X) + \varepsilon - \hat{f}(X)]^2 \\ &= \mathbb{E}[f(X) - \hat{f}(X)]^2 + \text{Var}(\varepsilon) \end{aligned}$$

Donde $\mathbb{E}[f(X) - \hat{f}(X)]^2$ es el único componente reducible. Para hacer predicciones requerimos de un conjunto de datos de entrenamiento y otro



Figura 1.14: División del conjunto de datos, retomado de Hastie, Tibshirani, y Friedman (2009, p. 222) [HTF09]

más de prueba –en el primero estimamos $f(\cdot)$ y en el segundo hacemos predicciones–.

Finalmente, la condición de inferencia nos permite construir pruebas de hipótesis, estimadores que cumplen ciertas propiedades, intervalos de confianza, etc. Así, dividiremos al conjunto de datos conforme se describe en la Figura 1.14.

Para proporcionar una comprensión precisa de la generalización de nuestro modelo óptimo final, se suele dividir nuestros datos sólo en dos conjuntos de datos: de entrenamiento y de prueba.

- Conjunto de entrenamiento: estos datos se utilizan para desarrollar conjuntos de características, entrenar nuestros algoritmos, ajustar hiperparámetros, comparar modelos y todas las demás actividades necesarias para elegir un modelo final (por ejemplo, el modelo que queremos poner en producción).
- Conjunto de pruebas: una vez elegido un modelo final, estos datos se utilizan para estimar una evaluación insesgada del rendimiento del modelo, a lo que nos referimos como error de generalización.

Acá unos recursos:

<https://mlu-explain.github.io/train-test-validation/>

1.2.5. Método de regresiones restringidas (Shrinkage methods).

Este tipo de métodos permiten:

1. Restringir los posibles valores de las estimaciones del parámetro β a un subconjunto, y
2. Seleccionar variables o regresores dentro de un conjunto de candidatos.

Véamos dos casos particulares: i) una regresión de cresta restringida (ridge regression) y ii) la regresión Lasso (least absolute shrinkage and selection operator).

Regresión Ridge.

La regresión de ridge restringe los coeficientes de la regresión mediante la imposición de una penalización en su magnitud. Los coeficientes estimados por este método resultan de resolver el problema de minimizar los residuales al cuadrado sujeto a que los coeficientes sumen un valor dado. Es decir:

$$\hat{\beta}^{Ridge} = \min_{\beta} \left[\sum_{i=1}^N (y_i - \mathbf{X}_i' \beta)^2 + \lambda \sum_{k=2}^K \beta_k^2 \right] \quad (1.34)$$

Donde $\lambda \geq 0$ y suponemos que β_1 es el término constante de la regresión, el cual no se optimiza.

La idea de la penalización de los parámetros se deriva de que la forma equivalente del problema en la ecuación (1.34) es:

$$\hat{\beta}^{Ridge} = \min_{\beta} \left[\sum_{i=1}^n (y_i - \mathbf{X}_i \beta)^2 \right]$$

Sujeto a:

$$\sum_{k=2}^K \beta_k^2 \leq t$$

Note que la penalización no aplica al término constante, ¿por qué?, para garantizar que la estimación del hiperplano asociado pasa por la media de Y y no por el cero (0).

¿Qué implicaciones tiene para la estimación? Es posible estimar el modelo de la ecuación (1.34) pero ajustando las variables incluidas con la resta de la media. Esto, ya que el término constante se puede estimar considerando que este toma el valor de la media de Y dado por:

$$\bar{Y} = \frac{\sum_{i=1}^n y_i}{n}$$

Para los restantes coeficientes los determinaremos mediante un procedimiento dado por:

$$\min_{\beta_R, \lambda} [(\mathbf{Y} - \mathbf{X} \beta_R)' (\mathbf{Y} - \mathbf{X} \beta_R) + \lambda \beta_R' \beta_R] \quad (1.35)$$

Donde β_R contiene sólo las pendientes. Para determinar un valor estimado debemos resolver el problema descrito en la ecuación (1.35):

$$\begin{aligned}\frac{\partial}{\partial \beta_R} S(\beta_R) &= \frac{\partial}{\partial \beta_R} ((\mathbf{Y} - \mathbf{X}\beta_R)'(\mathbf{Y} - \mathbf{X}\beta_R) + \lambda \beta_R' \beta_R) \\ &= \frac{\partial}{\partial \beta_R} (\mathbf{Y}'\mathbf{Y} - 2\mathbf{Y}'\mathbf{X}\beta_R + \beta_R' \mathbf{X}'\mathbf{X}\beta_R + \lambda \beta_R' \beta_R) \\ &= -2\mathbf{X}'\mathbf{Y} + 2\mathbf{X}'\mathbf{X}\beta_R + 2\lambda \beta_R\end{aligned}$$

Determinando el mínimo:

$$\begin{aligned}-2\mathbf{X}'\mathbf{Y} + 2\mathbf{X}'\mathbf{X}\hat{\beta}_R^{Ridge} + 2\lambda \hat{\beta}_R^{Ridge} &= 0 \\ -\mathbf{X}'\mathbf{Y} + (\mathbf{X}'\mathbf{X} + \lambda \mathbb{I}_{K-1})\hat{\beta}_R^{Ridge} &= 0\end{aligned}$$

Por lo tanto, el estimador estará dado por:

$$\hat{\beta}_R^{Ridge} = (\mathbf{X}'\mathbf{X} + \lambda \mathbb{I}_{K-1})^{-1} \mathbf{X}'\mathbf{Y} \quad (1.36)$$

Así, el problema de la estimación es que tiene 1 grado de libertad, λ , que es un valor de penalización y que resultta arbitrario-determinado de forma iterativa hasta alcanzar uno que minimice la ecuación (1.34).

Regresión Lasso (Least Absolute Shrinkage and Selection Operator).

La regresión Lasso se define por la solución al problema:

$$\hat{\beta}^{Lasso} = \min_{\beta} \left[\sum_{i=1}^n (y_i - \mathbf{X}_i\beta)^2 \right] \quad (1.37)$$

Sujeto a:

$$\sum_{k=2}^K |\beta_k| \leq t$$

Donde $\lambda \geq 0$ y suponemos que β_1 es el término constante de la regresión. El término constante se estima considerando que este toma el valor de la media de Y dado por:

$$\bar{Y} = \frac{\sum_{i=1}^n y_i}{n}$$

La idea de la penalización de los parámetros se deriva de la forma equivalente del problema en la ecuación (1.37) es:

$$\hat{\beta}^{Lasso} = \min_{\beta} \left[\sum_{i=1}^n (y_i - \mathbf{X}_i\beta)^2 + \lambda \sum_{k=2}^K |\beta_k| \right] \quad (1.38)$$

Aplicación de regresión lineal con métodos de restricción (Shrinkage Methods) y regularización de los parámetros.

Ver Python Notebook: 02_Ejemplo_Datos_Inmuebles24.ipynb

1.3. Aplicación de modelos clasificación mediante clustering.

En esta sección abordaremos el caso del clústerin, el cual es un problema que podemos enunciar de esta forma. Supongamos que tenemos un conjunto de entrenamiento $\{x_1, x_2, \dots, x_n\}$ de características respecto del cual queremos construir un grupo o grupos de datos cohesivos denominados clústeres. En este caso, para cada una de las x_i características no tenemos una etiqueta y_i . Esta situación de inexistencia de etiquetas es lo que le da el nombre de aprendizaje no supervisado.

Ahora, analicemos diferentes técnicas de aprendizaje no supervisado.

1.3.1. Análisis de Componentes Principales

El análisis de componentes principales (PCA, por sus siglas en inglés), siguiendo a James, Witten, Hastie y Tibshirani (2013, Cap. 10) [Jam+13], es una herramienta importante y útil de aprendizaje no supervisado para la reducción de dimensiones. También lo podemos ver como un método para identificar un subespacio en el cual los datos se pueden representar, puesto que hay información redundante. El PCA se puede utilizar como una herramienta para el preprocesamiento de datos antes de que se apliquen técnicas de aprendizaje supervisado. El PCA produce una representación de baja dimensión de un conjunto de datos. Encuentra secuencialmente un conjunto de combinaciones lineales de los predictores que tienen una varianza máxima y son ortogonales entre sí.

El análisis de componentes principales asume que las variables individuales pueden ser descritas por una combinación lineal de factores; es decir, esos factores representan completamente la varianza de las variables. El primer componente principal de un conjunto de características denominadas como $X_1, X_2, \dots, X_K$ es la combinación lineal de las características o predictores X :

$$Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \dots + \phi_{K1}X_K \quad (1.39)$$

que tiene la mayor varianza, donde ϕ_{j1} satisface:

$$\sum_{j=1}^K \phi_{j1}^2 = 1 \quad (1.40)$$

Los coeficientes ϕ_{ji} se llaman cargas y se pueden encontrar utilizando una descomposición en valores singulares de la matriz X de dimensión $n \times K$. El segundo componente principal es la combinación lineal de $X_1, \dots, X_K$ que tiene máxima varianza entre todas las combinaciones lineales que son ortogonales con Z_1 . Así, el segundo componente sería:

$$Z_2 = \phi_{12}X_1 + \phi_{22}X_2 + \dots + \phi_{K2}X_K \quad (1.41)$$

donde el coeficiente ϕ_2 es el vector de carga del segundo componente principal, satisfaciendo $\phi_1 \cdot \phi_2 = 0$, es decir, $\phi_1 \perp \phi_2$. Este proceso continúa hasta que se encuentren todos los componentes principales $K^* = \min(n-1, K)$. Aquí, los productos internos $\phi_i \cdot \phi_j = 0$ y $Z_i \cdot Z_j = 0$ para todos $i \neq j$.

El significado de las cargas se puede interpretar de la siguiente manera:

1. El vector de carga ϕ_1 con elementos $\phi_{11}, \phi_{21}, \dots, \phi_{K1}$ define una dirección en el espacio de características a lo largo de la cual los datos varían más.
2. Las direcciones de los componentes principales $\phi_1, \phi_2, \dots, \phi_K$ son la secuencia ordenada de vectores singulares derechos de la matriz X , y las varianzas de los componentes son $1/n$ veces los cuadrados de los valores singulares.
3. El vector de carga ϕ_1 define la línea en el espacio K -dimensional que está más cerca de las n observaciones en términos de distancia euclidiana cuadrada media.

Asumiendo que las variables han sido centradas para tener media cero, la varianza total observada es:

$$\sum_{j=1}^K \text{Var}(X_j) = \sum_{j=1}^K \frac{1}{n} \sum_{i=1}^n x_{ij}^2. \quad (1.42)$$

y la varianza explicada por el m -ésimo componente principal es:

$$\text{Var}(Z_m) = \frac{1}{n} \sum_{i=1}^n z_{im}^2. \quad (1.43)$$

Por lo tanto, la varianza proporcional explicada por el m -ésimo componente principal se da por la cantidad positiva entre 0 y 1.

$$\frac{\sum_{i=1}^n z_{im}^2}{\sum_{j=1}^K \sum_{i=1}^n x_{ij}^2}. \quad (1.44)$$

Una de las desventajas de usar PCA es que las características transformadas Z no son tan intuitivas como las medidas originales X ; reduciendo las posibilidades de interpretación.

1.3.2. Análisis Factorial

El análisis factorial o análisis de factores usa la correlación entre los artículos o elementos individuales para reducirlos a un número pequeño de dimensiones independientes o factores, sin suponer la unidimensionalidad de la escala. La matriz de correlación de los elementos indica cuales exhiben patrones similares de respuesta. Esos elementos son agrupados en factores.

En contraste con el análisis de componentes principales, el análisis de factores o factorial asume que la varianza de la variable puede ser separada en 2 partes. La primera está determinada por la varianza conjunta de todas las variables incluidas en el análisis. La otra parte está determinada por la varianza específica de la variable en cuestion. Bajo el enfoque de análisis de factores, los factores explican solo el primer componente de la varianza.

1.3.3. Clúster

El término clúster refiere a un grupo de individuos u objetos que convergen alrededor de cierto punto y son relativamente cercanos en su posición. En astronomía, existen clústeres de estrellas; en química hay clústeres de átomos. En el campo de la economía en ocasiones se refiere a técnicas que consideran grupos dentro del total de la población. Por ejemplo, las empresas que buscan analizar un grupo de potenciales clientes podrían buscar dividir el total de consumidores en segmentos o clústeres.

El proposito del análisis de clúster es identificar clústeres homogéneos dentro de un conjunto de individuos u objetos heterogéneos. No obstante, hay un mal entendido muy común en el análisis de clúster. Este es la creencia de que existe sólo un tipo de técnica de análisis de clúster. En la realidad, existen muchas técnicas de clustering. Esto se deriva de que existen una amplia variedad de métricas de distancia y algoritmos de agrupamiento que pueden ser usados.

En general, se pueden identificar 2 tipos generales de métodos de clúster:

1. Análisis de agrupamiento del tipo k -Means
2. Análisis de clúster jerárquico

El agrupamiento o clustering se refiere a un conjunto muy amplio de técnicas para encontrar subgrupos, o clústeres, en un conjunto de datos. El objetivo del agrupamiento es encontrar una partición de los datos en grupos distintos de modo que las observaciones dentro de cada grupo sean bastante similares entre sí en algún sentido. Tal sentido de similitud a menudo es una consideración específica del dominio que debe realizarse con base en el conocimiento de los datos que se estudian. La similitud debe estar relacionada con algún resultado vago o posibles múltiples resultados/propósitos.

En las bibliotecas, organizamos los libros por diferentes categorías y subcategorías, mientras que tales selecciones de categorías y subcategorías se basan en las necesidades de los clientes que a menudo no están claramente definidas. En casa, organizamos nuestras cosas en ropa, zapatos, utensilios de cocina y otros para conveniencia cuando los usamos. Por lo tanto, el agrupamiento debe tener algunos propósitos que son difíciles de caracterizar usando una medida de resultado simple. Por esta razón, el aprendizaje supervisado no es aplicable al problema.

Un buen ejemplo de agrupamiento en el comercio sería el agrupamiento para la segmentación del mercado. Supongamos que tenemos acceso a grandes datos (por ejemplo, ingreso medio del hogar, ocupación, distancia desde el área urbana más cercana) de un gran número de personas que pueden o no ser nuestros clientes existentes. Nuestro objetivo es identificar subgrupos de personas que podrían ser más receptivas a una forma particular de publicidad, o agruparlos (en términos de datos) de acuerdo con la probabilidad de comprar un producto en particular.

A diferencia del PCA, que busca una representación de baja dimensión de

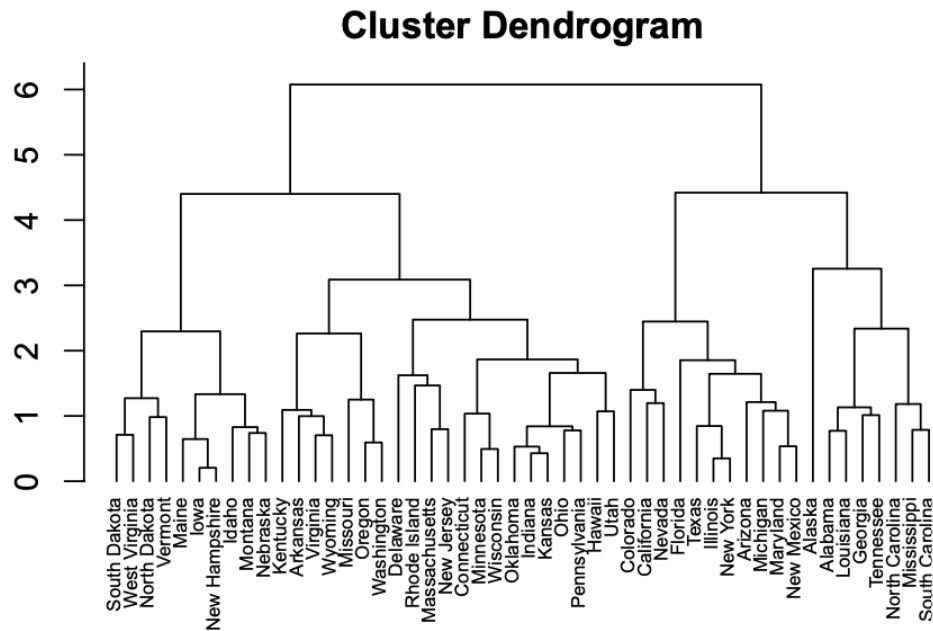


Figura 1.15: US Arrest Hierarchical Clustering, retomado de Chang (2020, p 216) [Cha20]

las observaciones que explica una buena fracción de la varianza, el agrupamiento busca subgrupos homogéneos entre las observaciones.

Hay dos métodos de agrupamiento comúnmente utilizados: el agrupamiento K-medias y el agrupamiento jerárquico. En el agrupamiento K-medias, buscamos particionar las observaciones en un número preespecificado de clústeres, mientras que en el agrupamiento jerárquico no sabemos de antemano cuántos clústeres queremos. En cambio, el agrupamiento jerárquico terminará con una representación visual en forma de árbol de las observaciones, llamada dendrograma, que nos permite ver de una vez el agrupamiento obtenido para cada número posible de clústeres (Figura 1.15).

K-Means Clustering

Algoritmo K-Means

El clústering K-Means es un método de análisis que agrupa observaciones en clústeres. Las principales diferencias entre k-means y el clústering jerárquico

es que los usuarios de k-means deciden el número de clústeres desde el inicio. Es decir, desde un inicio, determinamos una partición inicial que asumimos.

El algoritmo de k-means tiene los siguientes requerimientos:

1. Conocimiento sobre el mejor número de clústeres.
2. Las variables deberían estar transformadas como con la transformación z .

El algoritmo K-medias es uno de los métodos de agrupamiento más simples y populares. Sea $X = \{x_i; i = 1, \dots, n\}$ y el clúster $C = \{c_j; j = 1, \dots, k\}$. Para un k dado, el objetivo del agrupamiento es encontrar C , tal que

$$\min_{c_j; j} \sum_{i=1}^n \|x_i - c_j\| \quad (1.45)$$

El algoritmo K-medias se describe de la siguiente manera:

1. Escoger aleatoriamente k puntos de datos de X como el clúster inicial C .
2. Reasignar todos $x_i \in X$ al promedio de clúster más cercano c_j .
3. Actualizar todos $c_j \in C$ con la media de sus clústeres correspondientes.
4. Repetir los pasos 2 y 3 hasta que las asignaciones de clúster no cambien.

La convergencia del algoritmo está garantizada en un número finito de iteraciones. Sin embargo, cuando la distancia $\sum_{i=1}^n \|x_i - c_j\|$ es una función no convexa, la convergencia puede llevar a un óptimo local. El algoritmo también es sensible a los valores atípicos y podría llevar a algunos clústeres vacíos. Aquí hemos asumido que k es fijo. El problema de agrupamiento con k variable es muy desafiante desde el punto de vista del cálculo.

El agrupamiento K-medias (k-medoids) ajusta exactamente k clústeres como se especifica y la asignación final de clústeres depende de los centros de clústeres iniciales elegidos y del clúster final asignado. La Figura 1.16 ilustra este proceso o algoritmo de determinación de centriodes.

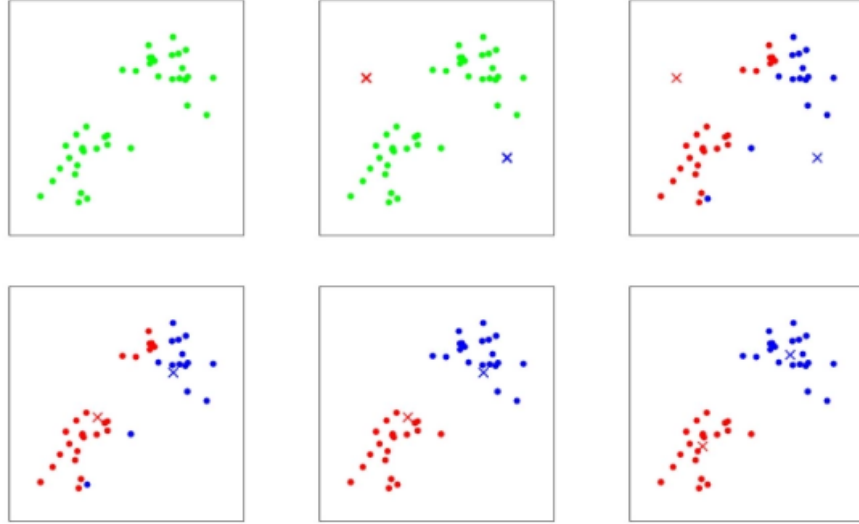


Figura 1.16: K-Means algorithm, retomado de Ng (2023, p 146) [Ng23]

Clustering Jerárquico

El agrupamiento jerárquico busca construir una jerarquía de clústeres. Las estrategias para el agrupamiento jerárquico pueden ser aglomerativas o divisivas. Una estrategia aglomerativa es un enfoque de abajo hacia arriba, es decir, cada observación comienza en su propio clúster, y pares de clústeres se fusionan a medida que se asciende en la jerarquía. Una estrategia divisiva es un enfoque de arriba hacia abajo, es decir, todas las observaciones comienzan en un clúster, y se realizan divisiones de forma recursiva a medida que se desciende en la jerarquía. Por su parte, el agrupamiento divisivo con una búsqueda exhaustiva tiene una complejidad de $O(2^n)$, pero es común utilizar heurísticas más rápidas para elegir las divisiones, como K-Medias. Los resultados del agrupamiento jerárquico generalmente se presentan en un dendrograma.

Para decidir qué clústeres se deben combinar (estrategia aglomerativa) o dónde se debe dividir un clúster (estrategia divisiva), se requiere una medida de disimilitud o similitud entre conjuntos de observaciones, como distancia o enlace entre pares de observaciones. La distancia puede ser Euclidiana, Euclidiana al cuadrado, Manhattan, máxima, Mahalanobis, entre otras. El criterio de enlace puede ser agrupamiento de enlace completo, $\max\{d(a, b); a \in A, b \in B\}$, agrupamiento de enlace simple, $\min\{d(a, b); a \in A, b \in B\}$, o

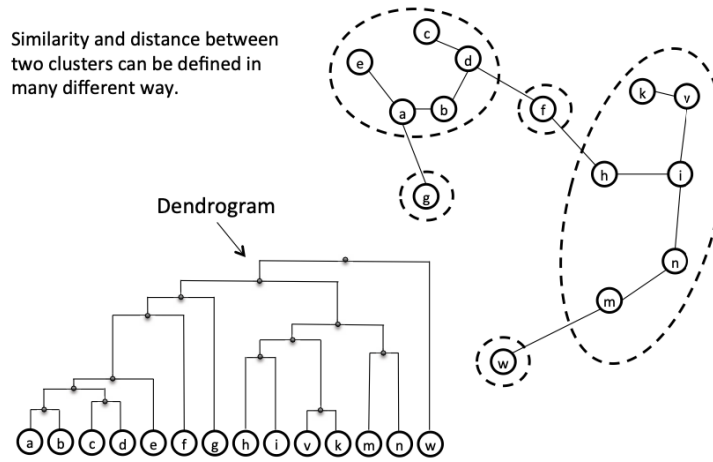


Figura 1.17: Agglomerative Clustering Algorithm and Dendrogram, retomado de Chang (2020, p 219) [Cha20]

agrupamiento de enlace promedio, $\frac{1}{|A| \cdot |B|} \sum_{a \in A} \sum_{b \in B} d(a, b)$. El enlace simple sufre de encadenamiento porque para fusionar dos grupos, solo se necesita que un par de puntos esté cerca, independientemente de todos los demás. Por lo tanto, los clústeres pueden estar demasiado dispersos. En contraste, el enlace completo evita el encadenamiento, pero sufre de hacinamiento. El enlace promedio intenta, utilizando la disimilitud promedio por pares, establecer un equilibrio.

Una lección importante que la experiencia ha mostrado es que las medidas de distancia en el análisis de clúster deben basarse en unidades comparables de medida. Si las características consideradas en el análisis de clúster están en unidades distintas, las variables podrían pasarse por una de las posibles técnicas de unificación de medida. Usualmente, esto se logra mediante la aplicación de la transformación z de todas las variables con el objetivo de estandarizarlas.

Existen 5 métodos de vinculación para clústering jerárquico aglomerativo:

1. Método simple de vinculación que usa la cercanía de 2 observaciones de 2 clústeres como la base de la medida de distancia.
2. Método de vinculación completa que, en contraste, usa las 2 observaciones de 2 clústeres más lejanos como la base de la distancia.

3. Método de vinculación del centroide que calcula el punto medio para cada clúster de sus observaciones.
4. Método de vinculación promedio que determina la media de la distancia entre las observaciones de 3 clústeres.
5. Método de Ward (propuesto por Joe H. Ward en 1963) que vincula clústeres que optimizan un criterio específico: la suma de los errores al cuadrado. Este criterio minimiza la varianza total dentro del clúster. Al igual que otros métodos jerárquicos, este inicia mirando cada observación como si fuera en si mismo un clúster. En este caso, la suma de errores al cuadrado asume el valor de cero, como la media de cada observación. Para calcular la suma de errores para todas las posibles combinaciones de siguientes vínculos se elije los clústeres que tienden a incrementar poco la suma de errores.

1.3.4. Método de K-means con datos de epidemiológicos.

El código del ejemplo se encuentra en GitHub con el nombre:

“07_UnsupervisedClustering/03 Defunciones/Unsupervised_Clustering_Algorithms_KMeans” en GitHub.

1.3.5. Agrupamiento de datos de series temporales

Las observaciones reales de series temporales se pueden agrupar, así como las características extraídas de las mismas, en clusters. Las características se extraen en los dominios de tiempo, frecuencia y ondículas. Esta sección sigue de cerca la discusión de Kotsakos, Trajcevski, Gunopulos y Aggarwal (2014) [Kot+14].

La agrupación mediante características del dominio temporal son autocorrelaciones, autocorrelaciones parciales y correlaciones cruzadas. Mientras que, en el dominio de la frecuencia, se extraen características como el periodograma y las ordenadas espectrales y cepstrales. Por último, las características extraídas en el dominio de wavelets son las transformadas discretas de ondículas, las varianzas de ondículas y las correlaciones de ondículas.

Medidas de Distancia

En muchos casos, se utiliza el análisis de clustering tradicional, es decir, el clustering jerárquico, para agrupar series temporales. En este caso, primero

se define una medida de distancia adecuada que hereda las características dinámicas de la serie temporal para comparar series temporales, y luego se aplica un análisis de clustering jerárquico estándar utilizando la distancia definida.

Sea $\mathbf{X} = \{x_{ij} : i = 1, \dots, I; j = 1, \dots, J\} = \{\mathbf{x}_i = (x_{i1}, \dots, x_{ij}, \dots, x_{iJ})' : i = 1, \dots, I\}$ la matriz de datos donde x_{ij} representa la j -ésima variable observada en el i -ésimo elemento y $\mathbf{x}_i$ representa el vector de la i -ésima observación. La clase más común de medida de distancia utilizada en el análisis de clusters es la clase de distancia de Minkowski.

$${}_r d_{il} = \left[\sum_{j=1}^J |x_{ij} - x_{lj}|^r \right]^{\frac{1}{r}}, \quad r \geq 1. \quad (1.46)$$

Para $r = 1$, tenemos la distancia Manhattan:

$${}_1 d_{il} = \sum_{j=1}^J |x_{ij} - x_{lj}| \quad (1.47)$$

y para $r = 2$, tenemos la distancia Euclidiana:

$${}_2 d_{il} = \left[\sum_{j=1}^J (x_{ij} - x_{lj})^2 \right]^{\frac{1}{2}} \quad (1.48)$$

Las versiones ponderadas de las anteriores distancias de clase Minkowski son:

$${}_r \tilde{d}_{il} = \left[\sum_{j=1}^J w_j^r |x_{ij} - x_{lj}|^r \right]^{\frac{1}{r}}, \quad r \geq 1 \quad (1.49)$$

y entonces,

$$\begin{aligned}
{}_1\tilde{d}_{il} &= \sum_{j=1}^J w_j^1 |x_{ij} - x_{lj}| \quad (r = 1) \quad (\text{distancia Manhattan ponderada}) \\
{}_2\tilde{d}_{il} &= \left[\sum_{j=1}^J w_j^2 (x_{ij} - x_{lj})^2 \right]^{\frac{1}{2}} \quad (r = 2) \quad (\text{distancia Euclidiana ponderada})
\end{aligned} \tag{1.50}$$

dónde w_j ($j = 1, \dots, J$) representa una ponderación adecuada para la j -ésima variable.

Para utilizar las medidas de distancia en clustering jerárquico es útil obtener las distancias para cada par de unidades en forma de una matriz cuadrada. La matriz de distancia Minkowski puede ser representada de la siguiente manera:

$${}_r\mathbf{D} = \left\{ {}_r d_{il} = \left[\sum_{j=1}^J |x_{ij} - x_{lj}|^r \right]^{\frac{1}{r}} : i, l = 1, \dots, I \right\}, \quad r \geq 1 \tag{1.51}$$

Clustering Jerárquico Aglomerativo

Los métodos aglomerativos producen una serie de particiones de los datos: el primero consiste en un cluster I de un solo miembro; el último consiste en un solo cluster que contiene todas las unidades I . El clustering aglomerativo comienza con I clusters, donde cada uno incluye exactamente un punto de los datos. Posteriormente, sucede una serie de operaciones de unión que eventualmente fuerza a todos los objetos en un mismo grupo. El clustering aglomerativo general puede ser resumido por el siguiente procedimiento:

1. Empieza con I clusters únicos (singleton). Calcula la matriz de distancias para los I clusters;
2. En la matriz de distancia, busca la mínima distancia $d(C_p, C_q) = \min_{\substack{1 \leq p, q \leq I \\ p \neq q}} d(C_p, C_q)$, donde $d(\cdot, \cdot)$ es la función de distancia, y por último combina los cluster C_c y $C_{c'}$ para formar el nuevo cluster $C_{cc'}$;

3. Actualiza la matriz de distancia calculando las distancias entre el cluster $C_{cc'}$ y los demás clusters;
4. Repite los pasos 2 y 3 hasta que solo quede un cluster.

La unión de un par de clusters, o la formación de un nuevo cluster depende de como esté definida la función de distancia entre dos clusters. A continuación se presentan de forma breve algunas formas de definir la función de distancia.

- *Método de enlace simple*: La distancia entre un par de clusters está determinada por las dos unidades más cercanas a los diferentes clusters. El clustering por enlace simple tiende a generar clusters alargados, lo que causa el efecto de encadenamiento. Como resultado, dos clusters con propiedades muy diferentes pueden estar conectados debido a la existencia de perturbaciones. Sin embargo, si los cluster están muy separados, el método de enlace simple funciona bien.
- *Método de enlace completo*: A diferencia del enlace simple, el método de enlace completo utiliza la distancia más lejana de un par de objetos para definir la distancia entre grupos.
- *Método de enlace de promedios grupales*: La distancia entre dos clusters se define como el promedio de las distancias entre todos los pares de puntos de datos, cada uno de los cuales proviene de un cluster diferente.
- *Método de enlace de promedios ponderados*: Similar a los promedios grupales, el enlace promedio también se utiliza para calcular la distancia entre dos clusters. La diferencia radica en que las distancias entre el cluster recién formado y el resto se ponderan en función del número de puntos de datos en cada cluster.
- *Método de enlace de centroides*: Dos clusters se unen según la distancia de sus centroides (media).
- *Método de Ward o de la mínima varianza*: El objetivo del método de Ward es minimizar el aumento de la suma intraclase de los errores cuadrados.

Clustering No Jerárquico

A diferencia del clustering jerárquico, que genera un nivel sucesivo de clusters mediante fusiones o divisiones iterativas, el clustering no jerárquico o por particiones asigna un conjunto de puntos de datos en c clusters sin ninguna estructura jerárquica. Este proceso suele acompañar la optimización de una función criterio, normalmente la minimización de una función objetivo que representa la variabilidad interna de los clusters.

Método de clustering por c-Medias

También conocido como k-Medias, el método c-Medias busca una partición óptima de los datos minimizando el criterio de suma de errores al cuadrado con un procedimiento de optimización iterativo, que pertenece a la categoría de algoritmos hill-climbing. El procedimiento básico de clustering de c-Medias se resume de la siguiente manera:

1. Inicie con c particiones aleatorias o basadas en algún conocimiento previo. Calcule los prototipos de clusters (centroides o medias), es decir, calcule la media de cada cluster considerando solo las observaciones que pertenecen a cada cluster.
2. Asigne cada unidad del conjunto de datos al grupo más cercano utilizando una medida de distancia adecuada entre cada par de unidades y centroides.
3. Recalcular los prototipos de clusters (centroides o medias) en función de la partición actual.
4. Repita los pasos 2 y 3 hasta que no haya cambios para cada cluster.

El método de c-Medias formalizado matemáticamente, siguiendo a Kaufman y Rousseeuw (1990) [KR90], es de la siguiente forma:

$$\text{mín} : \sum_{i=1}^I \sum_{c=1}^C u_{ic} d_{ic}^2 = \sum_{i=1}^I \sum_{c=1}^C u_{ic} \|\mathbf{x}_i - \mathbf{h}_c\|^2, \quad (1.52)$$

$$\sum_{c=1}^C u_{ic} = 1, \quad u_{ic} \geq 0, \quad u_{ic} \in \{0, 1\} \quad (1.53)$$

donde u_{ic} indica el grado de pertenencia de la unidad i -ésima al clúster c -ésimo; $u_{ic} \in \{0, 1\}$, es decir, $u_{ic} = 1$ cuando la unidad i -ésima pertenece

al clúster c -ésimo; $u_{ic} = 0$ en caso contrario; $d_{ic}^2 = \|\mathbf{x}_i - \mathbf{h}_c\|^2$ indica la distancia euclidiana al cuadrado entre el objeto i -ésimo y el centroide del clúster c -ésimo.

Método de clustering por c-Medoides.

Las unidades se clasifican en clusters representados por uno de los puntos de datos del cluster. Estos puntos de datos son los prototipos, los llamados medoides. Cada medoide sintetiza la información del cluster y representa las características prototípicas de los clusters, y luego sintetiza las características de las unidades pertenecientes a cada cluster. Siguiendo el método de agrupamiento de c-medoides, minimizamos la función objetivo representada por la suma de la disimilitud de las unidades con sus unidades representativas más cercanas. El método de clustering por c-medoides primero calcula un conjunto de unidades representativas, los medoides. Tras encontrar el conjunto de medoides, cada unidad del conjunto de datos se asigna a las unidades medoides más cercanas. El algoritmo sugerido por Kaufman y Rousseeuw (1990) [KR90] para el método de clustering por c-medoides se desarrolla en dos fases:

Fase 1 (*CONSTRUCCIÓN*): Esta fase selecciona secuencialmente c unidades “centralmente localizadas” para ser utilizadas como medoides iniciales.

Fase 2 (*INTERCAMBIO*): Si la función objetivo puede reducirse intercambiando una unidad seleccionada con una no seleccionada, entonces se realiza el intercambio. Esto se repite hasta que la función objetivo ya no pueda ser disminuida. Luego, considerando un conjunto de I unidades representado por $\mathbf{X}$ (conjunto de las observaciones) y un subconjunto de $\mathbf{X}$ con C unidades representado por $\tilde{\mathbf{X}}$ (conjunto de los medoides), donde $C \ll I$, podemos formalizar el modelo de la siguiente manera:

$$\text{mín} : \sum_{i=1}^I \sum_{c=1}^C u_{ic} \tilde{d}_{ic}^2 = \sum_{i=1}^I \sum_{c=1}^C u_{ic} \|\mathbf{x}_i - \tilde{\mathbf{x}}_c\|^2, \quad (1.54)$$

$$\sum_{c=1}^C u_{ic} = 1, \quad u_{ic} \geq 0, \quad u_{ic} \in \{0, 1\} \quad (1.55)$$

donde u_{ic} indica el grado de pertenencia de la unidad i -ésima al cluster c -ésimo; $u_{ic} \in \{0, 1\}$, es decir, $u_{ic} = 1$ cuando la unidad i -ésima pertenece al

cluster c -ésimo; $u_{ic} = 0$ en caso contrario; $\tilde{d}_{ic}^2 = \|\mathbf{x}_i - \tilde{\mathbf{x}}_c\|^2$ indica la distancia euclidiana al cuadrado entre el objeto i -ésimo y el medoide del clúster c -ésimo.

Algunos criterios útiles de validez para determinar el número de clusters son los siguientes:

Criterio de Calinski y Harabasz

Calinski y Harabasz (1974) sugieren tomar el valor de g , el número de clústeres, el cual corresponde al valor máximo de CH_C :

$$CH_C = \frac{\text{trace}(\mathbf{B})}{(C-1)} : \frac{\text{trace}(\mathbf{W})}{(I-C)}.$$

donde $\mathbf{B}$ es la matriz de dispersión entre grupos y $\mathbf{W}$ es la matriz de dispersión dentro de los grupos. La evaluación de este criterio en un número dado de grupos, C , requiere conocer la pertenencia a los grupos para determinar las matrices $\mathbf{B}$ y $\mathbf{W}$. En general, el número de grupos elegido depende del método de clúster utilizado.

Criterio de la silueta

Este criterio fue propuesto por Rousseeuw (1987). Considere una unidad $i \in (1, \dots, I)$ perteneciente al cluster $p \in (1, \dots, C)$. Por ejemplo, mediante un algoritmo de clustering *c-Medias*, esto significa que la unidad i -ésima está más cerca del centroide del cluster p -ésimo que de cualquier otro centroide.

Sea la distancia promedio (euclidiana al cuadrado) de la unidad i -ésima a todas las demás unidades que pertenecen al clúster p denotada por a_{ip} . Asimismo, sea la distancia promedio de esta unidad a todas las unidades que pertenecen a otro clúster q , $q \neq p$, llamada d_{iq} .

Finalmente, sea b_{ip} el mínimo d_{iq} calculado sobre $q = 1, \dots, C$, $q \neq p$, lo cual representa la disimilitud de la unidad i -ésima con respecto a su cluster vecino más cercano.

Entonces, la silueta del objeto i -ésimo se define de la siguiente manera:

$$S_i = \frac{b_{ip} - a_{ip}}{\max\{a_{ip}, b_{ip}\}} \quad (1.56)$$

donde el denominador es un término de normalización. Mientras mayor sea el valor de S_i , mejor es la asignación de la unidad i -ésima al cluster c -ésimo. La silueta, definida como el promedio de S_i para $i = 1, \dots, I$, es:

$$SIL = \frac{1}{I} \sum_{i=1}^I S_i \quad (1.57)$$

La mejor partición se logra cuando la silueta se maximiza, lo cual implica minimizar la distancia intra-cluster (a_{ip}) mientras se maximiza la distancia inter-cluster (b_{ip}).

1.3.6. Método de K-means para series de tiempo.

Las distancias de clase Minkowski (Euclidiana, Manhattan) que vimos arriba comparan dos series punto a punto, en el mismo instante j : la distancia entre $\mathbf{x}_i$ y $\mathbf{x}_l$ solo usa x_{ij} contra x_{lj} , nunca x_{ij} contra $x_{l,j+1}$ o $x_{l,j-1}$. Esto funciona bien cuando dos series están perfectamente alineadas en el tiempo, pero falla cuando dos series tienen la **misma forma** pero desfasada –por ejemplo, dos perfiles de precio con el mismo patrón diario, pero donde uno alcanza su pico una hora antes que el otro– ya que la distancia Euclidiana penalizaría ese desfase como si las series fueran muy distintas, aunque su forma sea casi idéntica.

Dynamic Time Warping (DTW). DTW resuelve este problema permitiendo una alineación **elástica** entre las dos series: en vez de comparar siempre x_{ij} con x_{lj} , permite que cada punto de una serie se alinee con uno o varios puntos cercanos de la otra, buscando la alineación (el *warping path*) que minimiza la distancia acumulada. Formalmente, dadas dos series $\mathbf{x} = (x_1, \dots, x_T)$ y $\mathbf{y} = (y_1, \dots, y_S)$, DTW se calcula con la siguiente recurrencia de programación dinámica sobre una matriz de costos acumulados $\gamma(t, s)$:

$$\gamma(t, s) = (x_t - y_s)^2 + \min \{ \gamma(t-1, s), \gamma(t, s-1), \gamma(t-1, s-1) \} \quad (1.58)$$

con $\gamma(1, 1) = (x_1 - y_1)^2$ como condición inicial. El valor final $\gamma(T, S)$ es la distancia DTW entre las dos series, y el camino de mínimo costo que la produjo es el warping path. Siguiendo a Kotsakos, Trajcevski, Gunopulos y Aggarwal (2014) [Kot+14] –y al trabajo original de Berndt y Clifford (1994) [BC94], que introdujo DTW para minería de series de tiempo–, esta flexibilidad tiene un costo computacional real: calcular γ completa para un solo par de series de longitud T toma $O(T^2)$ operaciones (contra $O(T)$ de la distancia Euclidiana), lo cual se vuelve costoso rápidamente conforme T crece –es buena práctica, antes de correr DTW sobre miles de pares de series largas, evaluar si se puede

trabajar con una versión resumida de cada serie (por ejemplo, promedios por periodo) sin perder la forma que nos interesa comparar.

Soft-DTW. Una limitación adicional de DTW es que el operador mín en la ecuación (1.58) no es diferenciable, lo que impide usar DTW directamente dentro de algoritmos que requieren gradientes (por ejemplo, para calcular un promedio o *baricentro* representativo de varias series de un mismo cluster). Soft-DTW (Cuturi y Blondel, 2017) [CB17] resuelve esto reemplazando el mín por una versión suavizada, el *soft-min*, controlada por un parámetro $\gamma > 0$:

$$\text{mín}^\gamma\{a_1, \dots, a_n\} = -\gamma \log \sum_{i=1}^n e^{-a_i/\gamma} \quad (1.59)$$

Cuando $\gamma \rightarrow 0$, mín^γ converge al mínimo exacto (recuperamos DTW estándar); valores de γ más grandes suavizan más la alineación. Esta suavidad hace a soft-DTW diferenciable (útil para calcular baricentros y para usarla como función de pérdida en modelos entrenables), a costa de un cómputo todavía más pesado que el DTW estándar –razón de más para resumir o reducir la longitud de las series antes de aplicarlo a conjuntos de datos grandes.

El código del ejemplo se encuentra en GitHub con el nombre:

“08_Unsupervised_Clustering_Time_Series” en GitHub.

Para la recolección de datos se usan:

“03_Ejemplo_Conexión_ServicioWeb” y “04_Generando_Datos_PML” en GitHub.

1.4. Aplicación de modelos de regresión de respuesta binaria

1.4.1. Métodos de estimación basados en verosimilitud

Introducción

En esta sección, discutiremos el tratamiento general para la estimación por máxima verosimilitud. La diferencia significativa en estos casos es que asumiremos una distribución subyacente. Noten que los metodos utilizados anteriormente no requieren de que asumamos una distribución particular para realizar la estimación.

El planteamiento general del método de máxima verisimilitud considera un conjunto de observaciones independientes e idénticamente distribuidas; descritas por la familia: $\{y_i, \mathbf{x}_i : y_i \in \mathbb{R}, \mathbf{x}_i \in \mathbb{R}^K\}$.

En estos casos también supondremos que buscamos estimar la regresión de y_i en $\mathbf{x}_i$ y no el caso contrario. Por lo tanto, el método requiere de la función de densidad condicional de y_i en $\mathbf{x}_i$, es decir:

$$f(y_i|\mathbf{x}_i)$$

Así, al método en realidad se le conoce como método de máxima verosimilitud condicional. Su aplicación es amplia en casos como:

1. Modelos lineales: regresión de una sola ecuación, sistemas de ecuaciones, modelos de datos panel y modelos generalizados con heterocedasticidad y autocorrelación
2. Modelos no lineales: respuesta binaria, respuesta categórica, respuesta multinivel
3. Modelos de conteo y duración

Ejemplo. Sea y_i^* una variable aleatoria latente que se pueda expresar como:

$$y_i^* = \mathbf{x}_i \boldsymbol{\theta} + \varepsilon_i$$

Donde ε_i es independiente de $\mathbf{x}_i$, el cual es un vector de $1 \times K$ que contiene un término constante. $\boldsymbol{\theta}$ es un vector de $K \times 1$ parámetros a estimar. Finalmente, asumiremos que:

$$\varepsilon_i \sim Normal(0, 1)$$

Supongamos que en lugar de y_i^* nosotros sólo podemos observar la variable binaria indicadora:

$$y_i = \begin{cases} 1 & \text{si } y_i^* > 0 \\ 0 & \text{si } y_i^* \leq 0 \end{cases}$$

En este caso, la pregunta que pretendemos responder sería: ¿cuál es la probabilidad de que $y_i = 1$ condicional en que tenemos el vector $\mathbf{x}_i$?

$$\begin{aligned} P(y_i = 1) &= P(y_i^* > 0 | \mathbf{x}_i) \\ &= P(\mathbf{x}_i \boldsymbol{\theta} + \varepsilon_i > 0 | \mathbf{x}_i) \\ &= P(\varepsilon_i > -\mathbf{x}_i \boldsymbol{\theta} | \mathbf{x}_i) \end{aligned}$$

Dada la distribución ε_i , entonces:

$$P(\varepsilon_i > -\mathbf{x}_i\boldsymbol{\theta}|\mathbf{x}_i) = \int_{-\mathbf{x}_i\boldsymbol{\theta}}^{\infty} f(\varepsilon_i|\mathbf{x}_i)d\varepsilon_i$$

Sabemos que toda función de densidad cumple con:

$$\begin{aligned} 1 &= \int_{-\infty}^{\infty} f(\varepsilon_i|\mathbf{x}_i)d\varepsilon_i \\ &= \int_{-\infty}^{-\mathbf{x}_i\boldsymbol{\theta}} f(\varepsilon_i|\mathbf{x}_i)d\varepsilon_i + \int_{-\mathbf{x}_i\boldsymbol{\theta}}^{\infty} f(\varepsilon_i|\mathbf{x}_i)d\varepsilon_i \\ &= \int_{-\infty}^{-\mathbf{x}_i\boldsymbol{\theta}} f(\varepsilon_i|\mathbf{x}_i)d\varepsilon_i + \int_{-\infty}^{\mathbf{x}_i\boldsymbol{\theta}} f(\varepsilon_i|\mathbf{x}_i)d\varepsilon_i \\ &= \Phi(-\mathbf{x}_i\boldsymbol{\theta}) + \Phi(\mathbf{x}_i\boldsymbol{\theta}) \\ &= (1 - \Phi(\mathbf{x}_i\boldsymbol{\theta})) + \Phi(\mathbf{x}_i\boldsymbol{\theta}) \\ &= P(\varepsilon_i < -\mathbf{x}_i\boldsymbol{\theta}|\mathbf{x}_i) + P(\varepsilon_i > -\mathbf{x}_i\boldsymbol{\theta}|\mathbf{x}_i) \end{aligned}$$

De esta forma tenemos dos ecuaciones:

$$\begin{aligned} P(y_i = 1|\mathbf{x}_i) &= P(\varepsilon_i > -\mathbf{x}_i\boldsymbol{\theta}|\mathbf{x}_i) = \Phi(\mathbf{x}_i\boldsymbol{\theta}) \\ P(y_i = 0|\mathbf{x}_i) &= P(\varepsilon_i < -\mathbf{x}_i\boldsymbol{\theta}|\mathbf{x}_i) = 1 - \Phi(\mathbf{x}_i\boldsymbol{\theta}) \end{aligned}$$

Estas dos funciones de probabilidad son independientes. De esta forma, la función de densidad condicional de y_i será:

$$\begin{aligned} f(y_i|\mathbf{x}_i) &= P(y_i = 1|\mathbf{x}_i)^{y_i} \cdot P(y_i = 0|\mathbf{x}_i)^{1-y_i} \\ &= \Phi(\mathbf{x}_i\boldsymbol{\theta})^{y_i} \cdot (1 - \Phi(\mathbf{x}_i\boldsymbol{\theta}))^{1-y_i} \end{aligned}$$

Marco general de estimación por máxima verosimilitud

Sea $f(y_i|\mathbf{x}_i)$ la función de densidad condicional de y_i dado $\mathbf{x}_i$. Sea $\boldsymbol{\theta}$ un conjunto de parámetros de la función. Entonces la función de densidad conjunta de variables aleatorias independientes $\{y_i : y_i \in \mathbb{R}\}$ dados los valores $\{\mathbf{x}_i : \mathbf{x}_i \in \mathbb{R}^K\}$ estará dada por:

$$\Pi_{i=1}^n f(y_i|\mathbf{x}_i; \boldsymbol{\theta}) = f(y_1, y_2, \dots, y_n|\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n; \boldsymbol{\theta}) = L(\boldsymbol{\theta}) \quad (1.60)$$

A la ecuación (1.60) se le conoce como ecuación de verosimilitud. El problema de máxima verosimilitud entonces será:

$$\max_{\boldsymbol{\theta} \in \Theta} \Pi_{i=1}^n f(y_i|\mathbf{x}_i; \boldsymbol{\theta}) = \max_{\boldsymbol{\theta} \in \Theta} L(\boldsymbol{\theta}) \quad (1.61)$$

Dado que el logaritmo natural es una transformación monótona, podemos decir que el problema de la ecuación (1.61) es equivalente a:

$$\max_{\boldsymbol{\theta} \in \Theta} \ln L(\boldsymbol{\theta}) = \max_{\boldsymbol{\theta} \in \Theta} \ln \prod_{i=1}^n f(y_i | \mathbf{x}_i; \boldsymbol{\theta}) = \max_{\boldsymbol{\theta} \in \Theta} \sum_{i=1}^n \ln f(y_i | \mathbf{x}_i; \boldsymbol{\theta}) \quad (1.62)$$

Para solucionar el problema se tiene que determinar las condiciones de primer y segundo orden, las cuales serán:

$$\frac{\partial}{\partial \boldsymbol{\theta}} \ln L(\boldsymbol{\theta}) = \nabla \ln L(\boldsymbol{\theta}) \quad (1.63)$$

$$\frac{\partial^2}{\partial^2 \boldsymbol{\theta}} \ln L(\boldsymbol{\theta}) = \frac{\partial}{\partial \boldsymbol{\theta}} \ln L(\boldsymbol{\theta}) \cdot \frac{\partial}{\partial \boldsymbol{\theta}} \ln L(\boldsymbol{\theta}') = H(\boldsymbol{\theta}) \quad (1.64)$$

La solución estará dada por aquel valor de $\hat{\boldsymbol{\theta}}$ que hace:

$$\frac{\partial}{\partial \boldsymbol{\theta}} \ln L(\hat{\boldsymbol{\theta}}) = 0$$

A su vez, la varianza será aquella que resulta de:

$$Var[\hat{\boldsymbol{\theta}} | \mathbf{X}] = (-\mathbb{E}_{\hat{\boldsymbol{\theta}}}[H(\boldsymbol{\theta})])^{-1}$$

Pruebas de hipótesis

1. Para el caso de pruebas de hipótesis sobre cada uno de los coeficientes estimados $\hat{\theta}_i$, dada la construcción de la función de verosimilitud y que se trata de grandes muestras, la función de distribución de ese coeficiente será una normal, por lo que las hipótesis planteadas serán asumiendo:

$$Z = \frac{\hat{\theta}_i - \theta_i}{\sqrt{Var(\hat{\theta}_i)}} \sim N(0, Var(\hat{\theta}_i)) \quad (1.65)$$

2. Para pruebas globales, utilizaremos la prueba de Wald:

$$W = c(\hat{\boldsymbol{\theta}})'(\hat{C} Var(\hat{\boldsymbol{\theta}}) \hat{C}')^{-1} c(\hat{\boldsymbol{\theta}}) \sim \chi_{(Q)}^2 \quad (1.66)$$

Donde $c(\hat{\boldsymbol{\theta}})$ es una función de los coeficientes, $\hat{C}$ es el jacobiano de $c(\hat{\boldsymbol{\theta}})$, Q es el número de restricciones.

3. Alternativamente, para pruebas sobre los coeficientes, podremos utilizar pruebas de razón de verosimilitud:

$$LR = 2[\ln L(\hat{\boldsymbol{\theta}}_0) - \ln L(\hat{\boldsymbol{\theta}}_R)] \sim \chi_{(Q)}^2 \quad (1.67)$$

1.4.2. Estimación de modelos no lineales

Modelos de respuesta binaria

Planteamiento general. En el caso de modelos de respuesta binaria, asumiremos que y_i es una variable aleatoria que toma sólo valores de 0 y 1. Los ejemplos pueden ser amplios. Decisiones sobre consumir o no, pagar o no, ir a una escuela determinanda A o B, viajar por tierra o aire. En general, diremos que 1 es éxito y 0 es fracaso.

Como en el caso de modelos lineales, diremos que y_i es la variable dependiente y $\mathbf{x}_i = (x_{i1}, x_{i2}, \dots, x_{iK})$ un vector de variables independientes o explicativas. Así, en los modelos no lineales de respuesta binaria estaremos interesados en las probabilidades:

$$P(y_i = 1|\mathbf{x}_i) = P(y_i = 1|x_{i1}, x_{i2}, \dots, x_{iK})$$

Notemos que, en este caso, hemos planteado una nueva hipótesis de cómo se puede modelar el comportamiento de y_i . Es decir, estamos asumiendo que la respuesta 0 o 1 se puede modelar como el resultado de un modelo probabilístico.

Dicho lo anterior, y de forma similar al de la regresión lineal, podemos establecer el efecto marginal para el caso de una variable independiente x_j continua:

$$EMg_j = \frac{\partial}{\partial x_j} P(y_i = 1|x_{i1}, x_{i2}, \dots, x_{iK})$$

Cuando la variable independiente es una variable dummy:

$$\begin{aligned} EMg_j &= P(y_i = 1|x_{i1}, x_{i2}, \dots, x_{ij} = 1, \dots, x_{iK}) - \\ &\quad P(y_i = 1|x_{i1}, x_{i2}, \dots, x_{ij} = 0, \dots, x_{iK}) \end{aligned}$$

Este tipo de modelos se basan en el modelo básico de Bernoulli, cuya función de densidad de probabilidad condicional y podemos plantearlo como:

$$\begin{aligned} P(y_i = 1|\mathbf{x}_i) &= p(x) \\ P(y_i = 0|\mathbf{x}_i) &= 1 - p(x) \\ Var(y_i|\mathbf{x}_i) &= p(x) \cdot (1 - p(x)) \end{aligned}$$

Algunos planteamientos adicionales

Cuando la variable dependiente es binaria (0 y 1) no se puede implementar una ecuación lineal de la forma tradicional, ya que no es posible determinar una ecuación:

$$y_i = \mathbf{x}_i\boldsymbol{\beta} + \varepsilon_i$$

Por otro lado, tambien enfrentaremos el mismo problema con modelos de probabilidad lineal:

$$P(y_i = 1|\mathbf{x}_i) = \mathbf{x}_i\boldsymbol{\beta} + \varepsilon_i$$

Y:

$$P(y_i = 0|\mathbf{x}_i) = \mathbf{x}_i\boldsymbol{\beta} + \varepsilon_i$$

En ambos casos es difícil lograr la estimación.

Modelos Logit y Probit

Este tipo de modelos suponen que existe una variable latente que se puede expresar como una ecuación lineal dada por:

$$y_i^* = \mathbf{x}_i\boldsymbol{\beta} + \varepsilon_i$$

Donde ε_i es una variable aleatoria con función de densidad con media cero y distribución simétrica alrededor de cero. Dado lo anterior, para nosotros sólo es visible que:

$$y_i = \begin{cases} 1 & \text{si } y_i^* > 0 \\ 0 & \text{si } y_i^* < 0 \end{cases}$$

De esta forma, tenemos una estructura de la probabilidad dada por:

$$P(y_i = 1|\mathbf{x}_i) = P(\mathbf{x}_i\boldsymbol{\beta} + \varepsilon_i > 0) = P(\varepsilon_i > -\mathbf{x}_i\boldsymbol{\beta}) = G(\mathbf{x}_i\boldsymbol{\beta})$$

$$P(y_i = 0|\mathbf{x}_i) = P(\mathbf{x}_i\boldsymbol{\beta} + \varepsilon_i < 0) = P(\varepsilon_i < -\mathbf{x}_i\boldsymbol{\beta}) = 1 - G(\mathbf{x}_i\boldsymbol{\beta})$$

Donde $\mathbf{x}_i$ es un vector de dimensión $K \times 1$ que contiene al menos el término constante y $\boldsymbol{\beta}$ es un vector de parámetros a estimar, de forma que asumiremos:

$$\mathbf{x}_i\boldsymbol{\beta} = \beta_1 + x_{i2}\beta_2 + \dots + x_{iK}\beta_K$$

Asumiremos que $G(\cdot)$ es una función de densidad acumulada de forma que:

$$0 < G(\mathbf{x}_i\boldsymbol{\beta}) < 1, \forall \mathbf{x}_i\boldsymbol{\beta} \in \mathbb{R}$$

En este caso, utilizaremos dos modelos que dependen de la forma funcional de $G(\cdot)$ que está determinada por la distribución de ε_i . De esta forma, tendremos dos modelos: Probit y Logit.

1. **Modelo Probit**, cuando la función de distribución de probabilidad es una normal:

$$G(\mathbf{x}_i\boldsymbol{\beta}) = \Phi(\mathbf{x}_i\boldsymbol{\beta}) = \int_{-\infty}^{\mathbf{x}_i\boldsymbol{\beta}} \phi(\nu)d\nu$$

Donde $\phi(\cdot)$ es la densidad de probabilidad normal estándar:

$$\phi(\mathbf{x}_i\boldsymbol{\beta}) = \frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}(\mathbf{x}_i\boldsymbol{\beta})^2}$$

2. **Modelo Logit**, cuando la función de distribución tiene forma analítica cerrada del tipo:

$$G(\mathbf{x}_i\boldsymbol{\beta}) = \Lambda(\mathbf{x}_i\boldsymbol{\beta}) = \frac{e^{\mathbf{x}_i\boldsymbol{\beta}}}{1 + e^{\mathbf{x}_i\boldsymbol{\beta}}} = \frac{1}{1 + e^{-\mathbf{x}_i\boldsymbol{\beta}}}$$

Donde $\lambda(\cdot)$ es la función de densidad acumulada logística. Esta función tiene más aplicaciones relacionadas con la función sigmoide que usaremos en redes neuronales.

Sin importar el modelo que estemos ocupando, la forma de interpretar el modelo es mediante el efecto marginal, cuando x_j sea una variable continua:

$$EMg_j = \frac{\partial}{\partial x_j}P(y = 1|\mathbf{x}_i) = \frac{\partial}{\partial x_j}G(\mathbf{x}_i\boldsymbol{\beta}) = g(\mathbf{x}_i\boldsymbol{\beta})\beta_j$$

Por su parte, cuando x_j sea una variable dicotómica:

$$\begin{aligned} EMg_j &= P(y = 1|\mathbf{x}_i, x_j = 1) - P(y = 1|\mathbf{x}_i, x_j = 0) \\ &= G(\mathbf{x}_i\boldsymbol{\beta}|x_j = 1) - G(\mathbf{x}_i\boldsymbol{\beta}|x_j = 0) \\ &= G(\beta_1 + x_2\beta_2 + \dots + \beta_j + \dots + x_K\beta_K) \\ &\quad - G(\beta_1 + x_2\beta_2 + \dots + 0 + \dots + x_K\beta_K) \end{aligned}$$

Estimación

Supongamos n observaciones de una variable aleatoria o una muestra de variables independientes e idénticamente distribuidas. En estos casos, la función de densidad para cada y_i , $i = 1, 2, \dots, n$, estará dada por:

$$f(y_i|\mathbf{x}_i; \boldsymbol{\beta}) = [G(\mathbf{x}_i\boldsymbol{\beta})]^{y_i} [1 - G(\mathbf{x}_i\boldsymbol{\beta})]^{1-y_i}, \text{ donde } y_i = 0, 1$$

En estos términos, la función de verosimilitud estará dada por:

$$\begin{aligned} L(\boldsymbol{\beta}) &= \prod_{i=1}^n f(y_i|\mathbf{x}_i; \boldsymbol{\beta}) \\ &= \prod_{i=1}^n [G(\mathbf{x}_i\boldsymbol{\beta})]^{y_i} [1 - G(\mathbf{x}_i\boldsymbol{\beta})]^{1-y_i} \end{aligned}$$

En versión logaritmica:

$$\begin{aligned} \ln L(\boldsymbol{\beta}) &= \sum_{i=1}^n \ln(f(y_i|\mathbf{x}_i; \boldsymbol{\beta})) \\ &= \sum_{i=1}^n y_i \ln[G(\mathbf{x}_i\boldsymbol{\beta})] + \sum_{i=1}^n (1 - y_i) \ln[1 - G(\mathbf{x}_i\boldsymbol{\beta})] \end{aligned}$$

Las condiciones de primer orden serán:

$$\frac{\partial}{\partial \boldsymbol{\beta}} \ln L(\boldsymbol{\beta}) = \sum_{i=1}^n y_i \frac{g(\mathbf{x}_i\boldsymbol{\beta})}{G(\mathbf{x}_i\boldsymbol{\beta})} \mathbf{x}_i' - \sum_{i=1}^n (1 - y_i) \frac{g(\mathbf{x}_i\boldsymbol{\beta})}{1 - G(\mathbf{x}_i\boldsymbol{\beta})} \mathbf{x}_i'$$

Para llegar a las condiciones de segundo orden conviene primero reescribir la condición de primer orden de forma compacta. Sumando los dos términos de la ecuación anterior sobre un denominador común $G(\mathbf{x}_i\boldsymbol{\beta})[1 - G(\mathbf{x}_i\boldsymbol{\beta})]$ y usando que $y_i[1 - G(\mathbf{x}_i\boldsymbol{\beta})] - (1 - y_i)G(\mathbf{x}_i\boldsymbol{\beta}) = y_i - G(\mathbf{x}_i\boldsymbol{\beta})$, la condición de primer orden se reduce a:

$$\frac{\partial}{\partial \boldsymbol{\beta}} \ln L(\boldsymbol{\beta}) = \sum_{i=1}^n \frac{[y_i - G(\mathbf{x}_i\boldsymbol{\beta})] g(\mathbf{x}_i\boldsymbol{\beta})}{G(\mathbf{x}_i\boldsymbol{\beta})[1 - G(\mathbf{x}_i\boldsymbol{\beta})]} \mathbf{x}_i'$$

Esta forma –el residual generalizado $[y_i - G(\mathbf{x}_i\boldsymbol{\beta})]$ ponderado por $g/[G(1 - G)]$ – es la que sigue Greene (2012, cap. de modelos de respuesta binaria) [Gre12]. Derivando una vez más respecto de $\boldsymbol{\beta}$, para una función índice G arbitraria (Logit, Probit, etc.) el Hessiano exacto también incluye un término con $g'(\cdot)$, la derivada de la densidad. Sin embargo, lo que necesitamos para el

error estándar es la **matriz de información esperada**, $-\mathbb{E}[H(\boldsymbol{\beta})]$, y bajo especificación correcta $\mathbb{E}[y_i - G(\mathbf{x}_i\boldsymbol{\beta})|\mathbf{x}_i] = 0$, lo que anula el término que involucra a y_i y a $g'(\cdot)$, dejando:

$$-\mathbb{E}[H(\boldsymbol{\beta})] = \sum_{i=1}^n \frac{g(\mathbf{x}_i\boldsymbol{\beta})^2}{G(\mathbf{x}_i\boldsymbol{\beta})[1 - G(\mathbf{x}_i\boldsymbol{\beta})]} \mathbf{x}_i' \mathbf{x}_i$$

Nótese que y_i **no** aparece en esta expresión: la matriz de información esperada solo depende de las probabilidades ajustadas $G(\mathbf{x}_i\boldsymbol{\beta})$, no de los resultados observados. Por lo tanto:

$$\begin{aligned} Var(\hat{\boldsymbol{\beta}}) &= [-\mathbb{E}[H(\boldsymbol{\beta})]]^{-1} \\ &= \left[\sum_{i=1}^n \frac{g(\mathbf{x}_i\boldsymbol{\beta})^2}{G(\mathbf{x}_i\boldsymbol{\beta})[1 - G(\mathbf{x}_i\boldsymbol{\beta})]} \mathbf{x}_i' \mathbf{x}_i \right]^{-1} \end{aligned}$$

1.4.3. Modelos de respuesta multinomial y ordenada

En esta sección, analizaremos modelos de respuesta discreta con más de 2 resultados posibles, los cuales se dividen en:

1. Modelos de respuesta multinomial (o no ordenada): son modelos conocidos como de respuesta nominal, donde los valores de los distintos resultados son arbitrarios y no tienen un efecto en la estimación. Ejemplos de estos modelos son casos de selección de seguro, lugar de hospedaje, clasificación de correos o documentos, etc.
2. Modelos de respuesta ordenada: en estos modelos se asigna a cada nivel o tipo de respuesta un valor no arbitrario. Por ejemplo, modelos de calificación de crédito, modelos de preferencia de bienes, clasificación de respuestas abiertas en encuestas, etc.

Modelos de respuesta multinomial

Logit multinomial

Este primer modelo aplica en situaciones en las que la unidad de respuesta o elección depende de las características individuales de los elementos de la muestra, pero no de los atributos de la elección.

Dado lo anterior, definiremos el modelo en términos de variables aleatorias que representan a la población subyacente. Sea y_i una variable aleatoria que toma valores en un conjunto $\{0, 1, 2, \dots, J\}$. Sea $\mathbf{x}_i$ un conjunto de valores o regresores para $i = 1, 2, \dots, n$.

Como en el modelo de respuesta binaria, estimaremos el efecto de una variable en la probabilidad de ocurrencia de uno de los valores de respuesta. Dichas variables de respuesta estarán dados por:

$$P(y_i = j | \mathbf{x}_i), \text{ para } j = 1, 2, \dots, J$$

Dado que la suma de probabilidades debe ser 1, se suele tomar como práctica estimar $P(y_i = 0 | \mathbf{x}_i)$ como diferencia del resto de los casos $j = \{1, 2, \dots, J\}$. Así, sea $\mathbf{x}_i$ un vector de dimensión $1 \times K$. De esta forma, el modelo Logit tendrá como probabilidad de cada una de las respuestas a:

$$P(y_i = j | \mathbf{x}_i) = \frac{e^{\mathbf{x}_i \beta_j}}{1 + e^{\mathbf{x}_i \beta_1} + e^{\mathbf{x}_i \beta_2} + \dots + e^{\mathbf{x}_i \beta_J}}$$

Donde cada una de las β_j , $j = 1, \dots, J$ es de dimensión $K \times 1$. A esta forma funcional de probabilidad también se le conoce como función *softmax*. Es fácil observar que si la suma de probabilidades es 1, entonces, la probabilidad de $y = 0 | \mathbf{x}_i$ estará dada por:

$$\begin{aligned} P(y_i = 0 | \mathbf{x}_i) &= \frac{e^0}{1 + e^{\mathbf{x}_i \beta_1} + e^{\mathbf{x}_i \beta_2} + \dots + e^{\mathbf{x}_i \beta_J}} \\ &= \frac{1}{1 + e^{\mathbf{x}_i \beta_1} + e^{\mathbf{x}_i \beta_2} + \dots + e^{\mathbf{x}_i \beta_J}} \end{aligned}$$

De esta forma, tenemos:

$$\begin{aligned} 1 &= \sum_{j=0}^J P(y_i = j | \mathbf{x}_i) \\ &= \frac{1}{1 + e^{\mathbf{x}_i \beta_1} + e^{\mathbf{x}_i \beta_2} + \dots + e^{\mathbf{x}_i \beta_J}} \\ &\quad + \frac{e^{\mathbf{x}_i \beta_1}}{1 + e^{\mathbf{x}_i \beta_1} + e^{\mathbf{x}_i \beta_2} + \dots + e^{\mathbf{x}_i \beta_J}} \\ &\quad + \dots + \\ &\quad + \frac{e^{\mathbf{x}_i \beta_J}}{1 + e^{\mathbf{x}_i \beta_1} + e^{\mathbf{x}_i \beta_2} + \dots + e^{\mathbf{x}_i \beta_J}} \end{aligned}$$

Para el proceso de estimación, consideraremos la función de verosimilitud dada por:

$$L(\beta_1, \beta_2, \dots, \beta_J) = \prod_{i=1}^n [(P(j=0|\mathbf{x}_i))^{I(0)} \cdot (P(j=1|\mathbf{x}_i))^{I(1)} \dots (P(j=J|\mathbf{x}_i))^{I(J)}]$$

Donde $I(j)$ es la función indicadora que toma el valor de 1 o 0 dependiendo de cual de las opciones posibles $j = 0, 1, 2, \dots, J$ es cierta para cada individuo $i = 1, 2, \dots, n$. De forma similar al caso de respuesta binaria en los modelos Logit Multinomiales se interpretan en sus efectos marginales y no de forma directa en los coeficientes β_j .

Para una variable x_{ik} continua y para cada opción $j = 0, 1, 2, \dots, J$ el efecto marginal será:

$$\begin{aligned} EMg_k &= \frac{\partial P(y_i = j|\mathbf{x}_i)}{\partial x_{ik}} \\ &= P(y_i = j|\mathbf{x}_i) \cdot \left[\beta_{jk} - \frac{\sum_{h=1}^J \beta_{hk} \cdot e^{\mathbf{x}_i \beta_h}}{1 + e^{\mathbf{x}_i \beta_1} + e^{\mathbf{x}_i \beta_2} + \dots + e^{\mathbf{x}_i \beta_J}} \right] \end{aligned}$$

Donde β_{hk} es el k -ésimo elemento del vector β_h . En el caso de que x_{ik} se una variable dicotómica (o categórica) y para cada opción $j = 0, 1, 2, \dots, J$ el efecto marginal será:

$$EMg_k = P(y_i = j|\mathbf{x}_i, x_{ik} = 1) - P(y_i = j|\mathbf{x}_i, x_{ik} = 0)$$

Modelos de respuesta ordenada: Logit y Probit Ordinal

Sea y_i una variable que representa una respuesta ordenada que toma los valores de $\{0, 1, 2, \dots, J\}$. En estos casos, en que importa el orden de la respuesta, existen dos mecanismos de estimación: el Modelo Probit y el Modelo Logit.

Modelo Probit Ordinal. Este modelo se puede derivar de forma similar al modelo probit de dos respuestas 0 y 1, probit binario. Así, partimos de una variable latente y_i^* definida como:

$$y_i^* = \mathbf{x}_i \beta + \varepsilon_i; \quad \varepsilon_i \sim N(0, 1)$$

Donde β es un vector $(K-1) \times 1$ ya que en este caso asumiremos que no se considera un término constante, por las razones que mostramos más

adelante. Sean $\alpha_1, \alpha_2, \dots, \alpha_J$ un conjunto de umbrales que particionan la probabilidad de la siguiente forma:

$$\begin{aligned}
y_i = 0 & \quad \text{si} \quad y_i^* \leq \alpha_1 \\
y_i = 1 & \quad \text{si} \quad \alpha_1 < y_i^* \leq \alpha_2 \\
& \quad \vdots \\
y_i = j & \quad \text{si} \quad \alpha_j < y_i^* \leq \alpha_{j+1} \\
& \quad \vdots \\
y_i = J & \quad \text{si} \quad \alpha_J < y_i^*
\end{aligned}$$

Dado que ε_i tiene una distribución normal estándar, para cada uno de los casos anteriores podemos establecer:

$$\begin{aligned}
P(y_i = 0 | \mathbf{x}_i) &= P(y_i^* \leq \alpha_1 | \mathbf{x}_i) = P(\mathbf{x}_i \boldsymbol{\beta} + \varepsilon_i \leq \alpha_1 | \mathbf{x}_i) \\
P(y_i = 1 | \mathbf{x}_i) &= P(\alpha_1 < y_i^* \leq \alpha_2 | \mathbf{x}_i) = P(\alpha_1 < \mathbf{x}_i \boldsymbol{\beta} + \varepsilon_i \leq \alpha_2 | \mathbf{x}_i) \\
& \quad \vdots \\
P(y_i = j | \mathbf{x}_i) &= P(\alpha_j < y_i^* \leq \alpha_{j+1} | \mathbf{x}_i) = P(\alpha_j < \mathbf{x}_i \boldsymbol{\beta} + \varepsilon_i \leq \alpha_{j+1} | \mathbf{x}_i) \\
& \quad \vdots \\
P(y_i = J | \mathbf{x}_i) &= P(\alpha_J < y_i^* | \mathbf{x}_i) = P(\alpha_J < \mathbf{x}_i \boldsymbol{\beta} + \varepsilon_i | \mathbf{x}_i)
\end{aligned}$$

Desarrollando las ecuaciones anteriores, tenemos para el caso de $y_i = 0$:

$$\begin{aligned}
P(y_i = 0 | \mathbf{x}_i) &= P(\mathbf{x}_i \boldsymbol{\beta} + \varepsilon_i \leq \alpha_1 | \mathbf{x}_i) \\
&= P(\varepsilon_i \leq \alpha_1 - \mathbf{x}_i \boldsymbol{\beta} | \mathbf{x}_i) \\
&= \Phi(\alpha_1 - \mathbf{x}_i \boldsymbol{\beta})
\end{aligned}$$

En el caso de $y_i = j$, $j = 1, 2, \dots, J - 1$:

$$\begin{aligned}
P(y_i = j | \mathbf{x}_i) &= P(\alpha_j < \mathbf{x}_i \boldsymbol{\beta} + \varepsilon_i \leq \alpha_{j+1} | \mathbf{x}_i) \\
&= P(\alpha_j - \mathbf{x}_i \boldsymbol{\beta} < \varepsilon_i \leq \alpha_{j+1} - \mathbf{x}_i \boldsymbol{\beta} | \mathbf{x}_i) \\
&= P(\varepsilon_i \leq \alpha_{j+1} - \mathbf{x}_i \boldsymbol{\beta} | \mathbf{x}_i) - P(\varepsilon_i < \alpha_j - \mathbf{x}_i \boldsymbol{\beta} | \mathbf{x}_i) \\
&= \Phi(\alpha_{j+1} - \mathbf{x}_i \boldsymbol{\beta}) - \Phi(\alpha_j - \mathbf{x}_i \boldsymbol{\beta})
\end{aligned}$$

Finalmente, para el caso $y_i = J$:

$$\begin{aligned}
P(y_i = J|\mathbf{x}_i) &= P(\alpha_J < \mathbf{x}_i\boldsymbol{\beta} + \boldsymbol{\varepsilon}_i|\mathbf{x}_i) \\
&= P(\alpha_J - \mathbf{x}_i\boldsymbol{\beta} < \boldsymbol{\varepsilon}_i|\mathbf{x}_i) \\
&= 1 - P(\boldsymbol{\varepsilon}_i < \alpha_J - \mathbf{x}_i\boldsymbol{\beta}|\mathbf{x}_i) \\
&= 1 - \Phi(\alpha_J - \mathbf{x}_i\boldsymbol{\beta})
\end{aligned}$$

Es sencillo observar y analizar que:

$$1 = \sum_{j=0}^J P(y_i = j|\mathbf{x}_i)$$

Notemos que en este modelo los cambios en la probabilidad no serán determinados por $\mathbf{x}_i$, ya que lo que determina el cambio es el umbral α_j , $j = 0, 1, \dots, J$. También en este caso los coeficientes no se interpretan de forma directa, en cambio se utiliza el efecto marginal que tiene expresiones similares al del probit bivariado como más adelante mostraremos.

Modelo Logit Ordinal. Este modelo se puede derivar de forma similar al modelo Probit Ordinal con una variable latente y_i^* , también con $\boldsymbol{\beta}$ es un vector $(K - 1) \times 1$ ya que en este caso asumiremos que no se considera un término constante, por las razones que hemos mostrado anteriormente. Igualmente, sean $\alpha_1, \alpha_2, \dots, \alpha_J$ un conjunto de umbrales que particionan la probabilidad considerando una función logística $\Lambda(\cdot)$ de la siguiente forma:

$$\boldsymbol{\varepsilon}_i \sim \Lambda(\cdot)$$

$$P(y_i = J|\mathbf{x}_i) = P(\alpha_J < y_i^*|\mathbf{x}_i) = P(\alpha_J < \mathbf{x}_i\boldsymbol{\beta} + \boldsymbol{\varepsilon}_i|\mathbf{x}_i)$$

Desarrollando las ecuaciones de probabilidad de forma similar al Probit Multinomial tenemos para el caso de $y_i = 0$:

$$\begin{aligned}
P(y_i = 0|\mathbf{x}_i) &= P(y_i^* \leq \alpha_1|\mathbf{x}_i) \\
&= P(\mathbf{x}_i\boldsymbol{\beta} + \boldsymbol{\varepsilon}_i \leq \alpha_1|\mathbf{x}_i) \\
&= P(\boldsymbol{\varepsilon}_i \leq \alpha_1 - \mathbf{x}_i\boldsymbol{\beta}|\mathbf{x}_i) \\
&= \Lambda(\alpha_1 - \mathbf{x}_i\boldsymbol{\beta})
\end{aligned}$$

En el caso de $y_i = j$, $j = 1, 2, \dots, J - 1$:

$$\begin{aligned}
P(y_i = j | \mathbf{x}_i) &= P(\alpha_j < y_i^* \leq \alpha_{j+1} | \mathbf{x}_i) \\
&= P(\alpha_j < \mathbf{x}_i \boldsymbol{\beta} + \varepsilon_i \leq \alpha_{j+1} | \mathbf{x}_i) \\
&= P(\alpha_j - \mathbf{x}_i \boldsymbol{\beta} < \varepsilon_i \leq \alpha_{j+1} - \mathbf{x}_i \boldsymbol{\beta} | \mathbf{x}_i) \\
&= P(\varepsilon_i \leq \alpha_{j+1} - \mathbf{x}_i \boldsymbol{\beta} | \mathbf{x}_i) - P(\varepsilon_i < \alpha_j - \mathbf{x}_i \boldsymbol{\beta} | \mathbf{x}_i) \\
&= \Lambda(\alpha_{j+1} - \mathbf{x}_i \boldsymbol{\beta}) - \Lambda(\alpha_j - \mathbf{x}_i \boldsymbol{\beta})
\end{aligned}$$

Finalmente, para el caso $y_i = J$:

$$\begin{aligned}
P(y_i = J | \mathbf{x}_i) &= P(\alpha_J < y_i^* | \mathbf{x}_i) \\
&= P(\alpha_J < \mathbf{x}_i \boldsymbol{\beta} + \varepsilon_i | \mathbf{x}_i) \\
&= P(\alpha_J - \mathbf{x}_i \boldsymbol{\beta} < \varepsilon_i | \mathbf{x}_i) \\
&= 1 - P(\varepsilon_i < \alpha_J - \mathbf{x}_i \boldsymbol{\beta} | \mathbf{x}_i) \\
&= 1 - \Lambda(\alpha_J - \mathbf{x}_i \boldsymbol{\beta})
\end{aligned}$$

También, es sencillo observar y analizar que en los Logit Multinomiales:

$$1 = \sum_{j=0}^J P(y_i = j | \mathbf{x}_i)$$

Notemos que en este modelo los cambios en la probabilidad no serán determinados por $\mathbf{x}_i$, ya que lo que determina el cambio es el umbral α_j , $j = 0, 1, \dots, J$. También en este caso los coeficientes no se interpretan de forma directa, en cambio se utiliza el efecto marginal que tiene expresiones similares al del probit bivariado como más adelante mostraremos.

Estimación de modelos Probit y Logit Ordinal. Para la estimación implementaremos un proceso de maximización de de la siguiente función de verosimilitud:

$$L(\alpha, \boldsymbol{\beta}) = \prod_{i=1}^n P(y_i = 0 | \mathbf{x}_i)^{I(0)} \cdot P(y_i = 1 | \mathbf{x}_i)^{I(1)} \cdots P(y_i = J | \mathbf{x}_i)^{I(J)}$$

Donde $I(j)$ es la función indicadora que toma el valor de 0 y 1 si el individuo i -ésimo tiene como respuesta alguno de los posibles valores de $j = 0, 1, \dots, J$. La función es válida sin importar la función $\Phi(\cdot)$ o $\Lambda(\cdot)$ que utilicemos para determinar la $P(y_i = j | \mathbf{x}_i)$, para $j = 1, 2, \dots, J$. Por lo cual simbolizaremos indistintamente estas funciones como $G(\cdot)$.



Figura 1.18: División del conjunto de datos, retomado de Hastie, Tibshirani, y Friedman (2009, p. 222) [HTF09]

Finalmente, sin importar el modelo que estemos ocupando, Probit o Logit ordinal, la forma de interpretar el modelo es mediante el efecto marginal, cuando x_k sea una variable continua:

$$EMg_k = \frac{\partial}{\partial x_k} P(y_i = j | \mathbf{x}_i) = \frac{\partial}{\partial x_k} G(\alpha_j - \mathbf{x}_i \boldsymbol{\beta}) = -\beta_k \cdot g(\alpha_j - \mathbf{x}_i \boldsymbol{\beta})$$

Por su parte, cuando x_j sea una variable dicotómica:

$$\begin{aligned} EMg_k &= P(y_i = j | \mathbf{x}_i, x_{ik} = 1) - P(y_i = j | \mathbf{x}_i, x_{ik} = 0) \\ &= G(\alpha_j - \mathbf{x}_i \boldsymbol{\beta} | \mathbf{x}_i, x_{ik} = 1) - G(\alpha_j - \mathbf{x}_i \boldsymbol{\beta} | \mathbf{x}_i, x_{ik} = 0) \\ &= G(\alpha_j - (\beta_1 + x_2 \beta_2 + \dots + \beta_k + \dots + x_K \beta_K)) \\ &\quad - G(\alpha_j - (\beta_1 + x_2 \beta_2 + \dots + 0 + \dots + x_K \beta_K)) \end{aligned}$$

También es posible hacer un efecto marginal intra respuestas, es decir, para cambios entre una respuesta j y una $j + h$, la cual será:

$$EMg_{j-h} = \beta_k \cdot [g(\alpha_h - \mathbf{x}_i \boldsymbol{\beta}) - g(\alpha_j - \mathbf{x}_i \boldsymbol{\beta})], \text{ con } h < j$$

Donde $k = 1, 2, \dots, K - 1$ y $j = 0, 1, \dots, J$.

Ajuste y separación del conjunto de datos

De forma similar al caso de regresión, dividiremos al conjunto de datos conforme se describe en la Figura 1.18.

Evaluando los modelos de aprendizaje supervisado

Matriz de confusión

Una vez que se entrena un problema de aprendizaje automatizado supervisado en un conjunto de datos históricos, se prueba el modelo obtenido mediante el uso de datos del conjunto de entrenamiento. De esta forma, es

posible comparar las predicciones del modelo entrenado con los valores reales de la variable sujeta de análisis. La matriz de confusión proporciona un medio para evaluar el éxito de un problema de clasificación y dónde se cometen errores (es decir, dónde se vuelve 'confuso').

En el Cuadro 1.1 se muestra un ejemplo de la forma en que se suele mostrar una matriz de confusión.

		Predicciones	
		Positivas (1)	Negativas (0)
Real	Positivas (1)	True Positive (TP)	False Negative (FN)
	Negativas (0)	False Positive (FP)	True Negative (TN)

Cuadro 1.1: Matriz de Confusión

Con base en la matriz de confusión en el Cuadro 1.1, se pueden construir métricas que se calculan de la siguiente manera:

$$Precision = \frac{TP}{TP + FP} \quad (1.68)$$

$$Recuperacion = \frac{TP}{TP + FN} \quad (1.69)$$

$$F1 = 2 \times \frac{Precision \times Recuperacion}{Precision + Recuperacion} \quad (1.70)$$

La medida $F1$ o $F1 - score$ combina precisión y recuperación mediante la media armónica de los dos valores. De esta forma, siempre se ubicará entre ambos indicadores.

1.4.4. Regresión logística para el estudio de violencia.

Ver “09_Ejemplo_Logit_Delitos” en GitHub.

1.4.5. Regresión logística con información de videojuegos.

Planteamiento del caso

Supuestos y necesidades del caso:

1. Determinar cuáles son las features/características/steam tags que se asocian con videojuegos para PC exitosos.
2. Hay varias tiendas de videojuegos (Steam, itch, etc.) que utilizan estas tags para clasificar los video juegos.

Propuesta: Usar un modelo de respuesta ordenada

Sea y una variable que representa una respuesta ordenada que toma valores $\{0, 1, 2, \dots, J\} \in \mathbb{Z}$. El modelo de respuesta ordenada (conocido como Probit o Logit ordenado) para y (condicional en un vector de variables explicativas $\mathbf{X}$ —features, características, tags, etc.—) es derivado a partir de un modelo de variable latente y^* , el cual se puede escribir como:

$$y^* = \mathbf{X}\boldsymbol{\beta} + \varepsilon, \varepsilon|\mathbf{X} \sim f(\varepsilon) \quad (1.71)$$

Donde $f(\cdot)$ es una función de densidad de probabilidad simétrica cuya función de densidad acumulada será denotada por $F(\cdot)$, $\boldsymbol{\beta}$ es un vector de dimensión $K \times 1$ y, por razones que más adelante explicamos, $\mathbf{X}$ no tiene término constante. Por otro lado, sean $\mu_1 < \mu_2 < \dots < \mu_J$ puntos de corte o parámetros umbrales que definen lo siguiente:

$$\begin{aligned} y = 0 & \quad \text{si} \quad y^* \leq \mu_1 \\ y = 1 & \quad \text{si} \quad \mu_1 < y^* \leq \mu_2 \\ & \quad \vdots \\ y = J & \quad \text{si} \quad y^* > \mu_J \end{aligned} \quad (1.72)$$

De esta forma, y considerando el conjunto de desigualdades en la ecuación (1.72) y la ecuación (1.71), podemos determinar las siguientes probabilidades a partir de una partición de una función de distribución dada:

$$\begin{aligned} \mathbf{P}(y = 0|\mathbf{X}) &= \mathbf{P}(y^* \leq \mu_1|\mathbf{X}) \\ &= \mathbf{P}(\mathbf{X}\boldsymbol{\beta} + \varepsilon \leq \mu_1|\mathbf{X}) \\ &= \mathbf{P}(\varepsilon \leq \mu_1 - \mathbf{X}\boldsymbol{\beta}|\mathbf{X}) \\ &= F(\mu_1 - \mathbf{X}\boldsymbol{\beta}) \end{aligned}$$

$$\begin{aligned}
\mathbf{P}(y = 1|\mathbf{X}) &= \mathbf{P}(\mu_1 < y^* \leq \mu_2|\mathbf{X}) \\
&= \mathbf{P}(\mu_1 < \mathbf{X}\boldsymbol{\beta} + \varepsilon \leq \mu_2|\mathbf{X}) \\
&= \mathbf{P}(\mu_1 - \mathbf{X}\boldsymbol{\beta} < \varepsilon \leq \mu_2 - \mathbf{X}\boldsymbol{\beta}|\mathbf{X}) \\
&= F(\mu_2 - \mathbf{X}\boldsymbol{\beta}) - F(\mu_1 - \mathbf{X}\boldsymbol{\beta})
\end{aligned}$$

$\vdots$

$$\begin{aligned}
\mathbf{P}(y = J - 1|\mathbf{X}) &= \mathbf{P}(\mu_{J-1} < y^* \leq \mu_J|\mathbf{X}) \\
&= \mathbf{P}(\mu_{J-1} < \mathbf{X}\boldsymbol{\beta} + \varepsilon \leq \mu_J|\mathbf{X}) \\
&= \mathbf{P}(\mu_{J-1} - \mathbf{X}\boldsymbol{\beta} < \varepsilon \leq \mu_J - \mathbf{X}\boldsymbol{\beta}|\mathbf{X}) \\
&= F(\mu_J - \mathbf{X}\boldsymbol{\beta}) - F(\mu_{J-1} - \mathbf{X}\boldsymbol{\beta})
\end{aligned}$$

$$\begin{aligned}
\mathbf{P}(y = J|\mathbf{X}) &= \mathbf{P}(y^* > \mu_J|\mathbf{X}) \\
&= \mathbf{P}(\mathbf{X}\boldsymbol{\beta} + \varepsilon > \mu_J|\mathbf{X}) \\
&= \mathbf{P}(\varepsilon > \mu_J - \mathbf{X}\boldsymbol{\beta}|\mathbf{X}) \\
&= 1 - F(\mu_J - \mathbf{X}\boldsymbol{\beta})
\end{aligned}$$

Note que en este caso asumimos que $\mathbf{X}$ no tiene término constante, ya que la incorporamos mediante los umbrales μ_j , $j = 1, 2, \dots, J$. Un caso particular de esta modelación son los modelos de clasificación binarios en los que $J = 1$. En la figura 1.19 ilustramos la partición de la función de densidad $f(\cdot)$ que resultaría de las ecuaciones anteriore.

De esta manera, los resultados de la estimación se pueden interpretar a través de sus efectos marginales dados por:

$$\frac{\partial \mathbf{P}(y = j|\mathbf{X})}{\partial x_k} = -\beta_k(f(\mu_j - \mathbf{X}\boldsymbol{\beta}) - f(\mu_{j-1} - \mathbf{X}\boldsymbol{\beta})) \text{ para } 0 < j < J \quad (1.73)$$

Así, este efecto se interpreta como la contribución que tiene la variable x_k a la probabilidad de que la variable y tome el valor de j

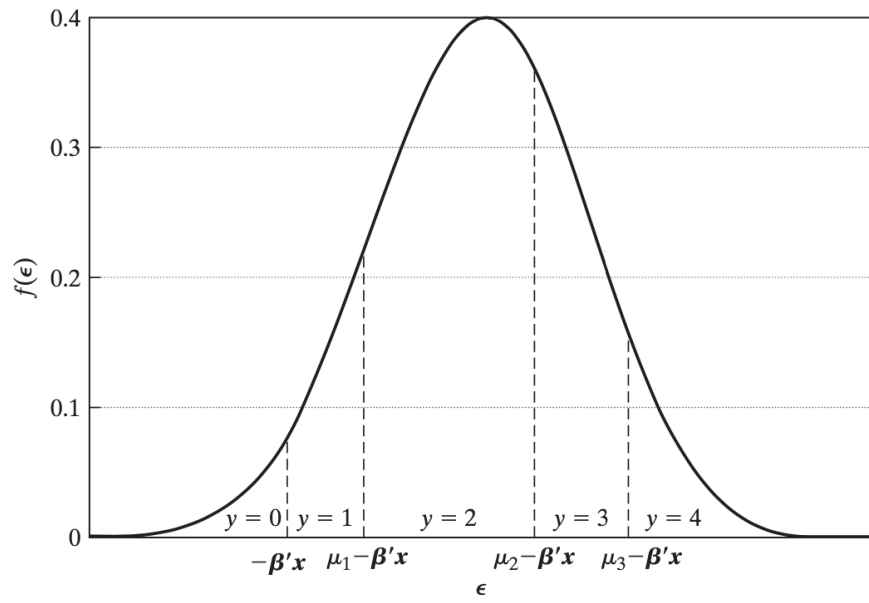


Figura 1.19: Probabilidades en un modelo de respuesta ordenada, considerando $J = 4$ (retomado de Greene (2012, 788) [Gre12])

Aplicación al caso de videojuegos

En el caso de los videojuegos, podemos aplicar el modelo de respuesta ordenada asumiendo algunas cosas. Primero, F tendrá una forma de función logística, para facilitar el proceso computacional. Segundo, la variable y será particionada en rangos de ingresos obtenidos por los videojuegos en su primer año (y^*)—año de lanzamiento—, por ejemplo:

$$y^* = \mathbf{X}\boldsymbol{\beta} + \varepsilon, \varepsilon|\mathbf{X} \sim f(\varepsilon)$$

Donde y^* son los ingresos anuales, $\mathbf{X}$ son las características y otras variables de los videojuegos, $\boldsymbol{\beta}$ son los coeficientes y ε es el término de error.

$$\begin{aligned} y = 0 & \quad \text{if} \quad y^* \leq 1\text{M USD} \\ y = 1 & \quad \text{if} \quad 1 \text{ MM USD} < y^* \leq 5 \text{ MM USD} \\ y = 2 & \quad \text{if} \quad 5 \text{ MM USD} < y^* \leq 20 \text{ MM USD} \\ y = 3 & \quad \text{if} \quad y^* > 20 \text{ MM USD} \end{aligned}$$

$$\begin{aligned} \mathbf{P}(y = 0|\mathbf{X}) &= \mathbf{P}(y^* \leq \mu_1|\mathbf{X}) \\ &= \mathbf{P}(\mathbf{X}\boldsymbol{\beta} + \varepsilon \leq \mu_1|\mathbf{X}) \\ &= \mathbf{P}(\varepsilon \leq \mu_1 - \mathbf{X}\boldsymbol{\beta}|\mathbf{X}) \\ &= F(\mu_1 - \mathbf{X}\boldsymbol{\beta}) \end{aligned}$$

$$\begin{aligned} \mathbf{P}(y = 1|\mathbf{X}) &= \mathbf{P}(\mu_1 < y^* \leq \mu_2|\mathbf{X}) \\ &= \mathbf{P}(\mu_1 < \mathbf{X}\boldsymbol{\beta} + \varepsilon \leq \mu_2|\mathbf{X}) \\ &= \mathbf{P}(\mu_1 - \mathbf{X}\boldsymbol{\beta} < \varepsilon \leq \mu_2 - \mathbf{X}\boldsymbol{\beta}|\mathbf{X}) \\ &= F(\mu_2 - \mathbf{X}\boldsymbol{\beta}) - F(\mu_1 - \mathbf{X}\boldsymbol{\beta}) \end{aligned}$$

$$\begin{aligned} \mathbf{P}(y = 2|\mathbf{X}) &= \mathbf{P}(\mu_2 < y^* \leq \mu_3|\mathbf{X}) \\ &= \mathbf{P}(\mu_2 < \mathbf{X}\boldsymbol{\beta} + \varepsilon \leq \mu_3|\mathbf{X}) \\ &= \mathbf{P}(\mu_2 - \mathbf{X}\boldsymbol{\beta} < \varepsilon \leq \mu_3 - \mathbf{X}\boldsymbol{\beta}|\mathbf{X}) \\ &= F(\mu_3 - \mathbf{X}\boldsymbol{\beta}) - F(\mu_2 - \mathbf{X}\boldsymbol{\beta}) \end{aligned}$$

$$\begin{aligned}
\mathbf{P}(y = 3|\mathbf{X}) &= \mathbf{P}(y^* > \mu_3|\mathbf{X}) \\
&= \mathbf{P}(\mathbf{X}\boldsymbol{\beta} + \varepsilon > \mu_3|\mathbf{X}) \\
&= \mathbf{P}(\varepsilon > \mu_3 - \mathbf{X}\boldsymbol{\beta}|\mathbf{X}) \\
&= 1 - F(\mu_3 - \mathbf{X}\boldsymbol{\beta})
\end{aligned}$$

Supongamos que tenemos n observaciones o, en este caso, videojuegos, los cuales son indexados con $i = 1, 2, 3, \dots, n$. Así, indexaremos a la variable de respuesta y como y_i , por lo que diremos que buscamos estimar las siguientes probabilidades:

$$\begin{aligned}
\mathbf{P}(\text{Revenue}_i \leq 1\text{M USD}|\mathbf{X}_i) &= \mathbf{P}(\varepsilon_i \leq \mu_1 - \mathbf{X}_i\boldsymbol{\beta}|\mathbf{X}_i) \\
&= F(\mu_1 - \mathbf{X}_i\boldsymbol{\beta})
\end{aligned}$$

$$\begin{aligned}
\mathbf{P}(1 \text{ MM USD} < \text{Revenue}_i \leq 5 \text{ MM USD}|\mathbf{X}_i) &= \mathbf{P}(\mu_1 - \mathbf{X}_i\boldsymbol{\beta} < \varepsilon_i \leq \mu_2 - \mathbf{X}_i\boldsymbol{\beta}|\mathbf{X}_i) \\
&= F(\mu_2 - \mathbf{X}_i\boldsymbol{\beta}) - F(\mu_1 - \mathbf{X}_i\boldsymbol{\beta})
\end{aligned}$$

$$\begin{aligned}
\mathbf{P}(5 \text{ MM USD} < \text{Revenue}_i \leq 20 \text{ MM USD}|\mathbf{X}_i) &= \mathbf{P}(\mu_2 - \mathbf{X}_i\boldsymbol{\beta} < \varepsilon_i \leq \mu_3 - \mathbf{X}_i\boldsymbol{\beta}|\mathbf{X}_i) \\
&= F(\mu_3 - \mathbf{X}_i\boldsymbol{\beta}) - F(\mu_2 - \mathbf{X}_i\boldsymbol{\beta})
\end{aligned}$$

$$\begin{aligned}
\mathbf{P}(\text{Revenue}_i > 20 \text{ MM USD}|\mathbf{X}_i) &= \mathbf{P}(\varepsilon_i > \mu_3 - \mathbf{X}_i\boldsymbol{\beta}|\mathbf{X}_i) \\
&= 1 - F(\mu_3 - \mathbf{X}_i\boldsymbol{\beta})
\end{aligned}$$

Donde, particularmente, F es una función logística. Nuestro objetivo es maximizar la función de verosimilitud (función que maximiza el valor de los parámetros μ_1 , μ_2 , μ_3 y $\boldsymbol{\beta}$ dada la información disponible):

$$\begin{aligned}
\mathbf{L}(\boldsymbol{\theta}|\text{Data}) &= \prod_{y_i=0} \mathbf{P}(\text{Revenue}_i \leq 1\text{M USD}|\mathbf{X}_i) \\
&\times \prod_{y_i=1} \mathbf{P}(1 \text{ MM USD} < \text{Revenue}_i \leq 5 \text{ MM USD}|\mathbf{X}_i) \\
&\times \prod_{y_i=2} \mathbf{P}(5 \text{ MM USD} < \text{Revenue}_i \leq 20 \text{ MM USD}|\mathbf{X}_i) \\
&\times \prod_{y_i=3} \mathbf{P}(\text{Revenue}_i > 20 \text{ MM USD}|\mathbf{X}_i) \\
&= \prod_{y_i=0} F(\mu_1 - \mathbf{X}_i\boldsymbol{\beta}) \prod_{y_i=1} (F(\mu_2 - \mathbf{X}_i\boldsymbol{\beta}) - F(\mu_1 - \mathbf{X}_i\boldsymbol{\beta})) \\
&\times \prod_{y_i=2} (F(\mu_3 - \mathbf{X}_i\boldsymbol{\beta}) - F(\mu_2 - \mathbf{X}_i\boldsymbol{\beta})) \prod_{y_i=3} (1 - F(\mu_3 - \mathbf{X}_i\boldsymbol{\beta}))
\end{aligned}$$

Donde:

$$\boldsymbol{\theta} = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \mu_3 \\ \boldsymbol{\beta} \end{bmatrix}$$

Así, el problema consiste en resolver:

$$\begin{aligned}
\max_{\boldsymbol{\theta}} \mathbf{L}(\boldsymbol{\theta}|\text{Data}) &= \prod_{\text{Revenue}_i \leq 1\text{M USD}} F(\mu_1 - \mathbf{X}_i\boldsymbol{\beta}) \\
&\times \prod_{1 \text{ MM USD} < \text{Revenue}_i \leq 5 \text{ MM USD}} (F(\mu_2 - \mathbf{X}_i\boldsymbol{\beta}) - F(\mu_1 - \mathbf{X}_i\boldsymbol{\beta})) \\
&\times \prod_{5 \text{ MM USD} < \text{Revenue}_i \leq 20 \text{ MM USD}} (F(\mu_3 - \mathbf{X}_i\boldsymbol{\beta}) - F(\mu_2 - \mathbf{X}_i\boldsymbol{\beta})) \\
&\times \prod_{\text{Revenue}_i > 20 \text{ MM USD}} (1 - F(\mu_3 - \mathbf{X}_i\boldsymbol{\beta}))
\end{aligned}$$

Datos

Usamos como fuente: <https://games-stats.com/>, una descarga de datos de 2021. La tabla de datos originalmente descargada contenía información de 53,650 videojuegos para computadora de los últimos 40 años, acumulando 18.5 billones de dólares.

Tags	Juegos	Tags	Juegos	Tags	Juegos	Tags	Juegos	Tags	Juegos
Indie	36,782	Combat	3,866	Multiple Endings	2,065	Historical	1,313	Music	910
Action	25,650	Female Protagonist	3,857	Roguelike	2,054	Walking Simulator	1,312	Competitive	882
Adventure	23,865	Violent	3,796	Puzzle Platformer	2,047	Resource Management	1,293	Dark Humor	881
Casual	23,618		0	War	1,994	Interactive Fiction	1,292	Aliens	880
Singleplayer	23,597	Open World	3,604	Shoot 'Em Up	1,993	Memes	1,252	Clicker	872
2D	12,837	Sexual Content	3,598	Linear	1,849	Dungeon Crawler	1,240	Nature	866
Strategy	11,854	Visual Novel	3,502	Character Customization	1,813	Score Attack	1,229	Driving	862
Simulation	11,569	Comedy	3,398	Hand-drawn	1,804	Dating Sim	1,225	Emotional	862
RPG	11,097	Turn-Based	3,359	Old School	1,794	Survival	1,212	Fight	857
Puzzle	9,354	Action-Adventure	3,342	Massively Multiplayer	1,738	Top-Down Shooter	1,212	Arena Shooter	846
Multiplayer	8,185	Co-op	3,295	Procedural Generation	1,738	Experimental	1,204	Detective	846
Atmospheric	8,117	Top-Down	3,289	Action RPG	1,735	Post-apocalyptic	1,201	Party	845
Early Access	7,604	Management	3,287	Turn-Based Strategy	1,734	Mod	1,181	1980s	837
Platformer	7,103	Third Person	3,240	Replay Value	1,706	RTS	1,167	Perma Death	836
Story Rich	6,711	Cartoon	3,210	Classic	1,672	Isometric	1,151	Immersive Sim	824
3D	6,584	Gore	3,205	Local Co-Op	1,651	Stealth	1,146	Investigation	820
Shooter	6,181	FPS	3,100	Online Co-Op	1,641	Dark Fantasy	1,129	Precision Platformer	815
Pixel Graphics	6,174	Sports	2,941	Survival Horror	1,598	Education	1,127	Economy	809
Fantasy	5,910	Mystery	2,704	Hidden Object	1,597	Narration	1,125	Beat 'em up	805
Colorful	5,629	Point & Click	2,671	Logic	1,586	1990s	1,112	NSFW	781
Arcade	5,564	Psychological	2,660	Mature	1,582	Drama	1,104	Time Management	782
Horror	5,504	Stylized	2,659	Magic	1,575	Text-Based	1,095	Beautiful	767
Anime	5,424	Space	2,611	Roguelite	1,558	Base Building	1,089	Cinematic	728
Cute	5,417	Minimalist	2,591	3D Platformer	1,519	Card Game	1,077	Metroidvania	727
First-Person	5,351	Physics	2,573	Futuristic	1,518	Abstract	1,072	Team-Based	726
Funny	5,180	PVP	2,494	Fast-Paced	1,480	Hentai	1,051	City Builder	718
Soundtrack	4,886	Choices Matter	2,464	Turn-Based Combat	1,467	Fighting	1,033	Real Time Tactics	702
VR	4,862	Building	2,456	PVE	1,466	Third-Person Shooter	1,029	Strategy RPG	696
Exploration	4,806	Sandbox	2,415	Crafting	1,451	4 Player Local	1,005	Loot	696
Retro	4,734	Cartoony	2,389	Bullet Hell	1,447	Tower Defense	997	Psychodelic	677
Sci-Fi	4,601	2D Platformer	2,378	Short	1,428	2.5D	992	Character Action Game	669
Great Soundtrack	4,580	Psychological Horror	2,323	Medieval	1,424	Cyberpunk	986	Twin Stick Shooter	662
Free to Play	4,473	Tactical	2,317	JRPG	1,396	Military	979	Conversation	657
Difficult	4,445	Side Scroller	2,243	Romance	1,373	Robots	963	Tabletop	653
Survival	4,410	Realistic	2,220	Hack and Slash	1,365	Action Roguelike	942	Level Editor	651
Family Friendly	4,231	Racing	2,198	Choose Your Own Adventure	1,325	Immersive	934	Runner	647
Dark	3,940	Controller	2,148	Turn-Based Tactics	1,325	Board Game	916	Life Sim	645
Relaxing	3,924	Local Multiplayer	2,136	RPGMaker	1,317	LGBTQ+	913	Wargame	636
									Philosophical
									413

There is a large quantity of tags found in the database, thus it makes complicated to classify a game into a single category. However, tag's value lies in the fact that users are the ones who tag a game. It tells us how a user perceives a game.

There are 417 tags in the database with a particular category for each one in which a game can be.

Figura 1.20: En la tabla de datos ubicamos 417 tags distintas

La tabla de datos de games-stats.com se utilizó como base de datos central debido a su tamaño y variables disponibles. Fue validado con otros datos encontrados en línea, y tenían un tamaño similar entre ellos. Se agregaron más variables con el propósito de enriquecer el análisis.

Hay una gran cantidad de etiquetas (tags) encontradas en la tabla de datos, por lo que resulta complicado clasificar un juego en una sola categoría. Sin embargo, el valor de la etiqueta radica en el hecho de que son los usuarios quienes etiquetan un juego. Esto nos dice cómo un usuario percibe un juego. En total ubicamos 417 tags distintas.

En este caso, requerimos una reducción de dimensionalidad. El modelo debía evaluar más de 450 etiquetas diferentes, algo inviable. Tener 450 etiquetas para evaluar reduce la simplicidad e interpretabilidad del modelo. Así, aplicamos un par de metodologías:

- Extreme Random Decision Tree method: Que es una metodología de Machine Learning que ayudará a considerar menos variables dada la relevancia para los ingresos.
- Spearman Correlation: que muestra la correlación con los ingresos de determinadas variables y muestra también la significación estadística basada en una prueba t. Así, la etiqueta de un jugador tiene la correlación significativa más alta con los ingresos.

El código del ejemplo se encuentra en GitHub con el nombre:
“10_Ejemplo_Logit_Ordenado” en GitHub.

2

Temas Selectos para el Análisis de Datos

2.1. Aplicación de modelos de análisis de texto.

2.1.1. Elementos de Procesamiento de Lenguaje y Minado de Texto

En esta sección usamos principalmente el documento de Jurafsky y Martin (2023) [JM23] y el de Shmueli, et al (2019) [Shm+19].

Expresiones regulares y tokenización

Comenzaremos con la herramienta más importante para describir patrones de texto: la expresión regular. Se pueden usar expresiones regulares para especificar cadenas que podríamos querer extraer de un documento, incluso cadenas como \$199 o \$24.99 para extraer tablas de precios de un documento, por ejemplo.

Luego pasaremos a un conjunto de tareas denominadas, colectivamente, normalización de texto, en las que las expresiones regulares juegan un papel importante. Normalizar el texto significa convertirlo a un formato estándar más conveniente. Por ejemplo, la mayor parte de lo que vamos a hacer con el lenguaje se basa en separar o tokenizar primero las palabras del texto en ejecución. Las palabras en inglés suelen estar separadas entre sí por espacios en blanco, pero los espacios en blanco no siempre son suficientes. Nueva York y rock ‘n’ roll a veces se tratan como palabras grandes a pesar de que

contienen espacios, mientras que a veces tendremos que separar I'm en las dos palabras I y am. Para procesar tweets o mensajes de texto necesitaremos tokenizar emoticones como :) o hashtags como #nlproc.

Algunos idiomas, como el japonés, no tienen espacios entre palabras, por lo que la tokenización de palabras se vuelve más difícil.

Otra parte de la normalización del texto es la lematización, la tarea de determinar que dos palabras tienen la misma raíz, a pesar de sus diferencias superficiales. La lematización es esencial para procesar lenguas morfológicamente complejas como el árabe.

La derivación (stemming) se refiere a una versión más simple de lematización en la que simplemente eliminamos los sufijos del final de la palabra. La normalización del texto también incluye la segmentación de oraciones: dividir un texto en oraciones individuales, usando señales como puntos de oración o signos de exclamación.

Finalmente, necesitaremos comparar palabras y otras cadenas. Introduciremos una métrica llamada distancia de edición que mide qué tan similares son dos cadenas en función del número de ediciones (inserciones, eliminaciones, sustituciones) que se necesitan para cambiar una cadena por otra. La distancia de edición es un algoritmo con aplicaciones en todo el procesamiento del lenguaje, desde la corrección ortográfica hasta el reconocimiento de voz y la resolución de correferencia.

Expresiones regulares

Uno de los éxitos no reconocidos en la estandarización de la informática ha sido la expresión regular (a menudo abreviada como 'regex'), un lenguaje para especificar cadenas de búsqueda de texto. Este lenguaje práctico se utiliza en todos los lenguajes informáticos, procesadores de texto y herramientas de procesamiento de textos como las herramientas Unix grep o Emacs.

Formalmente, una expresión regular es una notación algebraica para caracterizar un conjunto de cadenas. Las expresiones regulares son particularmente útiles para buscar en textos, cuando tenemos un patrón que buscar y un corpus de textos que buscar. Una función de búsqueda de expresiones regulares buscará en el corpus y devolverá todos los textos que coincidan con el patrón. El corpus puede ser un documento único o una colección. Las expresiones regulares tienen muchas variantes. Usar un probador de expresiones regulares en línea es una forma práctica de probar sus expresiones y explorar estas variaciones, por ejemplo: <https://regexr.com/>.

El tipo más simple de expresión regular es una secuencia de caracteres simples; poner caracteres de concatenación en secuencia se llama concatena-

Regex	Example Patterns Matched
/woodchucks/	“interesting links to <u>woodchucks</u> and lemurs”
/a/	“Mary Ann stopped by Mona’s”
/!/	“You’ve left the burglar behind again!” said Nori

Figure 2.1 Some simple regex searches.

Figura 2.1: Some simple regex searches, retomado de Jurafsky y Martin (2023, 3)

Regex	Match	Example Patterns
/[wW]oodchuck/	Woodchuck or woodchuck	“ <u>Woodchuck</u> ”
/[abc]/	‘a’, ‘b’, or ‘c’	“In uomini, in soldati”
/[1234567890]/	any digit	“plenty of 7 to 5”

Figure 2.2 The use of the brackets [] to specify a disjunction of characters.

Figura 2.2: The use of the brackets [] to specify a disjunction of characters, retomado de Jurafsky y Martin (2023, 3)

ción. Las expresiones regulares distinguen entre mayúsculas y minúsculas. Las figuras 2.1 y 2.2.

La expresión regular /[1234567890]/ especifica cualquier dígito. Si bien clases de caracteres como dígitos o letras son componentes importantes de las expresiones, pueden resultar incómodas (por ejemplo, es inconveniente especificar /[ABCDEFGH IJKLMN O PQRSTU VWXYZ]/ para que signifique “cualquier letra mayúscula”).

En los casos en los que hay una secuencia bien definida asociada con un conjunto de caracteres, los corchetes se pueden usar con el guión (-) para especificar cualquier carácter en un rango.

El patrón /[2 – 5]/ especifica cualquiera de los caracteres 2, 3, 4 o 5. El patrón /[b – g]/ especifica uno de los caracteres b, c, d, e, f o g. Algunos otros ejemplos se muestran en la figura 2.3.

Las llaves cuadradas también se pueden usar para especificar lo que no puede ser un solo carácter, mediante el uso del signo de potencia ^. Si el signo de potencia ^ es el primer símbolo después de la llave cuadrada abierta [, el patrón resultante se niega. Por ejemplo, el patrón /^[^a]/ coincide con cualquier carácter (incluidos los caracteres especiales) excepto a. Esto sólo es

Regex	Match	Example Patterns Matched
/[A-Z]/	an upper case letter	“we should call it ‘Drenched Blossoms’ ”
/[a-z]/	a lower case letter	“ <u>my</u> beans were impatient to be hoed!”
/[0-9]/	a single digit	“Chapter <u>1</u> : Down the Rabbit Hole”

Figure 2.3 The use of the brackets [] plus the dash - to specify a range.

Figura 2.3: The use of the brackets [] plus the dash - to specify a range, retomado de Jurafsky y Martin (2023, 3)

Regex	Match (single characters)	Example Patterns Matched
/[^A-Z]/	not an upper case letter	“Oyfn pripetchik”
/[^Ss]/	neither ‘S’ nor ‘s’	“ <u>I</u> have no exquisite reason for’t”
/[^.]/	not a period	“our resident Djinn”
/[e^]/	either ‘e’ or ‘^’	“look up <u>^</u> now”
/a^b/	the pattern ‘a^b’	“look up <u>a^b</u> now”

Figure 2.4 The caret ^ for negation or just to mean ^. See below re: the backslash for escaping the period.

Figura 2.4: The caret ^ for negation or just to mean ^. See below re: the backslash for escaping the period, retomado de Jurafsky y Martin (2023, 3)

cierto cuando el signo de intercalación es el primer símbolo después de la llave cuadrada abierta. Si aparece en cualquier otro lugar, normalmente representa un signo de potencia. La figura 2.4 muestra algunos ejemplos.

Word Tokenization

Tokenización consiste en dividir el texto en oraciones y estas, en palabras. Tokenización también se puede utilizar para expandir las contracciones que están marcadas por apóstrofes, por ejemplo, convirtiendo what’s en los dos tokens what are y we’re en we are. Algunas de estas contracciones ocurren en otros idiomas alfabéticos, incluidos artículos y pronombres en francés (j’ai, l’homme).

Dependiendo de la aplicación, los algoritmos de tokenización también pueden tokenizar expresiones de varias palabras como Nueva York o rock ‘n’ roll como un solo token, lo que requiere algún tipo de diccionario de expresiones de varias palabras. Por lo tanto, la tokenización está íntimamente ligada al reconocimiento de entidades, la tarea de detectar nombres, fechas y organizaciones.

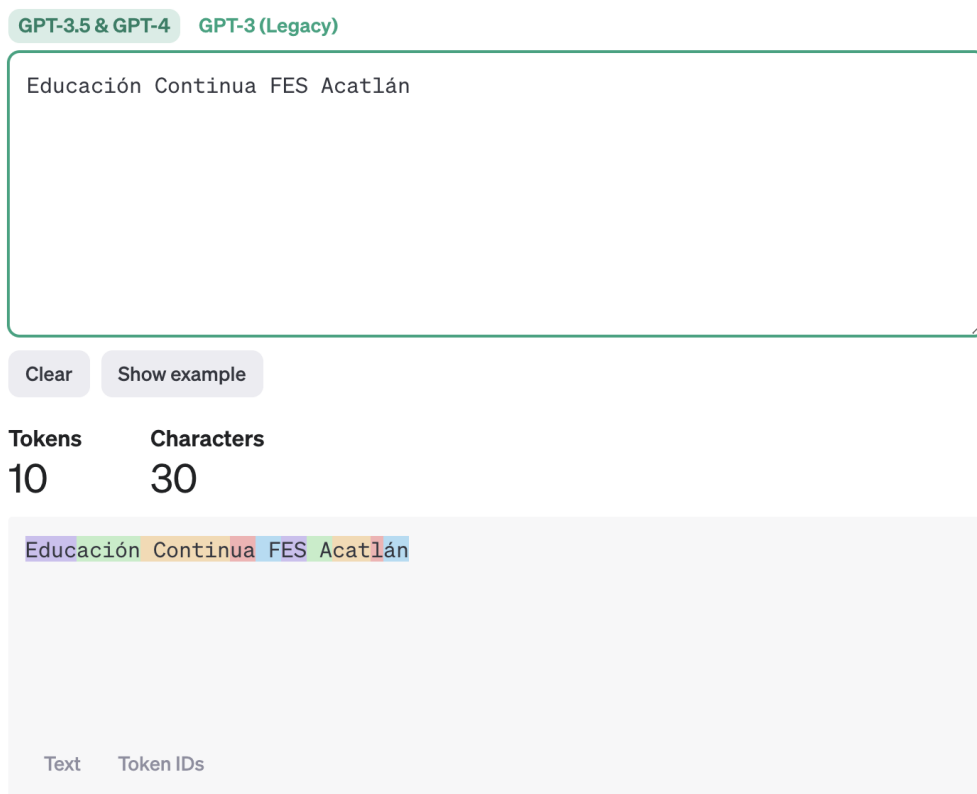


Figura 2.5: Ejemplo de Tokenización con el modelo de OpenAI <https://platform.openai.com/tokenizer>

Un estándar de tokenización comúnmente utilizado se conoce como tokenización de Penn Treebank, utilizado para los corpus analizados (bancos de árboles) publicados por el Linguistic Data Consortium (LDC), la fuente de muchos conjuntos de datos útiles. Este estándar separa las palabras y separa toda la puntuación.

Por ejemplo, esta herramienta nos permite contar tokens y entender como separaría el texto un modelo como el de Open AI (<https://platform.openai.com/tokenizer>). La figura 2.5 muestra algunos ejemplos.

Modelos de lenguaje N-gram

Los modelos que asignan probabilidades a las siguientes palabras, o secuencias de palabras en general, se denominan modelos de lenguaje o LM. ¿Por qué querríamos predecir las próximas palabras? ¡Resulta que los grandes modelos de lenguaje que revolucionaron el PLN moderno se entrenan simplemente prediciendo palabras! Los grandes modelos lingüísticos aprenden muchísimo sobre el lenguaje únicamente al entrenarlos para predecir las palabras siguientes a partir de palabras vecinas. Los modelos de lenguaje también pueden asignar una probabilidad a una oración completa.

En este capítulo presentamos el tipo de modelo de lenguaje más simple: el modelo de lenguaje de n-gramas. Un n-grama es una secuencia de n palabras: un 2-grama (que llamaremos bigrama) es una secuencia de dos palabras como “gira tu” o “tu turno”, y un 3-grama (un trigram) es una secuencia de tres palabras como “entrega tu tarea”.

Pero también (con un poco de ambigüedad terminológica) usamos la palabra ‘n-grama’ para referirnos a un modelo probabilístico que puede estimar la probabilidad de una palabra dadas las $n-1$ palabras anteriores y, por lo tanto, también asignar probabilidades a secuencias completas.

La expectativa es que al final del diplomado presentaremos los modelos de redes neuronales de lenguaje grande, mucho más poderosos, basados en la arquitectura transformers. Pero debido a que los n-grams tienen una formalización notablemente simple y clara, comenzamos nuestro estudio del modelado del lenguaje con ellos, introduciendo conceptos principales que desempeñan un papel en el modelado del lenguaje, conceptos como conjuntos de prueba y entrenamiento, perplejidad, muestreo e interpolación.

N-Grams

Comencemos con la tarea de calcular $P(w|h)$, la probabilidad de una palabra w dada una historia h . Supongamos que la historia h es “su agua es tan transparente que” y queremos saber la probabilidad de que la siguiente palabra sea:

$$P(él|su agua es tan transparente que) \tag{2.1}$$

Una forma de estimar esta probabilidad es a partir de recuentos de frecuencia relativa. Tomemos un corpus muy grande, cuente el número de veces que vemos que “su agua es tan transparente que” y cuente el número de veces que va seguido de ‘él’.

Esto sería responder a:

$$P(el|h) = \frac{C(\text{su agua es tan transparente que el})}{C(h)}, \quad (2.2)$$

$h = \text{“su agua es tan transparente que”}$

Con un corpus lo suficientemente grande, como la web, podemos calcular estos recuentos y estimar la probabilidad a partir de la ecuación (2.2). Parece una tarea titánica.

Por esta razón, necesitaremos introducir formas más inteligentes de estimar la probabilidad de una palabra w dada una historia h , o la probabilidad de una secuencia completa de palabras W . Comencemos formalizando un poco la notación.

Para representar la probabilidad de que una variable aleatoria particular X_i tome el valor “el”, o $P(X_i = el)$, usaremos la simplificación $P(el)$. Representaremos una secuencia de n palabras como $w_1 \dots w_n$ o $w_{1:n}$.

Para la probabilidad conjunta de que cada palabra en una secuencia tenga un valor particular $P(X_1 = w_1, X_2 = w_2, X_3 = w_3, \dots, X_n = w_n)$ usaremos $P(w_1, w_2, \dots, w_n)$.

Una cosa que podemos hacer es descomponer esta probabilidad usando la regla de la cadena de probabilidad:

$$\begin{aligned} P(X_1, \dots, X_n) &= P(X_1)P(X_2|X_1)P(X_3|X_{1:2})\dots P(X_n|X_{1:n-1}) \\ &= \prod_{k=1}^n P(X_k|X_{1:k-1}) \end{aligned} \quad (2.3)$$

Bayes ingenuo - Naive Bayes, clasificación de texto y sentimiento

Muchas tareas de procesamiento del lenguaje implican clasificación. En esta sección presentamos el algoritmo Naive Bayes y lo aplicamos a la categorización de texto, la tarea de asignar una etiqueta o categoría a un texto o documento completo.

Nos centramos en una tarea común de categorización de textos, el análisis de sentimientos, la extracción de sentimientos, la orientación positiva o negativa que un escritor expresa hacia algún objeto. Una reseña de una película, un libro o un producto en la web expresa el sentimiento del autor hacia el producto, mientras que un texto editorial o político expresa un sentimiento hacia un candidato o una acción política. Por lo tanto, extraer el sentimiento

del consumidor o del público es relevante para campos que van desde el marketing hasta la política.

La versión más simple del análisis de sentimientos es una tarea de clasificación binaria, y las palabras de la revisión brindan excelentes pistas.

La detección de spam es otra aplicación comercial importante, la tarea de clasificación binaria de asignar un correo electrónico a una de las dos clases de spam o no spam. Se pueden utilizar muchas características léxicas y de otro tipo para realizar esta clasificación. Por ejemplo, podría sospechar razonablemente de un correo electrónico que contenga frases como “SIN NINGÚN COSTO” o “Estimado Ganador”.

El objetivo de la clasificación es tomar una sola observación, extraer algunas características útiles y así clasificar la observación en una de un conjunto de clases discretas. Un método para clasificar texto es utilizar reglas escritas a mano por humanos.

La forma más común de clasificar textos en el procesamiento del lenguaje es mediante el aprendizaje automático supervisado, el tema de este capítulo. En el aprendizaje supervisado, tenemos un conjunto de datos de observaciones de entrada, cada una asociada con alguna salida correcta (una “señal de supervisión”). El objetivo del algoritmo es aprender cómo mapear desde una nueva observación hasta una salida correcta.

Formalmente, la tarea de la clasificación supervisada es tomar una entrada x y un conjunto fijo de clases de salida $Y = \{y_1, y_2, \dots, y_M\}$ y devolver una clase predicha $y \in Y$. Para la clasificación de texto, a veces hablemos de c (para “clase”) en lugar de y como nuestra variable de salida, y d (para “documento”) en lugar de x como nuestra variable de entrada.

En la situación supervisada tenemos un conjunto de entrenamiento de N documentos, cada uno de los cuales ha sido etiquetado manualmente con una clase: $\{(d_1, c_1), \dots, (d_N, c_N)\}$. Nuestro objetivo es aprender un clasificador que sea capaz de mapear desde un nuevo documento d a su clase correcta $c \in C$, donde C es un conjunto de clases de documentos útiles. Un clasificador probabilístico además nos dirá la probabilidad de que la observación esté en la clase. Esta distribución completa entre las clases puede ser información útil para decisiones posteriores; evitar tomar decisiones discretas desde el principio puede resultar útil al combinar sistemas.

Se utilizan muchos tipos de algoritmos de aprendizaje automático para crear clasificadores. Ahora presentamos el algoritmo Naive Bayes; el siguiente discutimos la regresión logística. Estos ejemplifican dos formas de hacer la clasificación. Los clasificadores generativos como el Naive Bayes construyen

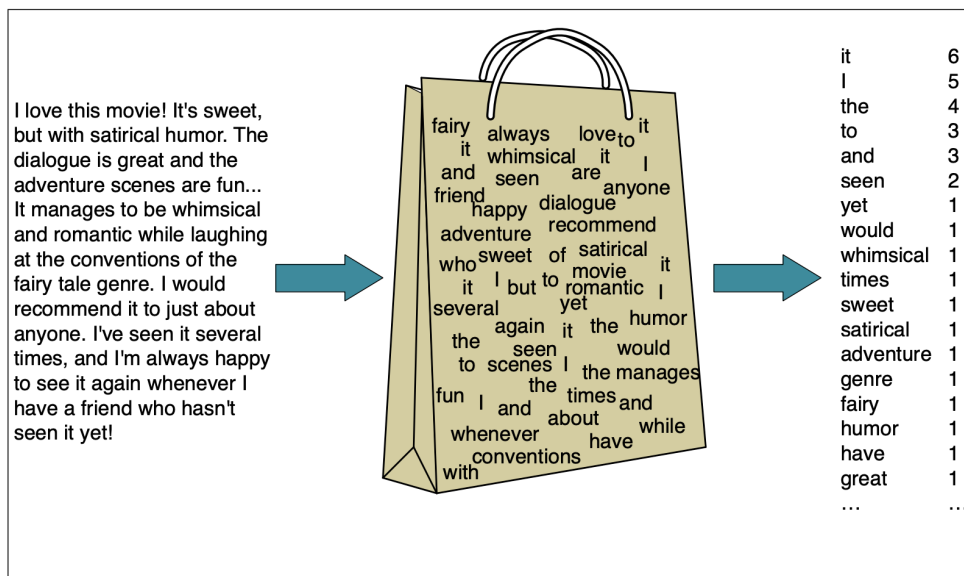


Figure 4.1 Intuition of the multinomial naive Bayes classifier applied to a movie review. The position of the words is ignored (the *bag-of-words* assumption) and we make use of the frequency of each word.

Figura 2.6: Retomado de Jurafsky y Martin (2023)

un modelo de cómo una clase podría generar algunos datos de entrada. Dada una observación, devuelven la clase que tiene más probabilidades de haber generado la observación. En cambio, los clasificadores discriminativos como la regresión logística aprenden qué características de la entrada son más útiles para discriminar entre las diferentes clases posibles. Si bien los sistemas discriminativos suelen ser más precisos y, por tanto, más utilizados, los clasificadores generativos siguen desempeñando un papel.

Naive Bayes Classifiers

Usaremos el clasificador multinomial Naive Bayes. La intuición del clasificador se muestra en la figura 2.6, donde representamos un documento de texto como si fuera una bolsa de palabras, es decir, un conjunto desordenado de palabras ignorando su posición, manteniendo solo su frecuencia en el documento. En el ejemplo de la figura, en lugar de representar el orden de las palabras en todas las frases como “Me encanta esta película” y “La recomendaría”, simplemente observamos que la palabra “I” apareció 5 veces en la extracción completa, la palabra “it” 6 veces, las palabras y así sucesivamente.

Naive Bayes es un clasificador probabilístico, lo que significa que para un

documento d , de todas las clases $c \in C$, el clasificador devuelve la clase $\hat{c}$ que tiene la probabilidad posterior máxima dado el documento. En la siguiente ecuación utilizamos la notación $\hat{}$ para referirnos a “nuestra estimación de la clase correcta”.

$$\hat{c} = \arg \max_{c \in C} P(c|d) \quad (2.4)$$

La intuición de la clasificación bayesiana es utilizar la regla de Bayes para transformar la ecuación (2.4) en otras probabilidades que tienen algunas propiedades útiles. La regla de Bayes se presenta en la ecuación (2.5); nos da una manera de descomponer cualquier probabilidad condicional $P(x|y)$ en otras tres probabilidades:

$$P(x|y) = \frac{P(y|x)P(x)}{P(y)} \quad (2.5)$$

Podemos sustituir la ecuación (2.5) en la ecuación (2.4):

$$\hat{c} = \arg \max_{c \in C} P(c|d) = \arg \max_{c \in C} \frac{P(d|c)P(c)}{P(d)} \quad (2.6)$$

Podemos simplificar convenientemente la ecuación (2.6) eliminando el denominador $P(d)$. Esto es posible porque calcularemos $P(d|c)P(c)/P(d)$ para cada clase posible. Pero $P(d)$ no cambia para cada clase. Siempre estamos preguntando por la clase más probable para el mismo documento d , que debe tener la misma probabilidad $P(d)$. Así, podemos elegir la clase que maximice esta fórmula más simple:

$$\hat{c} = \arg \max_{c \in C} P(c|d) = \max_{c \in C} P(d|c)P(c) \quad (2.7)$$

Llamamos a Naive Bayes como un modelo generativo porque podemos leer la ecuación (2.7) establece una especie de suposición implícita sobre cómo se genera un documento: primero se toma una muestra de una clase a partir de $P(c)$, y luego las palabras se generan mediante una muestra de $P(d|c)$. (De hecho, podríamos imaginarnos generar documentos artificiales, o al menos su recuento de palabras, siguiendo este proceso).

Para volver a la clasificación, calculamos la clase más probable $\hat{c}$ dado algún documento d eligiendo la clase que tiene el producto más alto de dos probabilidades. La probabilidad previa de la clase $P(c)$ y la probabilidad del documento $P(d|c)$:

$$\hat{c} = \arg \max_{c \in C} \overbrace{P(d|c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}} \quad (2.8)$$

Sin pérdida de generalidad, podemos representar un documento d como un conjunto de características $f_1, f_2, \dots, f_n$:

$$\hat{c} = \arg \max_{c \in C} \overbrace{P(f_1, f_2, \dots, f_n | c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}} \quad (2.9)$$

Desafortunadamente, la ecuación (2.9) todavía es demasiado difícil de calcular directamente. Sin algunas suposiciones simplificadoras, estimar la probabilidad de cada combinación posible de características (por ejemplo, cada conjunto posible de palabras y posiciones) requeriría una gran cantidad de parámetros y conjuntos de entrenamiento increíblemente grandes. Por lo tanto, los clasificadores Naive Bayes parten de dos supuestos simplificadores.

La primera es la suposición de la bolsa de palabras discutida intuitivamente anteriormente. Asumimos que la posición no importa y que la palabra “amor” tiene el mismo efecto en la clasificación ya sea que aparezca como la primera, vigésima o última palabra en el documento. Por lo tanto, asumimos que las características $f_1, f_2, \dots, f_n$ solo codifican la identidad de la palabra y no la posición.

La segunda se denomina comúnmente supuesto ingenuo de Bayes. Se trata del supuesto de independencia condicional de que las probabilidades $P(f_i | c)$ son independientes dada la clase c y, por tanto, pueden multiplicarse “ingenuamente” de la siguiente manera:

$$P(f_1, f_2, \dots, f_n | c) = P(f_1 | c) \cdot P(f_2 | c) \cdot \dots \cdot P(f_n | c) \quad (2.10)$$

Así, la ecuación final para la clase seleccionada por el clasificador Naive Bayes será:

$$c_{NB} = \arg \max_{c \in C} P(c) \prod_{f \in F} P(f | c) \quad (2.11)$$

Para aplicar el clasificador Naive Bayes al texto, debemos considerar las posiciones de las palabras, simplemente recorriendo un índice a través de cada posición de las palabras en el documento:

posiciones $\leftarrow$ todas las posiciones de palabras en el documento de prueba

$$c_{NB} = \arg \max_{c \in C} P(c) \prod_{i \in \text{Posiciones}} P(w_i | c) \quad (2.12)$$

Los cálculos del clasificador Naive Bayes, al igual que los cálculos para el modelado del lenguaje, se realizan en el *log* espacio para evitar el desbordamiento y aumentar la velocidad. Así, la ecuación (2.12) generalmente se

expresa como:

$$c_{NB} = \arg \max_{c \in C} \log P(c) + \sum_{i \in \text{Posiciones}} \log P(w_i|c) \quad (2.13)$$

Al considerar características en el espacio logarítmico, la ecuación (2.13) calcula la clase predicha como una función lineal de las características de entrada. Los clasificadores que utilizan una combinación lineal de entradas para tomar una decisión de clasificación (como el Naive Bayes y también la regresión logística) se denominan clasificadores lineales.

Training the Naive Bayes Classifier

¿Cómo podemos conocer las probabilidades $P(c)$ y $P(f_i|c)$? Consideremos primero la estimación de máxima verosimilitud. Simplemente usaremos las frecuencias en los datos. Para la clase prior $P(c)$ preguntamos qué porcentaje de los documentos en nuestro conjunto de entrenamiento están en cada clase c . Sea N_c el número de documentos en nuestros datos de entrenamiento con clase c y N_{doc} el número total de documentos. Entonces:

$$\hat{P}(c) = \frac{N_c}{N_{doc}} \quad (2.14)$$

Para conocer la probabilidad $P(f_i|c)$, asumiremos que una característica es simplemente la existencia de una palabra en la bolsa de palabras del documento, por lo que queremos $P(w_i|c)$, que calculamos como la fracción de veces aparece la palabra w_i entre todas las palabras en todos los documentos de la clase c . Primero concatenamos todos los documentos con categoría c en un gran texto de “categoría c ”. Luego usamos la frecuencia de w_i en este documento concatenado para dar una estimación de máxima verosimilitud de la probabilidad:

$$\hat{P}(w_i|c) = \frac{\text{count}(w_i, c)}{\sum_{w \in V} \text{count}(w, c)} \quad (2.15)$$

Aquí el vocabulario V consiste en la unión de todos los tipos de palabras de todas las clases, no sólo de las palabras de una clase c .

Sin embargo, existe un problema con el entrenamiento de máxima verosimilitud. Imaginemos que estamos tratando de estimar la probabilidad de que la palabra “fantástico” dado la clase positivo, pero supongamos que no hay documentos de entrenamiento que contengan la palabra “fantástico” y estén clasificados como positivo. Quizás la palabra “fantástico” aparezca

(¿sarcásticamente?) en la clase negativo. En tal caso, la probabilidad de esta característica será cero:

$$\hat{P}(\text{"fantastico"}|\text{positivo}) = \frac{\text{count}(\text{"fantastico"}, \text{positivo})}{\sum_{w \in V} \text{count}(w, \text{positivo})} = 0 \quad (2.16)$$

Regresión logística

Este tipo de técnicas se suelen ilustrar con esta analogía. Los cuentos de detectives están tan llenos de pistas como los textos lo están de palabras. Sin embargo, para el pobre lector puede ser un desafío saber cómo ponderar las pistas del autor para realizar la crucial tarea de clasificación: decidir quién es el culpable.

La regresión logística es una de las herramientas analíticas más importantes en las ciencias sociales y naturales. En el procesamiento del lenguaje natural, la regresión logística es el algoritmo de aprendizaje automático supervisado de línea base para la clasificación, y también tiene una relación muy estrecha con las redes neuronales—una red neuronal puede ser vista como una serie de clasificadores de regresión logística apilados uno encima del otro. Así, las técnicas de clasificación y aprendizaje automático introducidas aquí jugarán un papel importante en su formación de científicos de datos.

La regresión logística se puede utilizar para clasificar una observación en una de dos clases (como ‘sentimiento positivo’ y ‘sentimiento negativo’), o en una de varias clases. Debido a que los marcos para las tareas de clasificación más simples son más simples, describiremos este caso especial de regresión logística en las siguientes secciones, y luego resumiremos brevemente el uso de la regresión logística multinomial para más de dos clases.

Clasificadores Generativos y Discriminativos. La diferencia más importante entre naive Bayes y regresión logística es que la regresión logística es un **clasificador discriminativo** mientras que naive Bayes es un **clasificador generativo**.

Estos son dos marcos diferentes para cómo construir un modelo de aprendizaje automático. Considera una metáfora visual: imagina que estamos tratando de distinguir imágenes de perros de imágenes de gatos. Un modelo generativo tendría que comprender cómo se ven los perros y los gatos. Literalmente podrías pedirle a dicho modelo que ‘genere’, es decir, dibuje, un perro. Dado una imagen de prueba, el sistema luego pregunta si es el modelo de gato o el modelo de perro el que mejor se ajusta (está menos sorprendido por) la imagen, y elige eso como su etiqueta.

Un modelo discriminativo, por otro lado, solo está tratando de aprender a distinguir las clases (quizás sin aprender mucho sobre ellas). Entonces, tal vez todos los perros en el conjunto de datos de entrenamiento llevan collares y los gatos no. Si esa única característica separa claramente las clases, entonces el modelo está satisfecho. Si le preguntas a un modelo así lo que sabe sobre los gatos, todo lo que puede decir es que no llevan collares.

Recordemos que el modelo Bayes ingenuo asigna una clase c a un documento d no por el cálculo directo de $P(c|d)$, sino calculando una verosimilitud y un prior:

$$\hat{c} = \arg \max_{c \in C} \overbrace{P(d|c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}} \quad (2.17)$$

Un modelo generativo como el Naive Bayes utiliza este término de verosimilitud, que expresa cómo generar las características de un documento *si supiéramos que es de la clase c* . Por el contrario, un modelo discriminativo en este escenario de categorización de texto intenta calcular directamente $P(c|d)$. Quizás aprenda a asignar un gran peso a las características del documento que mejoran directamente su habilidad para discriminar entre clases posibles, incluso si no pudiera generar un ejemplo de una de las clases.

Componentes de un clasificador de aprendizaje automático probabilístico. Como el Naive Bayes, la regresión logística es un clasificador probabilístico dentro del aprendizaje supervisado. Los clasificadores de aprendizaje automático requieren un corpus de entrenamiento de m pares entrada/salida $(x^{(i)}, y^{(i)})$. (Utilizaremos superíndices en paréntesis para referirnos a instancias individuales en el conjunto de entrenamiento –para la clasificación de sentimientos cada instancia podría ser un documento individual a ser clasificado). Un sistema de aprendizaje automático para clasificación tiene entonces cuatro componentes:

1. Una **representación de características** de entrada. Para cada observación de entrada $x^{(i)}$, esto será un vector de características $[x_1, x_2, \dots, x_n]$. Nos referiremos generalmente a la característica i para la entrada $x^{(i)}$ como $x_j^{(i)}$, a veces simplificado como x_i , pero también veremos la notación f_i , $f(x)$, o, para clasificación multiclase, $f_i(c, x)$.
2. Una **función de clasificación** que computa, para la clase estimada, vía $p(y|x)$. Más adelante introduciremos las herramientas de la función sigmoide y softmax para clasificación.

3. Una **función objetivo** para el aprendizaje, usualmente involucrando minimizar el error en ejemplos de entrenamiento. Introduciremos la función de pérdida de entropía cruzada.
4. Un **algoritmo para optimizar la función objetivo**. Introducimos el algoritmo de descenso de gradiente estocástico.

La regresión logística tiene dos fases:

Entrenamiento: Entrenamos el sistema (específicamente los pesos w y el sesgo b) utilizando descenso de gradiente estocástico y la pérdida de entropía cruzada.

Prueba: Dado un ejemplo de prueba x , calculamos $p(y|x)$ y devolvemos la probabilidad más alta ya sea $y = 1$ o $y = 0$.

La función sigmoide

El objetivo de la regresión logística binaria es entrenar un clasificador que pueda tomar una decisión binaria sobre la clase de una nueva observación de entrada. Aquí introducimos la función **sigmoide** que ayudará a tomar esta decisión.

Considera una sola observación de entrada x , la cual representaremos por un vector de características $[x_1, x_2, \dots, x_n]$. El clasificador de salida será 1 (significando que la observación es miembro de la clase) o 0 (la observación no es miembro de la clase). Queremos saber la probabilidad $P(y = 1|x)$ de que la observación sea miembro de la clase. Así que la decisión es “positivo” versus “negativo”, las características representan conteos de palabras en un documento, $P(y = 1|x)$ es la probabilidad de que el documento tenga sentimiento positivo, y $P(y = 0|x)$ es la probabilidad de que el documento tenga sentimiento negativo.

La regresión logística resuelve esta tarea aprendiendo, de un conjunto de entrenamiento, un vector de pesos y un término de sesgo. Cada peso w_i está asociado con una de las características de entrada x_i . El peso w_i representa cuán importante es esa característica de entrada para la decisión de clasificación, y puede ser positivo (proporcionando evidencia de que la instancia que se clasifica pertenece a la clase positiva) o negativo (proporcionando evidencia de que la instancia que se clasifica pertenece a la clase negativa). Así, esperaríamos que en una tarea de análisis de sentimiento la palabra **increíble** tenga un peso positivo alto, y **pésimo** un peso negativo muy alto.

El término de sesgo, también llamado el **intercepto**, es otro número real que se añade a las entradas ponderadas.

Para tomar una decisión en una instancia de prueba –después de haber aprendido los pesos en el entrenamiento– el clasificador primero multiplica cada x_i por su peso w_i , suma las características ponderadas, y añade el término de sesgo b . El número único resultante z expresa la suma ponderada de la evidencia para la clase.

$$z = \left(\sum_{i=1}^n w_i x_i \right) + b \quad (2.18)$$

En adelante representaremos tales sumas usando la notación de **producto punto** del álgebra lineal. El producto punto de dos vectores **a** y **b**, escrito como $\mathbf{a} \cdot \mathbf{b}$, es la suma de los productos de los elementos correspondientes de cada vector. Por lo tanto, lo siguiente es una representación equivalente a la Ecuación (2.18):

$$z = \mathbf{w} \cdot \mathbf{x} + b \quad (2.19)$$

Pero nótese que nada en la Ecuación (2.19) obliga a z a ser una probabilidad, es decir, a estar entre 0 y 1. De hecho, dado que los pesos son valores reales, la salida podría ser incluso negativa; z varía de $-\infty$ a ∞ .

Para crear una probabilidad, pasaremos z a través de la función **sigmoide**, $\sigma(z)$. La función sigmoide (llamada así porque se parece a una 's') también se llama la función **logística**, y le da su nombre a la regresión logística. La sigmoide tiene la siguiente ecuación, y se muestra gráficamente en la Figura 2.7:

$$\sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + \exp(-z)} \quad (2.20)$$

La función sigmoide tiene varias ventajas; toma un número real y lo mapea dentro del rango (0,1), que es justo lo que queremos para una probabilidad. Porque es casi lineal alrededor del 0 pero se aplana hacia los extremos, tiende a aplastar valores atípicos hacia 0 o 1. Es diferenciable.

Si aplicamos el sigmoide a la suma de las características ponderadas, obtenemos un número entre 0 y 1. Para hacerlo una probabilidad, solo necesitamos asegurarnos de que los dos casos, $p(y = 1)$ y $p(y = 0)$, sumen 1.

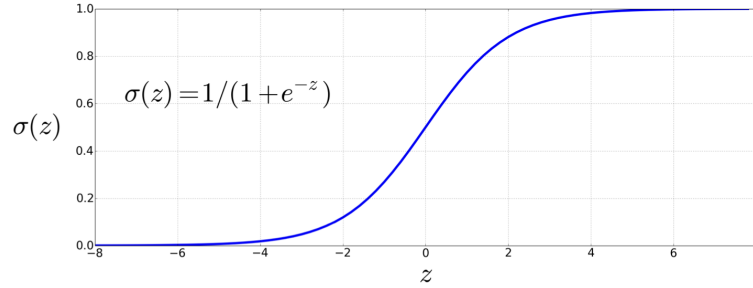


Figura 2.7: La función sigmoide $\sigma(z) = \frac{1}{1+e^{-z}}$ toma un valor real y lo mapea al rango $(0, 1)$. Es casi lineal alrededor de 0, pero los valores atípicos se comprimen hacia 0 o 1. Retomado de Jurafsky y Martin (2023)

Podemos hacer esto de la siguiente manera:

$$\begin{aligned}
 P(y = 1) &= \sigma(\mathbf{w} \cdot \mathbf{x} + b) \\
 &= \frac{1}{1 + \exp(-(\mathbf{w} \cdot \mathbf{x} + b))} \\
 P(y = 0) &= 1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b) \\
 &= 1 - \frac{1}{1 + \exp(-(\mathbf{w} \cdot \mathbf{x} + b))} \\
 &= \frac{\exp(-(\mathbf{w} \cdot \mathbf{x} + b))}{1 + \exp(-(\mathbf{w} \cdot \mathbf{x} + b))}
 \end{aligned} \tag{2.21}$$

La función sigmoide tiene la propiedad de que

$$1 - \sigma(x) = \sigma(-x) \tag{2.22}$$

así que también podríamos haber expresado $P(y = 0)$ como $\sigma(-(\mathbf{w} \cdot \mathbf{x} + b))$.

La entrada a la función sigmoide, el puntaje $z = \mathbf{w} \cdot \mathbf{x} + b$ (de la Ecuación (2.19)), a menudo se llama el **logit**. Esto es porque la función logit es la inversa de la sigmoide. La función logit es el logaritmo de la razón de probabilidades $\frac{p}{1-p}$:

$$\text{logit}(p) = \sigma^{-1}(p) = \ln \left(\frac{p}{1-p} \right) \tag{2.23}$$

Usar el término **logit** para z es una forma de recordarnos que al usar el sigmoide para convertir z (que varía de $-\infty$ a ∞) en una probabilidad,

estamos interpretando implícitamente a z no solo como cualquier número real, sino específicamente como un logaritmo de probabilidades.

Clasificación con Regresión Logística.

La función sigmoide de la sección anterior nos proporciona una manera de tomar una característica x y calcular la probabilidad $P(y = 1|x)$. ¿Cómo tomamos una decisión sobre qué clase aplicar a una característica de prueba x ? Para un x dado, decimos que sí si la probabilidad $P(y = 1|x)$ es mayor a 0.5, y no en caso contrario. Llamamos a esto el **límite de decisión**:

$$\text{decisión}(x) = \begin{cases} 1 & \text{si } P(y = 1|x) > 0.5 \\ 0 & \text{en caso contrario} \end{cases} \quad (2.24)$$

Veamos algunos ejemplos de aplicación de la regresión logística como clasificador para tareas de lenguaje.

Clasificación de Sentimientos

Supongamos que estamos haciendo clasificación de sentimientos binarios en textos de reseñas de películas, y nos gustaría saber si asignar la clase de sentimiento positivo o negativo a un documento de prueba.

Var	Definition	Value in Fig. 2.8
x_1	count(positive lexicon words $\in$ doc)	3
x_2	count(negative lexicon words $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "?" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	ln(word count of doc)	ln(66) = 4.19

Supongamos por el momento que ya hemos aprendido un peso real para cada característica de la función logística que hemos descrito.

Cada una de estas características, y los 6 pesos correspondientes a las 6 características son $[2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$, mientras que $b = 0.1$. (Más adelante discutiremos cómo se aprenden los pesos). El peso w_1 , por ejemplo, indica qué tan importante es la característica del número de palabras positivas del léxico (como *genial*, *agradable*, *disfrutable*, etc.) para una decisión de sentimiento positivo, mientras que w_2 nos dice la importancia de las palabras negativas del léxico. Nota que $w_1 = 2.5$ es positivo, mientras que $w_2 = -5.0$, lo que significa que las palabras negativas están negativamente asociadas con

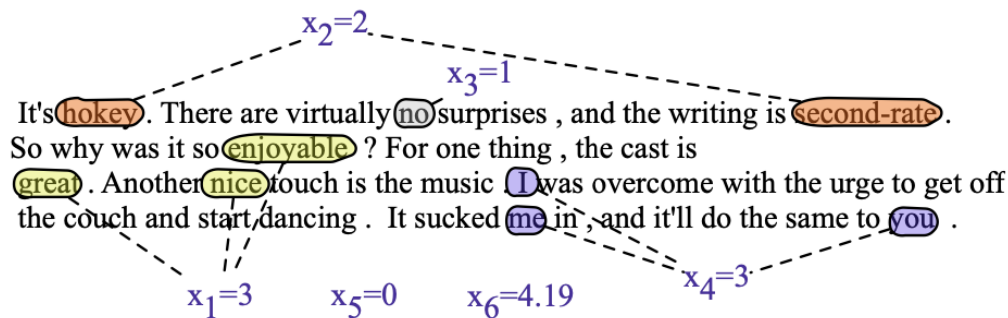


Figura 2.8: Un documento de prueba mini muestra que exhibe las características extraídas en el vector x . Retomado de Jurafsky y Martin (2023)

una decisión de sentimiento positivo, y son aproximadamente el doble de importantes que las palabras positivas.

Dadas estas 6 características y la revisión de entrada x , $P(+|x)$ y $P(-|x)$ se pueden calcular usando la Ecuación 2.21:

$$\begin{aligned}
 p(+|x) &= P(y = 1|x) = \sigma(\mathbf{w} \cdot \mathbf{x} + b) \\
 &= \sigma(\mathbf{w} \cdot \mathbf{x} + 0.1) \\
 &= \sigma(.833) \\
 &= 0.70
 \end{aligned} \tag{2.25}$$

$$\begin{aligned}
 p(-|x) &= P(y = 0|x) = 1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b) \\
 &= 0.30
 \end{aligned}$$

Otras tareas de clasificación y características

La regresión logística se aplica comúnmente a todo tipo de tareas de PLN, y cualquier propiedad de la entrada puede ser una característica. Considera la tarea de desambiguación de puntos: decidir si un punto es el final de una oración o parte de una palabra, clasificando cada punto en una de dos clases EOS (fin-de-oración) y no-EOS. Podríamos usar características como x_i abajo expresando que la palabra actual está en minúsculas (quizás con un peso positivo), o que la palabra actual está en nuestro diccionario de abreviaturas ("Prof.") (quizás con un peso negativo). Una característica también puede

expresar una combinación bastante compleja de propiedades. Por ejemplo, un punto después de una palabra en mayúscula es probablemente un EOS, pero si la palabra misma es *St.* y la palabra anterior está en mayúsculas, entonces el punto probablemente es parte de una abreviación de la palabra *street*.

$$x_1 = \begin{cases} 1 & \text{si Case}(w_i) = \text{Lower} \\ 0 & \text{en caso contrario} \end{cases} \quad (2.26)$$

$$x_2 = \begin{cases} 1 & \text{si } w_i \in \text{AcronymDict} \\ 0 & \text{de lo contrario} \end{cases} \quad (2.27)$$

$$x_3 = \begin{cases} 1 & \text{si } w_i = \text{St. y Case}(w_{i-1}) = \text{Cap} \\ 0 & \text{de lo contrario} \end{cases} \quad (2.28)$$

Diseñando características: Las características generalmente se diseñan examinando el conjunto de entrenamiento con un ojo en las intuiciones lingüísticas y la bibliografía existente en el dominio. Un análisis cuidadoso de errores en el conjunto de entrenamiento o en una versión temprana de un sistema a menudo proporciona información sobre las características.

Para algunas tareas es especialmente útil construir características complejas que son combinaciones de características más primitivas. Vimos tal característica para la desambiguación de periodos anteriormente, donde un punto en la palabra *St.* era menos probable que fuera el final de la oración si la palabra anterior estaba capitalizada. Las características de combinación o *interacciones de características* tienen que ser diseñadas a mano.

Para muchas tareas (especialmente cuando los valores de las características pueden referenciar palabras específicas) se necesitan grandes números de características. A menudo se crean automáticamente a partir de plantillas de características, especificaciones abstractas de características. Por ejemplo, una plantilla de bigrama podría especificar que se cree una característica para cada par de palabras antes de un punto en el conjunto de entrenamiento. Por lo tanto, el espacio de características es disperso, ya que solo creamos una característica si ese n-grama existe en esa posición en el conjunto de entrenamiento. La característica generalmente se crea como un hash del string en las descripciones. Un descriptor de la característica como “bigram(American breakfast)” se convierte en un número único entero i que se convierte en el número de característica f_i .

Para evitar el extensivo esfuerzo humano en el diseño de características, la investigación reciente en PLN se ha centrado en el *aprendizaje de representación*: formas de aprender características automáticamente de una manera no supervisada a partir de la entrada—este tema escapa de este diplomado, pero es una buena opción para estudios posteriores—.

Estandarizando las características de entrada: Cuando diferentes características de entrada tienen rangos de valores extremadamente diferentes, es común reescalarlos para que tengan rangos comparables. Estandarizamos los valores de entrada centrando los valores para que resulten en una media cero y una desviación estándar de uno (esta transformación a veces se llama la transformación-z). Es decir, si μ_i es la media de los valores de la característica x_i a través de las m observaciones en el conjunto de datos de entrada, y σ_i es la desviación está

$$\mu_i = \frac{1}{m} \sum_{j=1}^m x_i^{(j)}$$

$$\sigma_i = \sqrt{\frac{1}{m} \sum_{j=1}^m \left(x_i^{(j)} - \mu_i\right)^2}$$

$$x'_i = \frac{x_i - \mu_i}{\sigma_i}$$

Alternativamente, podemos **normalizar** los valores de las características de entrada para que estén entre 0 y 1:

$$x'_i = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$$

Tener datos de entrada con un rango comparable es útil al comparar valores entre características. **El escalado de datos es especialmente importante en grandes redes neuronales, ya que ayuda a acelerar el descenso de gradiente.**

Procesando muchos ejemplos a la vez

Hemos mostrado las ecuaciones para la regresión logística para un solo ejemplo. Pero en la práctica, por supuesto queremos procesar un conjunto de pruebas entero con muchos ejemplos. Supongamos que tenemos un conjunto de pruebas consistente en m ejemplos de prueba cada uno de los cuales nos gustaría clasificar. Continuaremos usando la notación inicial, en la cual un superíndice entre paréntesis se refiere al índice del ejemplo en algún conjunto

de datos (ya sea para entrenamiento o para pruebas). Así que en este caso cada ejemplo de prueba $x^{(i)}$ tiene un vector de características $x^{(i)}$, $1 \leq i \leq m$. (Como es usual, representaremos vectores y matrices en negrita).

Una forma de calcular cada valor de salida $y^{(i)}$ es simplemente tener un bucle ‘for’ y calcular cada ejemplo de prueba uno a la vez:

$$\begin{aligned} \text{foreach } x^{(i)} \text{ in input } [x^{(1)}, x^{(2)}, \dots, x^{(m)}] \\ y^{(i)} = \sigma(\mathbf{w} \cdot x^{(i)} + b) \end{aligned} \quad (2.29)$$

Para los primeros 3 ejemplos de prueba, entonces, estaríamos calculando las predicciones $y^{(i)}$ de la siguiente manera:

$$\begin{aligned} P(y^{(1)} = 1 | x^{(1)}) &= \sigma(\mathbf{w} \cdot x^{(1)} + b) \\ P(y^{(2)} = 1 | x^{(2)}) &= \sigma(\mathbf{w} \cdot x^{(2)} + b) \\ P(y^{(3)} = 1 | x^{(3)}) &= \sigma(\mathbf{w} \cdot x^{(3)} + b) \end{aligned}$$

Pero resulta que podemos modificar ligeramente nuestra ecuación original Ecuación (2.19) para hacer esto mucho más eficientemente. Usaremos aritmética de matrices para asignar una clase a todos los ejemplos con una sola operación de matriz.

Primero, empaquetaremos todos los vectores de características de entrada para cada entrada x en una única matriz de entrada X , donde cada fila i es un vector fila consistiendo en el vector de características para el ejemplo de entrada $x^{(i)}$ (es decir, el vector $x^{(i)}$). Asumiendo que cada ejemplo tiene f características y pesos, X será entonces una matriz de forma $[m \times f]$, de la siguiente manera:

$$X = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_f^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_f^{(2)} \\ x_1^{(3)} & x_2^{(3)} & \cdots & x_f^{(3)} \\ \vdots & \vdots & \ddots & \vdots \end{bmatrix} \quad (2.30)$$

Ahora, si introducimos b como un vector de longitud m que consiste en el sesgo escalar término $\mathbf{b}$ repetido m veces, $b = [\mathbf{b}, b, \dots, b]$, y $\mathbf{y}$ como el vector de salidas (un escalar $y^{(i)}$ para cada entrada $x^{(i)}$ y su vector de características $x^{(i)}$), y representamos el peso $\mathbf{w}$ como un vector columna, podemos calcular todas las salidas con una sola multiplicación de matriz y una adición.

Ahora, si introducimos $\mathbf{b}$ como un vector de longitud m que consiste en el término de sesgo escalar b repetido m veces, $\mathbf{b} = [b, b, \dots, b]$, y $\hat{y} = [\hat{y}^{(1)}, \hat{y}^{(2)}, \dots, \hat{y}^{(m)}]$ como el vector de salidas (un escalar $\hat{y}^{(i)}$ para cada entrada $x^{(i)}$ y su vector de características $x^{(i)}$), y representamos el vector de pesos w como un vector columna, podemos computar todas las salidas con una sola multiplicación de matrices y una adición:

$$y = \mathbf{X}\mathbf{w} + \mathbf{b} \quad (2.31)$$

Esta ecuación calcula lo mismo que nuestro bucle for en la Ecuación (2.29). Por ejemplo, $\hat{y}^{(1)}$, la primera entrada del vector de salida y , será correctamente:

$$\hat{y}^{(1)} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \dots & x_f^{(1)} \end{bmatrix} \cdot \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_f \end{bmatrix} + \mathbf{b} \quad (2.32)$$

Así, la Ecuación 2.30 con las dimensiones mostradas:

$$\begin{aligned} y &= \mathbf{X}\mathbf{w} + \mathbf{b} \\ (m \times 1) &= (m \times f)(f \times 1) + (m \times 1) \end{aligned} \quad (2.33)$$

2.1.2. Ejemplos y aplicaciones de minado de texto.

El código del ejemplo se encuentra en GitHub con el nombre: “11_Expresiones_Regulares” y “12_Leer_Datos_PDF” en GitHub.

2.1.3. Implementación de modelo de lenguaje natural con N-Gramas.

El código del ejemplo se encuentra en GitHub con el nombre: “13_N_Grams” en GitHub.

2.1.4. Clasificación de texto con Naive Bayes.

El código del ejemplo se encuentra en GitHub con el nombre: “14_Naive_Bayes” en GitHub.

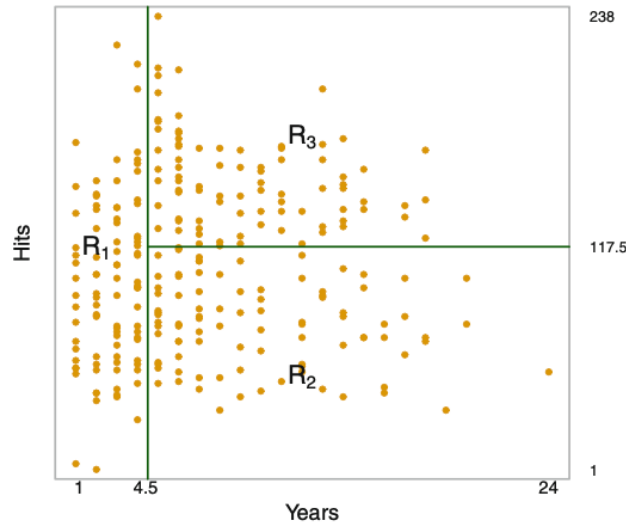


FIGURE 8.2. *The three-region partition for the **Hitters** data set from the regression tree illustrated in Figure 8.1.*

Figura 2.9: Ejemplo de regiones. Retomado de Hastie et al (2009, 305)

2.1.5. Clasificación de texto con regresión logística.

El código del ejemplo se encuentra en GitHub con el nombre: “15_Logistic_Regression” en GitHub.

2.2. Modelos de árboles de decisión y bosques aleatorios.

Métodos basados en árboles

En esta sección discutiremos los métodos basados en árboles para regresión y clasificación, siguiendo a James, Witten, Hastie y Tibshirani (2013, Cap. 8) [Jam+13]. Estos métodos implican estratificar o segmentar el espacio de los predictores en un número J de regiones. Podemos ver el ejemplo de la Figura 2.9.

En esta sección introduciremos las técnicas de bagging, random forest y boosting. Cada uno de ellos involucra múltiples árboles, los cuales son

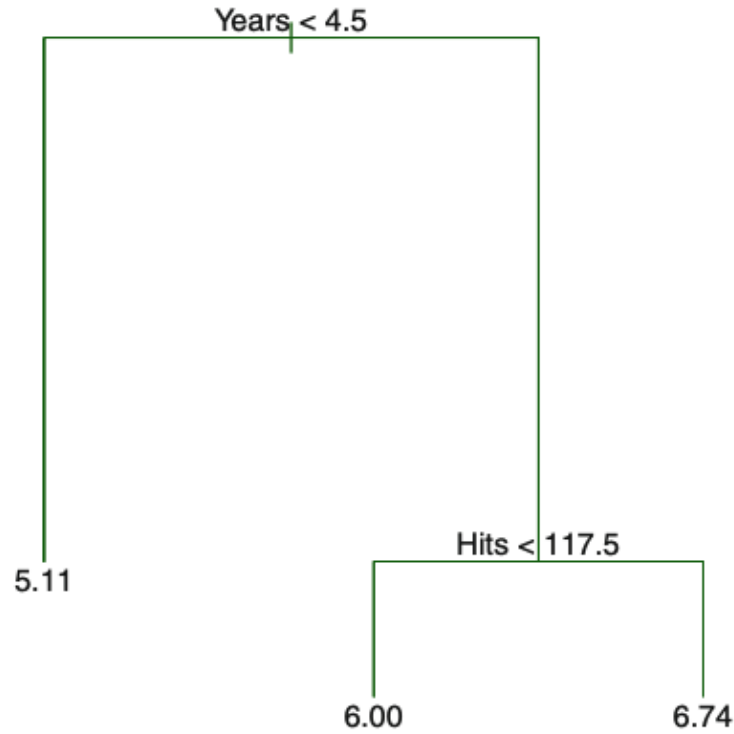


Figura 2.10: Ejemplo de árbol. Retomado de Hastie et al (2009, 304)

combinados para alcanzar un consenso en la predicción.

Bases de los árboles de decisión. Estos pueden aplicar a casos de regresión y clasificación. Una forma simple de verlo es la Figura 2.10, que muestra el proceso de segmentación a través de un árbol.

En estos ejemplos, esas R_1 , R_2 y R_3 son conocidas como nodos terminales u hojas del árbol. El proceso de delimitación de esos nodos terminales se lleva a cabo en 2 etapas:

1. Dividimos los predictores $(X_1, X_2, \dots, X_p)$ en J regiones distingibles, que denominamos $R_1, R_2, \dots, R_J$.
2. Para cada observación i que se ubique dentro de cada región R_j hacemos la misma predicción, la cual puede ser simplemente la media de los valores de la variable explicada en el conjunto de entrenamiento.

Pero ¿cómo construimos las regiones? La respuesta es responder a encontrar el valor mínimo o minimizar la suma de errores al cuadrado (RSS, por sus siglas en inglés) dadas las $R_1, R_2, \dots, R_J$:

$$RSS = \sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2 \quad (2.34)$$

Donde $\hat{y}_{R_j}$ es la media de las observaciones o variable explicada de entrenamiento dentro de cada región j -ésima. La implementación se puede ver como el siguiente proceso de 'división binaria recursiva':

- Seleccionamos el predictor X_j que se parte en las regiones: $\{X|X_j < s\}$ y $\{X|X_j \geq s\}$
- Corremos este análisis para todos los predictores $X_1, X_2, \dots, X_p$ y para cada cuota s para cada predictor y región R_j .

Un solo árbol, ajustado de esta manera, tiene una limitación importante: es **inestable** –pequeños cambios en los datos de entrenamiento pueden producir particiones muy distintas– y tiende a sobreajustar (varianza alta) si se deja crecer sin restricción. Bagging, random forest y boosting son tres formas distintas de resolver ese problema combinando *muchos* árboles, en vez de confiar en uno solo.

Bagging (Bootstrap Aggregating). La idea es simple: si tuviéramos muchas muestras de entrenamiento independientes, podríamos ajustar un árbol a cada una y promediar sus predicciones, lo que reduce la varianza sin aumentar el sesgo. Como en la práctica solo tenemos una muestra, bagging simula esa idea con **bootstrap**: genera B muestras de entrenamiento, cada una obtenida al muestrear con reemplazo del conjunto de datos original (mismo tamaño n , pero con observaciones repetidas y otras ausentes), ajusta un árbol grande (sin poda) a cada muestra bootstrap, y promedia las B predicciones:

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x) \quad (2.35)$$

para regresión (en clasificación, se usa el voto mayoritario entre los B árboles en vez del promedio). Como cada árbol individual se deja crecer profundo (bajo sesgo, alta varianza), y el promedio de muchos estimadores con esa propiedad reduce la varianza total, bagging típicamente mejora bastante

sobre un solo árbol podado. Una ventaja adicional: las observaciones que quedaron fuera de cada muestra bootstrap (en promedio, $\sim 37\%$ de los datos, las llamadas *out-of-bag* u OOB) se pueden usar para estimar el error de predicción sin necesidad de un conjunto de validación separado.

Random Forest. Bagging por sí solo tiene un problema: si hay un predictor muy fuerte, la mayoría de los árboles bootstrap lo van a usar en la primera partición, lo que hace que los B árboles se parezcan mucho entre sí (estén *correlacionados*). Promediar árboles correlacionados reduce la varianza menos que promediar árboles independientes. Random forest resuelve esto agregando aleatoriedad extra: en cada partición de cada árbol, en vez de considerar los p predictores completos, solo se considera un subconjunto aleatorio de tamaño $m < p$ (típicamente $m \approx \sqrt{p}$ para clasificación, o $m \approx p/3$ para regresión). Esto obliga a que distintos árboles usen distintos predictores como los más importantes en sus primeras particiones, **decorrelacionando** los árboles y mejorando la reducción de varianza del promedio final.

Boosting. A diferencia de bagging y random forest –donde los árboles se ajustan de forma independiente y en paralelo, cada uno sobre su propia muestra bootstrap–, boosting ajusta los árboles **secuencialmente**: cada árbol nuevo se entrena para corregir los errores (residuales) que dejó el conjunto de árboles ya ajustado, no sobre una muestra bootstrap nueva. El procedimiento general (boosting para regresión) es:

1. Inicializar $\hat{f}(x) = 0$ y los residuales $r_i = y_i$ para toda i .
2. Para $b = 1, 2, \dots, B$: ajustar un árbol **pequeño** $\hat{f}^b$ (con pocos nodos terminales) a los datos (X, r) ; actualizar el modelo agregando una versión "cogida" del nuevo árbol, $\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x)$; y actualizar los residuales, $r_i \leftarrow r_i - \lambda \hat{f}^b(x_i)$.
3. El modelo boosted final es $\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x)$.

Aquí $\lambda > 0$ es la **tasa de aprendizaje** (*learning rate*, típicamente un valor pequeño como 0.01 o 0.001): controla qué tan rápido aprende el ensamble, y valores más pequeños generalmente requieren más árboles (B más grande) pero producen mejores resultados al evitar que un solo árbol domine el ajuste. A diferencia de bagging/random forest, cada árbol de boosting es intencionalmente pequeño y débil (por eso boosting suele describirse como una forma de combinar muchos "aprendices débiles" en un aprendiz fuerte); y como el proceso es secuencial y depende de los residuales acumulados, boosting

puede sobreajustar si B es demasiado grande, a diferencia de bagging/random forest, donde agregar más árboles casi nunca empeora el desempeño. Las implementaciones modernas más usadas en la práctica (*gradient boosting*, y variantes optimizadas como XGBoost o LightGBM) generalizan esta misma idea –ajustar secuencialmente sobre el gradiente de una función de pérdida– más allá del error cuadrático.

Ejemplo de Random Forest

El código del ejemplo se encuentra en GitHub con el nombre: “01_Decision_Trees_and_Random_Forests_Mejorado” en GitHub.

2.3. Modelos de redes neuronales.

2.3.1. Introducción y Motivación

El procedimiento de estimación mediante redes neuronales es un método que puede ser usado tanto para clasificación como para predicción. En ocasiones, este es considerado como una caja negra en términos de la interpretación de los parámetros estimados.

Quizá la principal ventaja de este método es que tiene una amplia exactitud o capacidad predictiva. Por esta razón se suele considerar que las redes neuronales son una herramienta computacional fundamental.

Las redes neuronales (también denominadas redes neuronales artificiales) están basadas en el funcionamiento biológico del cerebro, donde las neuronas están interconectadas y aprenden de la experiencia.

El origen de la técnica se ubica en el trabajo de McCulloch - Pitts (1943) [MP43], en el cual se propone un modelo simplificado de una neurona basada en el funcionamiento biológico de las neuronas con el objetivo de establecer un resultado en términos de una proposición lógica.

Su estructura soporta/captura relaciones altamente complejas entre las variables usadas como predictores y la variable objetivo o explicada. Situación que no siempre es posible con otros métodos.

El uso moderno de las redes neuronales es llamado aprendizaje profundo (deep learning), ya que las redes modernas son profundas (deep), es decir, que tienen múltiples capas ocultas.

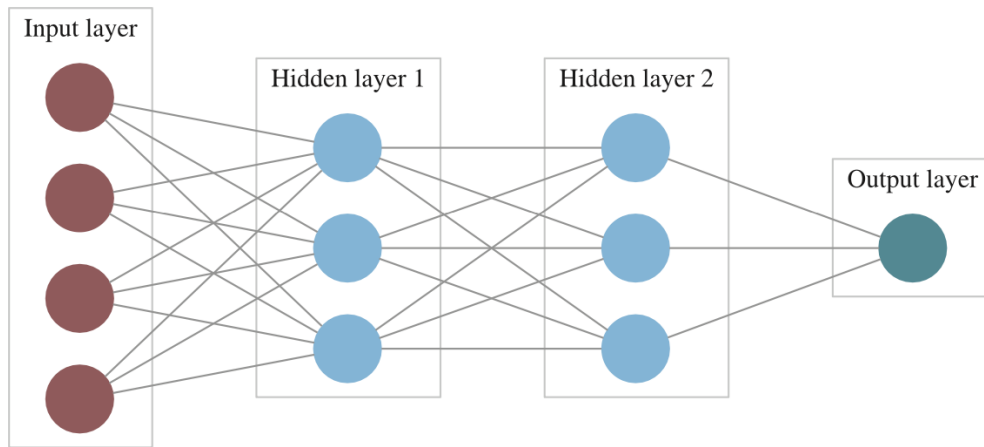


Figura 2.11: Ejemplo de redes neuronales. Retomado de Shmueli et al (2019, 285) [Shm+19]

2.3.2. Algunos conceptos relevantes

En comparación con un proceso de regresión, las redes neuronales no requieren que los usuarios/analistas determinen una forma funcional o forma específica de la relación entre los predictores y la variable objetivo. En cambio, las redes neuronales tratan de aproximar/inferir/determinar la relación a partir de los datos.

De hecho, la regresión lineal y logística son un caso particular de las redes neuronales en la que solo hay una capa que conjunta a los inputs y los outputs, y no hay capas ocultas. Las capas ocultas son aquellas que se ubican entre la capa de input y la capa de output. La Figura 2.11 ilustra esta estructura de capas.

Otros conceptos relevantes son los nodos input y output. Si tenemos p predictores, la capa de input usualmente incluya p -nodos. Así, las capas ocultas tienen como su input al output de la capa input, y así sucesivamente.

Unidades neuronales - neurona. En su corazón, una unidad neuronal (neurona) toma la suma ponderada de sus inputs con un término adicional (término de sesgo), θ_j .

Dado un conjunto de inputs $\{x_1, x_2, \dots, x_p\}$, una unidad toma un conjunto de pesos $\theta_j, w_{1j}, w_{2j}, \dots, w_{pj}$. Para computar el output de una capa de nodos oculta calculamos la suma ponderada de los input y aplicamos una función de

activación a ellos. Es decir, dado un conjunto de valores input $\{x_1, x_2, \dots, x_p\}$ calculamos el output del nodo $j = 1, 2, \dots, J$:

$$Z_j = \theta_j + \sum_{i=1}^p w_{ij}x_i = \theta_j + \mathbf{W}_j \cdot \mathbf{X} \quad (2.36)$$

Donde los pesos $\theta_j, w_{1j}, w_{2j}, \dots, w_{pj}$ se obtienen mediante el aprendizaje. Adicionalmente, θ_j también es conocido como el sesgo del nodo j , es una constante que controla el nivel de la contribución del nodo j .

Finalmente, aplicamos una función que denominaremos como función de activación para una unidad a . Así, podemos implementar una función de activación para el nodo j del cual obtenemos el output:

$$Output_j = a = f(Z_j) \quad (2.37)$$

Función que puede ser, por ejemplo, una función logística:

$$Output_j = f(\theta_j + \mathbf{W}_j \cdot \mathbf{X}) = \frac{1}{e^{-(\theta_j + \sum_{i=1}^p w_{ij}x_i)}} = \frac{1}{e^{-(\theta_j + \mathbf{W}_j \cdot \mathbf{X})}} \quad (2.38)$$

En este punto debemos observar que el proceso de determinación de los pesos $\theta_j, w_{1j}, w_{2j}, \dots, w_{pj}$ es iterativo y suele considerarse como valores iniciales a valores pequeños aleatorios. Para simplificar asumiremos que el término de sesgo, θ_j , está contenido como uno de los predictores $\mathbf{X}$ y se determina como parte de los pesos $\mathbf{W}_j$, de esta forma:

$$Z_j = \sum_{i=0}^p w_{ij}x_i = \mathbf{W}_j \cdot \mathbf{X} \quad (2.39)$$

Donde, el predictor ubicado en la posición $i = 0$ será un uno 1.

En relación a la función de activación, en la práctica se suele hacer uso de 3:

1. Sigmoide, la cual se ilustra en la Figura 2.12.

$$y = \sigma(Z) = \frac{1}{1 + e^{-Z}} \quad (2.40)$$

2. tanh, la cual se ilustra en la Figura 2.13.

$$y = \tanh(Z) = \frac{e^Z - e^{-Z}}{e^Z + e^{-Z}} \quad (2.41)$$

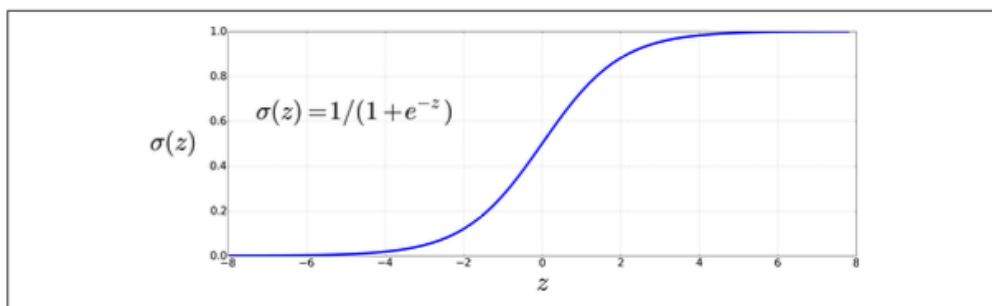


Figure 7.1 The sigmoid function takes a real value and maps it to the range $(0, 1)$. It is nearly linear around 0 but outlier values get squashed toward 0 or 1.

Figura 2.12: Ejemplo de la función de activación Sigmoide. Retomado de Jurafsky, et al (2023, Cap. 7, p.2) [JM23]

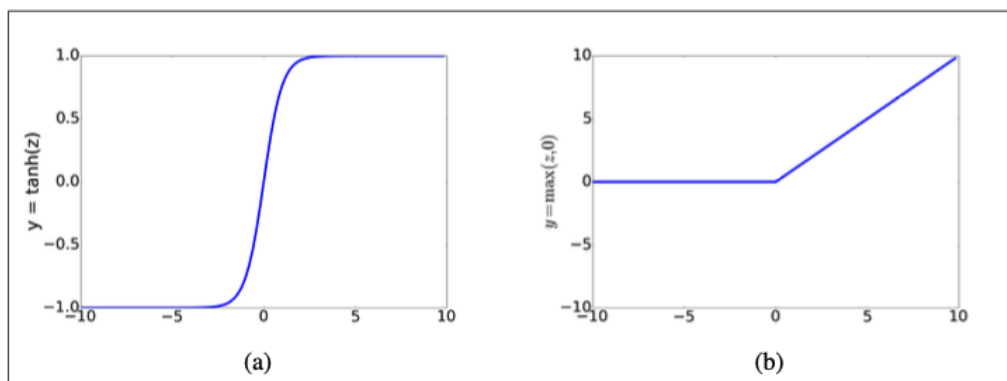


Figure 7.3 The tanh and ReLU activation functions.

Figura 2.13: Ejemplo de la función de activación $\tanh$. Retomado de Jurafsky, et al (2023, Cap. 7, p.4) [JM23]

3. ReLU (Rectified Linear Unit), la cual se ilustra en la Figura 2.13.

$$y = ReLU(Z) = \max(Z, 0) \quad (2.42)$$

Tomemos el siguiente ejemplo. Supongamos los valores para un nodo j :

$$\mathbf{W}_j = [0.5 \quad 0.2 \quad 0.3 \quad 0.9]$$

$$\mathbf{X} = \begin{bmatrix} 1 \\ 0.5 \\ 0.6 \\ 0.1 \end{bmatrix}$$

$$\begin{aligned} y &= \sigma(\mathbf{W}_j \cdot \mathbf{X}) \\ &= \frac{1}{1 + e^{-(\mathbf{W}_j \cdot \mathbf{X})}} \\ &= \frac{1}{1 + e^{-(0.5 + 0.2 \times 0.5 + 0.3 \times 0.6 + 0.9 \times 0.1)}} \\ &= \frac{1}{1 + e^{-(0.87)}} \\ &= 0.7 \end{aligned}$$

En terminos simples, este proceso aplica para cada uno de los casos en las Figuras 2.14 y 2.15.

Una unidad neuronal muy simple es conocida como el perceptron, el cual tiene como salida una respuesta binaria:

$$y = \begin{cases} 0 & \text{si } \mathbf{W}_j \cdot \mathbf{X} \leq 0 \\ 1 & \text{si } \mathbf{W}_j \cdot \mathbf{X} > 0 \end{cases} \quad (2.43)$$

2.3.3. Preprocesamiento de datos

Cuando usamos una función de activación logística las redes neuronales se desempeñan mejor cuando los predictores y la variable objetivo están en el intervalo $[0, 1]$; considerando:

$$X_{normalizada} = \frac{X - a}{b - a} \quad (2.44)$$

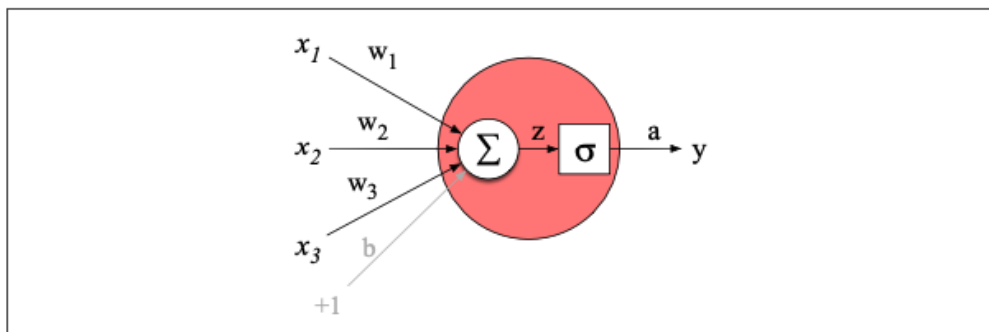


Figure 7.2 A neural unit, taking 3 inputs x_1 , x_2 , and x_3 (and a bias b that we represent as a weight for an input clamped at $+1$) and producing an output y . We include some convenient intermediate variables: the output of the summation, z , and the output of the sigmoid, a . In this case the output of the unit y is the same as a , but in deeper networks we'll reserve y to mean the final output of the entire network, leaving a as the activation of an individual node.

Figura 2.14: Ilustración de una unidad neuronal. Retomado de Jurafsky, et al (2023, Cap. 7, p.3) [JM23]

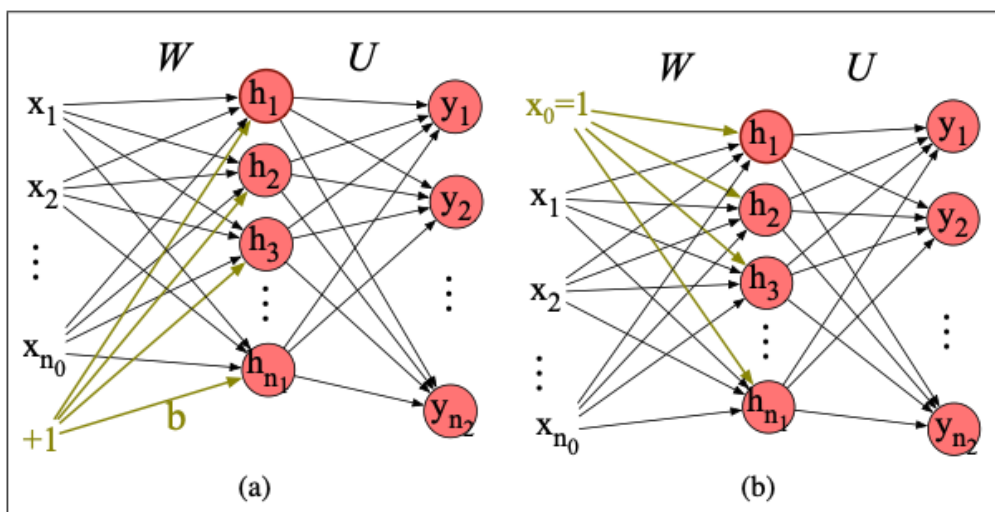


Figure 7.9 Replacing the bias node (shown in a) with x_0 (b).

Figura 2.15: Ilustración de una red neuronal con 1 capa oculta. Retomado de Jurafsky, et al (2023, Cap. 7, p.11) [JM23]

Donde $a = \min(X)$ y $b = \max(X)$.

Adicionalmente, existe una función de normalización de los valores contenidos en los vectores que se forman en cada nodo $Z_1, Z_2, \dots, Z_d$. Esta función es conocida como función Softmax:

$$\text{Softmax}(Z_j) = \frac{e^{z_j}}{\sum_{i=1}^d e^{z_i}} \quad (2.45)$$

Donde $1 \leq i \leq d$.

2.3.4. Entrenamiento del modelo

Entrenar un modelo significa estimar los pesos $\theta_j, w_{1j}, w_{2j}, \dots, w_{pj}$ que llevan al mejor resultado de predicción posible.

El mecanismo seguido para este proceso se le conoce como **back propagation**. Es el método más popular usado para determinar los pesos (aprender). Como su nombre lo indica, los errores son calculados en forma recursiva; iniciando en la última capa (la capa de output) y continuando con la previa hasta llegar a la capa de input.

El proceso de entrenamiento se puede pensar como los siguientes pasos:

1. Establecemos una función de pérdida que modele la distancia entre el output conseguido y el el objetivo. A esa función se le suele llamar **función de entropía cruzada**.
2. Para encontrar los parámetros que minimicen la función de pérdida, usaremos el algoritmo de optimización del gradiente descendiente.
3. Finalmente, implementamos un algoritmo de **backpropagation**.

2.3.5. Ejemplo de Redes Neuronales

El código del ejemplo se encuentra en GitHub con el nombre:

“02_Introduccion_a_Deep_Learning” en GitHub.

Un segundo ejemplo, más aplicado, entrena una red con una capa oculta sobre el conjunto de datos MNIST (clasificación de dígitos escritos a mano a partir de su imagen), lo que permite ver el mismo proceso de propagación hacia adelante y retropropagación de esta sección aplicado a un problema de clasificación multiclase real, con entradas de alta dimensión (cada imagen es un vector de 784 píxeles). El código se encuentra en GitHub con el nombre “03_Redres_Neuronales_Ejemplo_MNIST”.

2.4. Aplicaciones de modelos de lenguaje natural con bases vectoriales (word embeddings).

Los Word Embeddings o Vectores de Embeddings son una representación vectorial del texto. Esta representación es usada en los modelos de lenguaje natural basados en redes neuronales. Los Vectores de Embeddings pueden capturar el significado y contexto de palabras y frases.

Word embeddings:

1. Word embeddings o simplemente embeddings son el resultado de un proceso mediante el cual se utiliza alguna técnica/algoritmo que sea capaz de convertir palabras o texto a vectores de N dimensiones.
2. Estos vectores contienen cierto nivel de información semántica sobre el texto o palabra.
3. Por ejemplo, palabras que son muy similares van a tener valores cercanos en sus representaciones en vectores.
4. Existen diversos modelos capaces de generar el embedding de un texto. En términos generales hay dos familias: (a) embeddings ofrecidos como **servicio vía API** por proveedores comerciales –por ejemplo, el embedding de OpenAI, que es el mismo que utilizan internamente GPT-3 y GPT-4 para representar el texto de entrada– y (b) embeddings **abiertos y locales**, calculados con modelos descargables (típicamente vía la librería **transformers** o **sentence-transformers** de Hugging Face), que no requieren una llave de API ni tienen costo por consulta.
5. En los notebooks de este módulo usamos la segunda familia (modelos abiertos, ejecutados localmente), precisamente para que cualquier estudiante pueda reproducir los ejemplos sin depender de una cuenta de pago. Conceptualmente ambas familias resuelven el mismo problema –convertir texto en un vector que capture su significado–; la diferencia práctica es costo, control sobre el modelo y necesidad de conexión a un servicio externo.

Una forma intuitiva de explorar visualmente cómo estos vectores agrupan palabras semánticamente cercanas es el proyector de Tensorflow: <https://projector.tensorflow.org/>.

Creating Embeddings

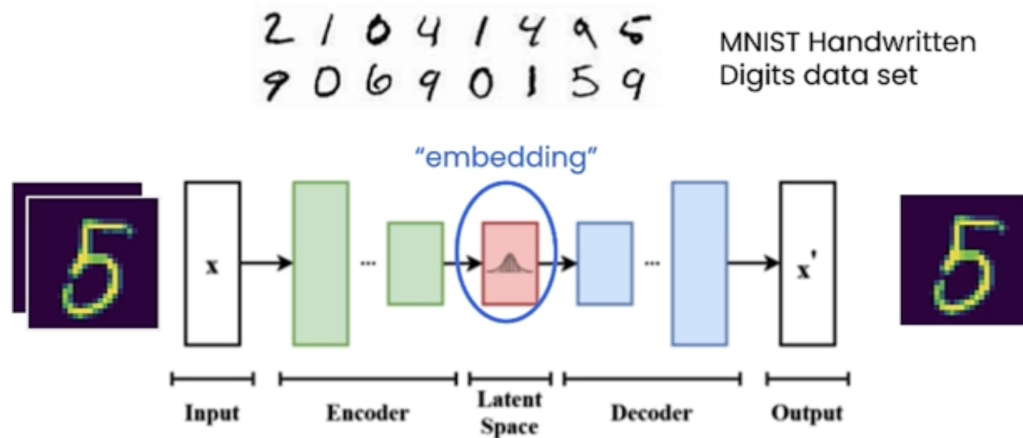


Figura 2.16: Retomado de <https://learn.deeplearning.ai/courses/vector-databases-embeddings-applications/lesson/1/introduction>

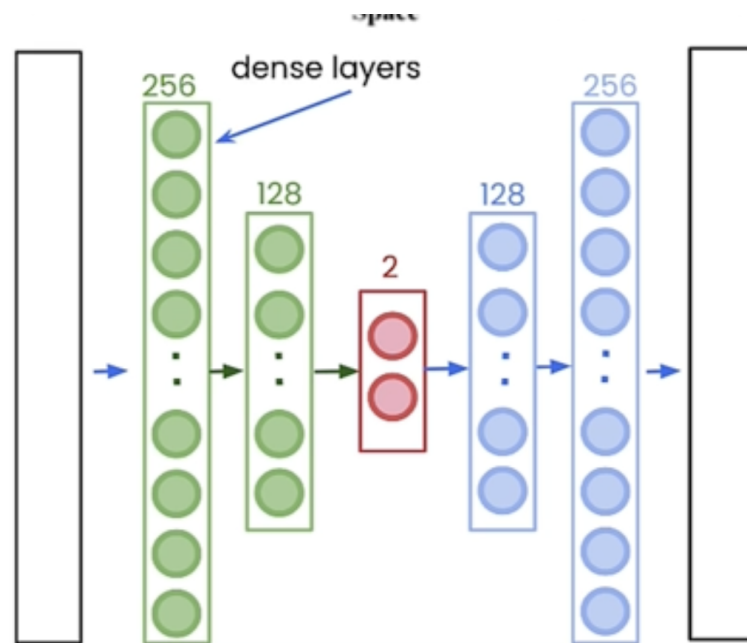


Figura 2.17: Retomado de <https://learn.deeplearning.ai/courses/vector-databases-embeddings-applications/lesson/1/introduction>

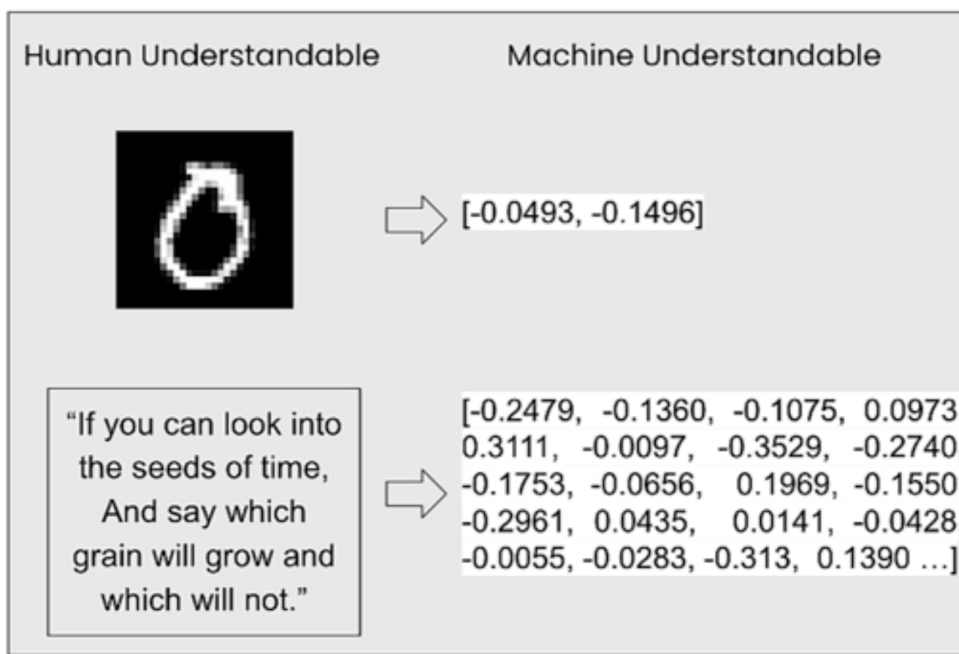


Figura 2.18: Retomado de <https://learn.deeplearning.ai/courses/vector-databases-embeddings-applications/lesson/1/introduction>



Figure 6.1 A two-dimensional (t-SNE) projection of embeddings for some words and phrases, showing that words with similar meanings are nearby in space. The original 60-dimensional embeddings were trained for sentiment analysis. Simplified from [Li et al. \(2015\)](#) with colors added for explanation.

Figura 2.19: Retomado de Jurafsky, et al (2023, Cap. 6, p.6) [JM23]

Las Figuras 2.16, 2.17 y 2.18 ilustran el proceso de principio a fin: un texto (o incluso una imagen, como veremos abajo) se transforma en un vector de N dimensiones mediante un modelo de embedding, y ese vector se puede comparar contra otros vectores para medir qué tan relacionados están dos contenidos entre sí. Las Figuras 2.19 y 2.20 retoman la discusión de Jurafsky y Martin (2023, Cap. 6) [JM23] sobre semántica vectorial: la idea de que el significado de una palabra puede aproximarse por la compañía que mantiene (las palabras con las que suele coaparecer), lo que da lugar a representaciones densas en las que la cercanía geométrica aproxima la cercanía semántica.

2.4.1. Distancias y similitud entre vectores

Una vez que tenemos vectores, necesitamos una forma de medir qué tan parecidos son dos de ellos. En la práctica se usan varias métricas, cada una con propiedades distintas:

- **Distancia euclidiana (L2):** la longitud del segmento más corto entre dos puntos/vectores.
- **Distancia de Manhattan (L1):** la distancia si solo pudiéramos movernos un eje a la vez (la suma de diferencias absolutas por componente).

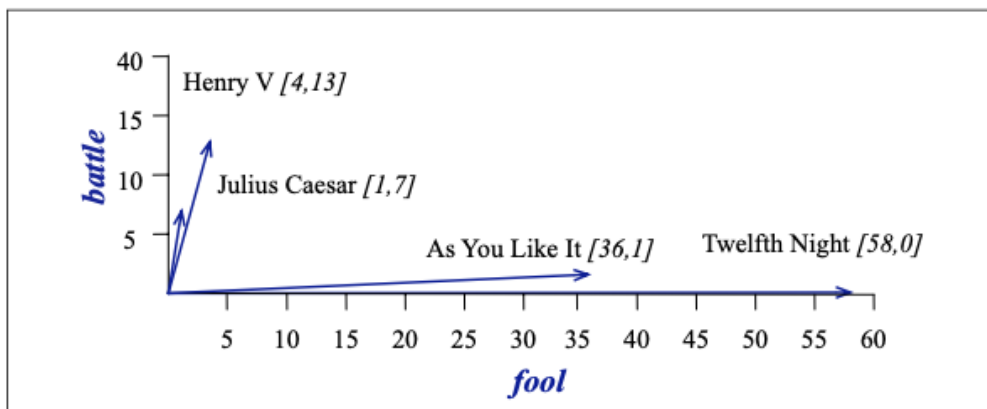


Figure 6.4 A spatial visualization of the document vectors for the four Shakespeare play documents, showing just two of the dimensions, corresponding to the words *battle* and *fool*. The comedies have high values for the *fool* dimension and low values for the *battle* dimension.

Figura 2.20: Retomado de Jurafsky, et al (2023, Cap. 6, p.8) [JM23]

- **Producto punto:** mide la magnitud de la proyección de un vector sobre otro; a diferencia de las anteriores, no es propiamente una distancia (no está acotado) y es sensible a la magnitud de los vectores, no solo a su dirección.
- **Similitud/distancia coseno:** mide la diferencia en *dirección* entre dos vectores, ignorando su magnitud (Figura 2.21). Es, junto con el producto punto, la métrica más usada en PLN para comparar embeddings de oraciones o documentos, precisamente porque lo que nos interesa es si "apuntan hacia el mismo significado", no qué tan largos son.

2.4.2. De palabras a oraciones: embeddings de token vs. de oración

Un mismo texto puede representarse a distintos niveles de granularidad. Un modelo como BERT produce, en su forma más básica, un vector por cada *token* (aproximadamente, cada palabra o subpalabra) –lo que llamamos un **embedding de token**, ilustrado en las Figuras 2.22, 2.23 y 2.24 siguiendo a Jurafsky y Martin (2023, Cap. 7) [JM23]. Sin embargo, muchas tareas (por

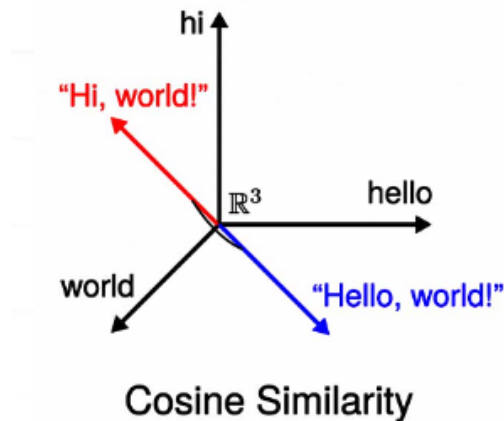


Figura 2.21: Similaridad de Coseno

ejemplo, comparar si dos oraciones dicen "lo mismo") requieren un único vector que resuma *toda* la oración, no uno por palabra.

La forma ingenua de obtener un embedding de oración –promediar los embeddings de sus tokens– funciona, pero pierde información sobre el orden y la interacción entre palabras. Un enfoque más adecuado son los **dual encoders** o modelos tipo SBERT (Sentence-BERT), entrenados específicamente para que la distancia entre dos embeddings de oración refleje qué tan similares son en significado, evaluado típicamente contra juicios de similitud hechos por humanos (una escala de 0 a 5; ver, por ejemplo, la tarea STS de <https://aclanthology.org/S17-2001.pdf>).

2.4.3. De embeddings a recuperación de información: introducción a RAG

Una de las aplicaciones prácticas más importantes de los embeddings de oración es la **Generación Aumentada con Recuperación** (RAG, por sus siglas en inglés, *Retrieval-Augmented Generation*). La idea es sencilla: en lugar de esperar que un LLM "sepa" todo de memoria (lo cual es imposible para información privada, reciente o muy específica de un dominio), (1) convertimos una colección de documentos en embeddings y los guardamos en una base vectorial; (2) cuando llega una pregunta, la convertimos también en un embedding y *recuperamos* los documentos cuyo embedding esté más cerca

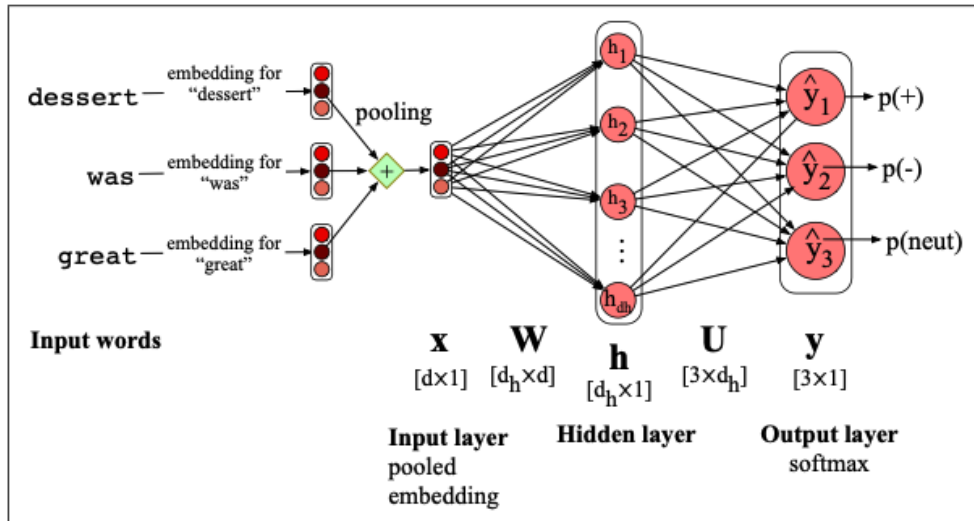


Figure 7.11 Feedforward network sentiment analysis using a pooled embedding of the input words.

Figura 2.22: Retomado de Jurafsky, et al (2023, Cap. 7, p.13) [JM23]

(por ejemplo, con similitud coseno); y (3) le entregamos esos documentos recuperados al LLM como contexto para que *genere* una respuesta fundamentada en ellos, en vez de inventarla. RAG es, en este sentido, el punto de encuentro entre esta sección (embeddings y similitud vectorial) y la siguiente (modelos de lenguaje grandes).

Ejemplos

El código de los ejemplos de esta sección se encuentra en GitHub con los nombres:

“04_Intro_to_Embeddings” (embeddings de imágenes y de oraciones, y comparación de las cuatro métricas de distancia descritas arriba), “05 Token_vs_Sentence_Embeddings” (embeddings de token con BERT vs. embeddings de oración con un dual encoder tipo SBERT) y “06_Using_Embeddings_in_RAG_Inference” (recuperación por similitud pura y con dual encoder, como paso previo a la generación aumentada).

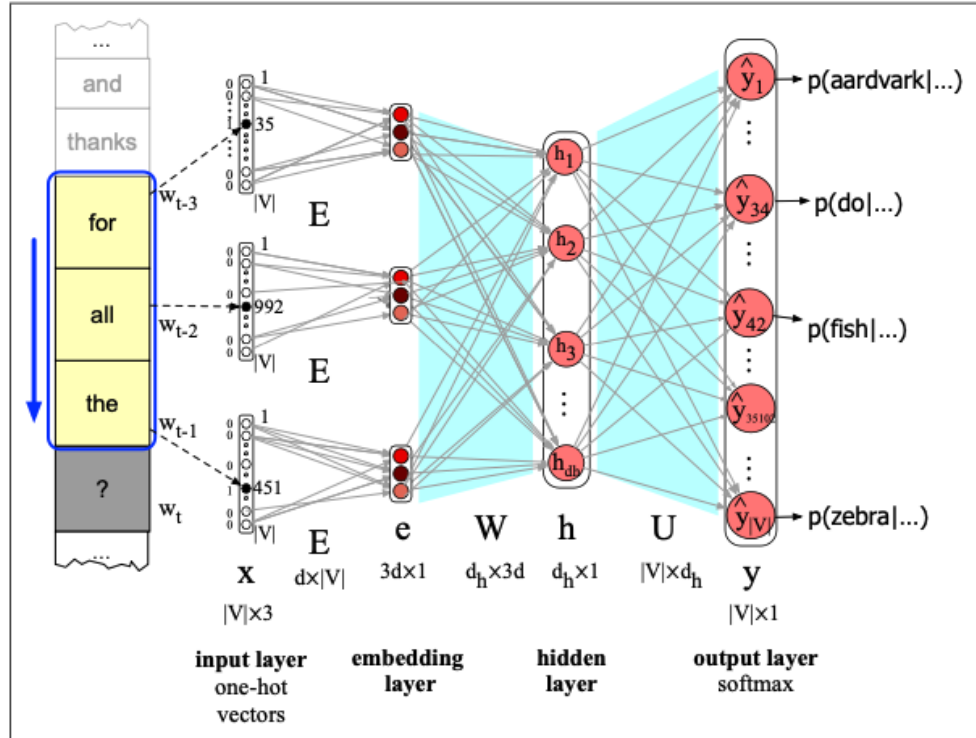


Figure 7.17 Forward inference in a feedforward neural language model. At each timestep t the network computes a d -dimensional embedding for each context word (by multiplying a one-hot vector by the embedding matrix E), and concatenates the 3 resulting embeddings to get the embedding layer e . The embedding vector e is multiplied by a weight matrix W and then an activation function is applied element-wise to produce the hidden layer h , which is then multiplied by another weight matrix U . Finally, a softmax output layer predicts at each node i the probability that the next word w_t will be vocabulary word V_i .

Figura 2.23: Retomado de Jurafsky, et al (2023, Cap. 7, p.23) [JM23]

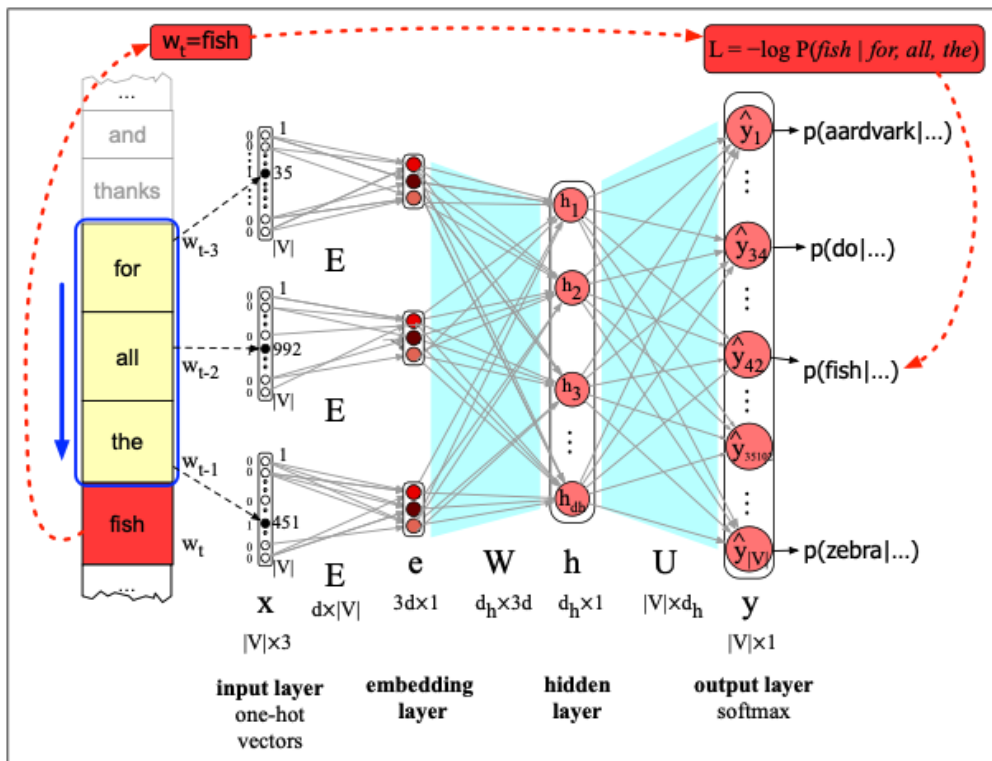


Figure 7.18 Learning all the way back to embeddings. Again, the embedding matrix E is shared among the 3 context words.

Figura 2.24: Retomado de Jurafsky, et al (2023, Cap. 7, p.24) [JM23]

2.5. Introducción a los LLMs y aplicaciones basadas en LLMs (Prompt Engineering).

2.5.1. ¿Cómo funciona un LLM? Breve introducción a la arquitectura Transformer

Antes de hablar de cómo *usar* un LLM (el resto de esta sección), vale la pena entender –aunque sea a nivel conceptual– qué lo hace posible. Los modelos de N-gramas de la sección anterior predicen la siguiente palabra usando solo una ventana fija de $n - 1$ palabras anteriores; esto es simple pero tiene un límite claro: no puede capturar dependencias de largo alcance (por ejemplo, relacionar un pronombre con un sujeto mencionado tres oraciones atrás). Los modelos de lenguaje modernos –incluyendo los LLMs que usaremos en el resto de esta sección– resuelven esto con la arquitectura **Transformer**, propuesta por Vaswani et al. (2017) [Vas+17] en el influyente artículo “*Attention Is All You Need*”.

La idea central: autoatención (self-attention). En vez de procesar el texto palabra por palabra en orden estricto (como hacían las redes recurrentes que dominaban el PLN antes de 2017), un Transformer procesa *toda* la secuencia a la vez, y para cada palabra calcula un promedio ponderado de las representaciones de *todas las demás palabras* de la secuencia –incluida ella misma–, donde los pesos (los *pesos de atención*) se aprenden en función de qué tan relevante es cada palabra para entender a las demás. Esto permite que el modelo relacione directamente dos palabras sin importar qué tan lejos estén una de otra en el texto, y –a diferencia del procesamiento secuencial de una red recurrente– permite calcular esos pesos para toda la secuencia en paralelo, lo que hizo posible entrenar modelos mucho más grandes con el mismo cómputo.

Formalmente, cada palabra (más precisamente, cada *token*, recordando la tokenización que vimos al inicio de este capítulo) se representa con tres vectores aprendidos: una **consulta** (*query*, Q), una **llave** (*key*, K) y un **valor** (*value*, V). La atención de un token hacia los demás se calcula como:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK'}{\sqrt{d_k}} \right) V \quad (2.46)$$

donde QK' mide qué tan similar es la consulta de un token con la llave de cada uno de los demás (un producto punto, como el que vimos al hablar de

similitud de embeddings), d_k es la dimensión de los vectores K (la división solo estabiliza la escala), y el softmax –la misma función que vimos en la sección de redes neuronales– convierte esos puntajes de similitud en pesos que suman 1. El resultado es, para cada token, una nueva representación que mezcla información de todo el contexto, ponderada por relevancia. En la práctica, los Transformers usan **atención multi-cabeza** (*multi-head attention*): varios bloques de atención como el de la ecuación (2.46) corriendo en paralelo, cada uno con sus propios Q, K, V aprendidos, de modo que distintas “cabezas” pueden aprender a capturar distintos tipos de relación (por ejemplo, una cabeza podría especializarse en relaciones sintácticas y otra en correferencia).

Un bloque Transformer completo combina una capa de autoatención con una red neuronal *feedforward* (como las de la sección anterior), y un LLM se construye apilando muchos de estos bloques –de ahí que hablemos de modelos con “muchas capas”, como en deep learning en general. Los LLMs de tipo GPT (*decoder-only*) usan una variante de la ecuación (2.46) llamada **atención causal o enmascarada**: al predecir el siguiente token, cada posición solo puede atender a los tokens *anteriores* a ella, nunca a los futuros –esto es exactamente lo que hace posible el mecanismo de “predecir la siguiente palabra” que describimos abajo para los base LLM. Finalmente, como la autoatención por sí sola no distingue el orden de las palabras (calcula el mismo promedio ponderado sin importar en qué posición esté cada token), los Transformers agregan una **codificación posicional** a cada embedding de entrada, para que el modelo sepa en qué lugar de la secuencia está cada token.

2.5.2. Guideline for Prompt engineering

Introducción (o ¿Qué es el procesamiento del lenguaje natural?)

En los últimos modelos de IA están desbloqueando áreas en las que el texto es abundante. En este momento, el procesamiento del lenguaje natural (PNL o NPL) es una de las áreas más populares de la inteligencia artificial (IA) gracias a aplicaciones como generadores de texto que:

- componen ensayos coherentes
- chatbots que engañan a las personas haciéndoles creer que son sensibles
- programas de conversión de texto a imagen que producen imágenes fotorrealistas

¿Qué es el procesamiento del lenguaje natural?

El procesamiento del lenguaje natural es la disciplina que busca construir máquinas que puedan manipular el lenguaje humano (o datos que se asemejan al lenguaje humano) en la forma en que se escribe, se habla y se organiza.

Evolucionó a partir de la lingüística computacional, que utiliza la informática para comprender los principios del lenguaje, pero en lugar de desarrollar marcos teóricos, el NLP es una disciplina de ingeniería que busca desarrollar tecnología para realizar tareas útiles.

El NLP se puede dividir en dos subcampos superpuestos:

- Comprensión del lenguaje natural, que se centra en el análisis semántico o la determinación del significado previsto del texto, y
- Generación del lenguaje natural, que se centra en la generación de texto por una máquina.

Nota: El NLP está separado del reconocimiento de voz, pero a menudo se usa junto con él, que busca analizar el lenguaje hablado en palabras, convirtiendo el sonido en texto y viceversa (ver, por ejemplo: <https://medium.com/@kbdhunga/summarizing-youtube-videos-using-openai-whisper-gpt-3-5690c9a57b78>).

¿Por qué es importante el NLP?

Porque hoy día los modelos de lenguaje más sofisticados –ya sea de proveedores comerciales (la familia GPT de OpenAI, Claude de Anthropic, Gemini de Google) o modelos abiertos que cualquiera puede descargar y ejecutar (la familia Llama de Meta, que usaremos en los notebooks de agentes de este módulo)– pueden generar prosa coherente sobre una amplia variedad de temas y sostener conversaciones de varios turnos con contexto.

Más allá de los chatbots, el NLP está integrado en productos que usamos todos los días sin notarlo: motores de búsqueda que interpretan la intención detrás de una consulta, sistemas de moderación que detectan discurso de odio en redes sociales, correctores ortográficos, traductores automáticos, y los sistemas de recomendación que deciden qué mostrarte a continuación.

El ritmo de avance del campo es rápido: las capacidades de estos modelos (y sus riesgos) cambian de una generación a otra, por lo que vale más entender los **principios** –qué es un LLM, cómo se entrena, qué puede y no puede

hacer bien— que memorizar las características de un modelo específico, que probablemente ya habrán cambiado para cuando termines este módulo.

¿Para qué se utiliza el procesamiento del lenguaje natural (PLN)?

El PLN no es una sola tarea sino una familia de problemas que comparten la misma materia prima (texto) pero resuelven objetivos distintos. La mayoría de las aplicaciones que veremos en el curso —y las que encontrarás fuera de él— caen en alguna de las siguientes categorías:

- **Análisis de sentimientos:** clasificar la intención emocional de un texto, típicamente en positivo/negativo/neutro. Es el caso de uso que ya vimos formalmente con Naive Bayes y regresión logística en este mismo capítulo (reseñas de películas, tweets).
- **Clasificación de toxicidad:** una rama más específica del análisis de sentimientos, donde el objetivo no es solo detectar intención hostil sino clasificar categorías particulares —amenazas, insultos, obscenidades, discurso de odio hacia grupos específicos— porque cada una puede requerir una respuesta de moderación distinta.
- **Traducción automática:** convertir texto de un idioma a otro. Los sistemas modernos (basados en arquitecturas transformer, que veremos más adelante) superaron a los sistemas basados en reglas y a los primeros sistemas estadísticos precisamente porque aprenden representaciones de significado compartidas entre idiomas, en vez de traducir palabra por palabra.
- **Reconocimiento de entidades nombradas (NER):** extraer menciones de nombres propios, organizaciones, ubicaciones o cantidades dentro de un texto y clasificarlas en categorías predefinidas. Es un paso frecuente antes de otras tareas —por ejemplo, para saber *de qué compañía* habla una noticia antes de analizar su sentimiento hacia ella.
- **Detección de spam:** un problema clásico de clasificación binaria (spam / no spam) que históricamente fue uno de los primeros éxitos comerciales de Naive Bayes aplicado a texto.
- **Corrección gramatical:** detectar y corregir errores de gramática u ortografía, ya sea con reglas explícitas o, en los sistemas modernos,

con modelos de lenguaje que reescriben.^{el} texto hacia la versión más probable.

- **Modelado de temas (*topic modeling*):** una tarea no supervisada que toma un corpus de documentos sin etiquetar y descubre los temas abstractos que lo componen –el equivalente, en texto, a lo que el clustering hace con datos numéricos (ver la sección de agrupamiento del Capítulo 1).
- **Generación de texto (generación de lenguaje natural):** producir texto nuevo, coherente y relevante al contexto. Es la tarea central detrás de los modelos de lenguaje grandes (LLMs) que veremos en la última sección de este capítulo.
- **Recuperación de información:** encontrar, dentro de una colección de documentos, los más relevantes para una consulta dada. Es el problema que enfrenta todo motor de búsqueda o sistema de recomendación, y es también la mitad "recuperación" del esquema RAG que veremos en la sección de embeddings.

Varias de estas tareas no son independientes entre sí: por ejemplo, un sistema de moderación de contenido típico combina NER (para saber a quién se refiere un comentario), clasificación de toxicidad (para decidir si es un ataque) y, cada vez más, un LLM (para decidir qué acción tomar). Vale la pena tenerlas en mente como un mapa de referencia antes de entrar al detalle técnico de los LLMs en la siguiente sección.

2.5.3. Diseño de prompts

Esta sección se enfoca en cómo diseñar instrucciones (*prompts*) efectivas para un LLM ajustado por instrucciones. Los principios que veremos son, en buena medida, agnósticos al proveedor: aplican tanto a modelos comerciales (Claude, de Anthropic; GPT, de OpenAI; Gemini, de Google) como a modelos abiertos que se pueden ejecutar sin depender de un servicio comercial (la familia Llama, que usamos en los notebooks de agentes de este módulo). Cuando sea útil un ejemplo concreto, usaremos Claude como referencia ilustrativa.

Tipos de LLM: modelos base vs. ajustados por instrucción

En el desarrollo de grandes modelos de lenguaje (LLM), ha habido en términos generales dos tipos de LLM: LLM básicos (base LLM) y LLM ajustados por instrucción (instruction-tuned LLM).

1. Un base LLM ha sido entrenado para predecir la siguiente palabra basándose en datos de entrenamiento de texto, a menudo entrenado con una gran cantidad de datos de Internet y otras fuentes para determinar cuál es la siguiente palabra más probable a seguir. Por ejemplo, "había una vez un unicornio", ¿pueden completar esto?, ¿puede predecir la siguiente palabra?, digamos "que vivía en un bosque mágico con todos los amigos unicornios"
2. Un instruction-tuned LLM ha sido capacitado para seguir instrucciones. Por ejemplo, si le preguntará ¿cuál es la capital de Francia?, la respuesta es "la capital de Francia es París". Es decir, está afinado con entradas y salidas que son instrucciones

Entonces, cuando utilicemos un instruction-tuned LLM piensa en darle instrucciones a otra persona, digamos a alguien que sea inteligente pero que no conozca los detalles de su tarea.

Entonces, cuando un LLM no funciona, a veces es porque las instrucciones no fueron lo suficientemente claras.

Parámetros que conviene entender antes de usar un LLM

Independientemente del proveedor, casi todos los LLM instruction-tuned exponen los mismos parámetros de configuración al usarlos vía API (no siempre visibles en una interfaz tipo chat, pero presentes por debajo):

- **Ventana de contexto (*context window*):** el número máximo de tokens (entrada + salida combinadas) que el modelo puede "ver".^{en} una sola conversación. Todo lo que quede fuera de esa ventana –mensajes muy antiguos en una conversación larga, o un documento demasiado extenso– el modelo simplemente no lo tiene disponible al generar su respuesta. Esta es una de las razones por las que RAG (que vimos en la sección de embeddings) es útil: en vez de intentar meter todo un documento en la ventana de contexto, se recupera solo la parte relevante para la pregunta.

Two Types of large language models (LLMs)

Base LLM

Predicts next word, based on text training data

Once upon a time, there was a unicorn that lived in a magical forest with all her unicorn friends

What is the capital of France?
What is France's largest city?
What is France's population?
What is the currency of France?

Instruction Tuned LLM

Tries to follow instructions

Fine-tune on instructions and good attempts at following those instructions.

RLHF: Reinforcement Learning with Human Feedback

Helpful, Honest, Harmless

What is the capital of France?
The capital of France is Paris.

Figura 2.25: LLM

- **Temperatura (*temperature*):** un parámetro que controla qué tan determinista o variable es el muestreo de la siguiente palabra a partir de las probabilidades que calcula el modelo. Con temperatura cercana a 0, el modelo casi siempre elige la palabra más probable (respuestas más consistentes y predecibles, útiles para tareas donde queremos poca variación, como extracción de datos); con temperatura más alta, el modelo muestrea con más aleatoriedad entre palabras plausibles (respuestas más variadas, útiles para tareas creativas, pero también más propensas a desviarse).
- **System prompt / mensaje de sistema:** una instrucción que se le da al modelo por fuera de la conversación visible, para fijar su comportamiento general (tono, rol, reglas que debe seguir) antes de recibir la primera pregunta del usuario. Lo veremos en la práctica en los notebooks de agentes.

Limitaciones a tener presentes

Un LLM –sin importar el proveedor– no es una base de datos ni un buscador: genera texto palabra por palabra según lo que es *probable* dado el contexto, no según lo que es necesariamente *cierto*. Esto tiene consecuencias

prácticas importantes:

- **Alucinaciones:** el modelo puede generar información que suena convincente y está redactada con seguridad, pero que es incorrecta o inventada –por ejemplo, una cita bibliográfica que no existe, o un dato numérico plausible pero falso. Esto es más probable cuanto más se aleje la pregunta de lo que el modelo vio durante su entrenamiento.
- **Corte de conocimiento (*knowledge cutoff*):** un LLM no sabe nada de eventos posteriores a la fecha en la que se recolectaron sus datos de entrenamiento, a menos que se le dé esa información como parte del prompt (por ejemplo, vía RAG) o tenga acceso a una herramienta de búsqueda (ver la sección de agentes, más adelante).
- **Sesgos heredados de los datos de entrenamiento:** como un LLM aprende patrones estadísticos de texto escrito por personas, puede reproducir –y en ocasiones amplificar– sesgos presentes en esos datos (de género, de origen, culturales, etc.). Esto es particularmente relevante para un curso dirigido a ciencias sociales y humanidades: cualquier aplicación de un LLM a un problema con implicaciones sociales debe evaluarse críticamente, no asumir que la respuesta del modelo es neutral por venir de una máquina.

¿Cómo puede ayudarte el uso de un LLM en tu día a día?

Más allá de la teoría, el valor práctico de un LLM ajustado por instrucciones depende de qué tan bien se formule la instrucción (el *prompt*). Algunas prácticas básicas de ingeniería de prompts que resultan útiles en el día a día son: (1) ser específico sobre el formato de salida esperado (tabla, lista, código, extensión máxima); (2) dar contexto o ejemplos de lo que se busca (*few-shot prompting*); (3) pedirle al modelo que razone paso a paso antes de responder cuando la tarea lo requiera (*chain-of-thought*); y (4) iterar –tratar la primera respuesta como un borrador y refinar la instrucción, no como un resultado final e inmodificable.

Sin embargo, un LLM que solo genera texto tiene límites claros: no puede consultar información posterior a su entrenamiento, no puede ejecutar cálculos con garantía de exactitud, y no puede actuar sobre el mundo exterior (leer un archivo, consultar una base de datos, navegar la web). La siguiente sección introduce la idea de **agente**, que es precisamente la respuesta a esta limitación.

2.5.4. Agentes y uso de herramientas (tool use)

Un **agente** es un LLM que, además de generar texto, puede decidir –como parte de su razonamiento– cuándo invocar una **herramienta** (*tool*) externa (una búsqueda web, una calculadora, una consulta a una base de datos, una llamada a una API) para completar una tarea, y luego incorporar el resultado de esa herramienta a su respuesta. La diferencia con un chatbot simple no es solo técnica: es un salto conceptual de “generar texto” a “razonar sobre qué acción tomar”.

Uno de los patrones más influyentes para este tipo de razonamiento es **ReAct** (*Reasoning + Acting*), propuesto por Yao et al. (2022) [Yao+22], en el que el modelo intercala explícitamente pasos de *pensamiento* (razonar sobre qué falta para responder), *acción* (invocar una herramienta) y *observación* (leer el resultado de esa herramienta), repitiendo el ciclo hasta tener evidencia suficiente para responder. Este patrón –pensar, actuar, observar, repetir– es, en esencia, el que replican los frameworks modernos de agentes (como LangChain, usado en los ejemplos de este módulo) cuando deciden si responder directamente o primero consultar una herramienta.

Es importante notar que dar herramientas a un modelo no es gratuito: introduce nuevas fuentes de riesgo que no existen en un chatbot simple:

- **Dependencia de servicios externos:** si la herramienta de búsqueda o la API fallan, el agente completo puede fallar o degradarse.
- **Latencia:** cada invocación de herramienta añade tiempo de espera; un agente con varias herramientas puede tardar considerablemente más que un chatbot simple.
- **Variabilidad:** los resultados de una búsqueda web cambian con el tiempo, por lo que la misma pregunta puede producir respuestas distintas en momentos distintos.
- **Seguridad:** una herramienta mal diseñada puede ser un vector de riesgo real. Por ejemplo, una calculadora que evalúe expresiones matemáticas usando `eval()` directamente sobre texto generado por el modelo (o, peor, sobre texto de un usuario) puede ejecutar código arbitrario; por eso los ejemplos de este módulo implementan la calculadora del agente validando la expresión mediante su árbol de sintaxis (AST) antes de evaluarla, en vez de usar `eval()` directamente.

Ejemplos

El código de los ejemplos se encuentra en GitHub con los nombres “Agente_Basico” (un agente conversacional con memoria, usando LangChain y Llama 3.3 70B vía Groq, todavía sin herramientas) y “Agente_Intermedio” (el mismo agente, ahora con dos herramientas: una búsqueda web y una calculadora segura), ambos en la carpeta “07 Building-a-Chat”. El segundo notebook es el que mejor ilustra el patrón de razonamiento descrito arriba: el modelo decide, pregunta por pregunta, si puede responder directamente o si necesita apoyarse en una herramienta.

Con esto cerramos el recorrido del Módulo V: partimos de la esperanza condicional y la regresión para explicar y predecir a partir de datos estructurados, pasamos por el aprendizaje no supervisado y los modelos categóricos, y terminamos con el análisis de texto y los modelos de lenguaje –hasta llegar a sistemas capaces de decidir, no solo de predecir, cuándo y cómo actuar sobre una tarea.

Bibliografía

- [Aba21] Alberto Abadie. «Using Synthetic Controls: Feasibility, Data Requirements, and Methodological Aspects». En: *Journal of Economic Literature* (2021). DOI: 10.1257/jel.20191450. URL: <https://doi.org/10.1257/jel.20191450>.
- [Ada20] Christopher P. Adams. *Learning Microeconometrics with R*. Boca Raton, FL: Chapman y Hall/CRC, 2020.
- [BC94] Donald J. Berndt y James Clifford. «Using Dynamic Time Warping to Find Patterns in Time Series». En: *Proceedings of the 3rd International Conference on Knowledge Discovery and Data Mining (AAAIWS'94)*. 1994, págs. 359-370.
- [CT05] A. Colin Cameron y Pravin K. Trivedi. *Microeconometrics: Methods and Applications*. Cambridge, UK: Cambridge University Press, 2005.
- [Cha20] Mark Chang. *Artificial Intelligence for Drug Development, Precision Medicine, and Healthcare*. Boca Raton, FL: Chapman y Hall/CRC, 2020.
- [Cun21] Scott Cunningham. *Causal Inference: The Mixtape*. New Haven, CT: Yale University Press, 2021.
- [CB17] Marco Cuturi y Mathieu Blondel. «Soft-DTW: A Differentiable Loss Function for Time-Series». En: *Proceedings of the 34th International Conference on Machine Learning (ICML)* (2017). URL: <https://arxiv.org/abs/1703.01541>.
- [Gre12] William H. Greene. *Econometric Analysis*. Boston, MA: Pearson, 2012.

- [HTF09] Trevor Hastie, Robert Tibshirani y Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2.^a ed. New York, NY: Springer, 2009. DOI: 10.1007/978-0-387-84858-7. URL: <https://doi.org/10.1007/978-0-387-84858-7>.
- [Jam+13] Gareth James et al. *An Introduction to Statistical Learning: With Applications in R*. New York, NY: Springer, 2013. DOI: 10.1007/978-1-4614-7138-7. URL: <https://doi.org/10.1007/978-1-4614-7138-7>.
- [JM23] Dan Jurafsky y James H. Martin. *Speech and Language Processing*. Online manuscript. 2023. URL: <https://web.stanford.edu/~jurafsky/slp3/>.
- [KR90] Leonard Kaufman y Peter J. Rousseeuw. *Finding Groups in Data: An Introduction to Cluster Analysis*. New York, NY: Wiley, 1990. DOI: 10.1002/9780470316801.
- [Kot+14] Dimitrios Kotsakos et al. «Time-Series Data Clustering». En: *Data Clustering: Algorithms and Applications*. Ed. por Charu C. Aggarwal y Chandan K. Reddy. Boca Raton, FL: Chapman y Hall/CRC, 2014, págs. 357-380.
- [LM12] Richard J. Larsen y Morris L. Marx. *An Introduction to Mathematical Statistics and Its Applications*. Boston, MA: Pearson, 2012.
- [MP43] Warren S. McCulloch y Walter Pitts. «A Logical Calculus of the Ideas Immanent in Nervous Activity». En: *Bulletin of Mathematical Biophysics* 5 (1943), págs. 115-133. DOI: 10.1007/BF02478259.
- [Ng23] Andrew Ng. *CS229 Lecture Notes*. Stanford University. 2023. URL: <https://cs229.stanford.edu/>.
- [Shm+19] Galit Shmueli et al. *Data Mining for Business Analytics: Concepts, Techniques and Applications in Python*. Hoboken, NJ: Wiley, 2019.
- [Vas+17] Ashish Vaswani et al. «Attention Is All You Need». En: *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*. 2017. URL: <https://arxiv.org/abs/1706.03762>.
- [Woo10] Jeffrey M. Wooldridge. *Econometric Analysis of Cross Section and Panel Data*. 2.^a ed. Cambridge, MA: The MIT Press, 2010.

[Yao+22] Shunyu Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629. 2022. URL: <https://arxiv.org/abs/2210.03629>.